

GITHUB COPILOT

FOR DEVELOPERS



LOGISTICS



Class Hours:

- Instructor will set class start and end times.
- There will be regular breaks in class.



Telecommunication:

- Turn off or set electronic devices to silent (not vibrate)
- Reading or attending to devices can be distracting to other students
- Try to delay until breaks or after class

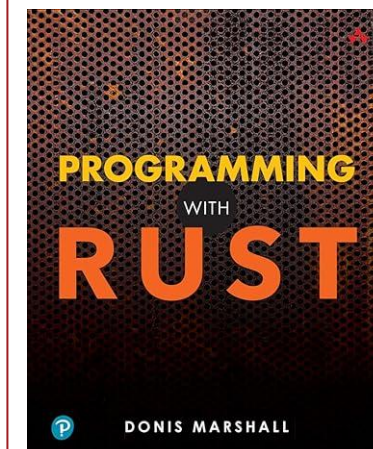
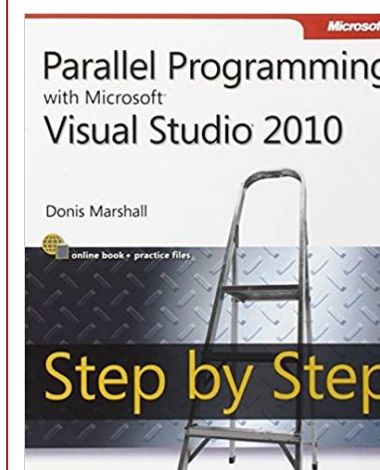
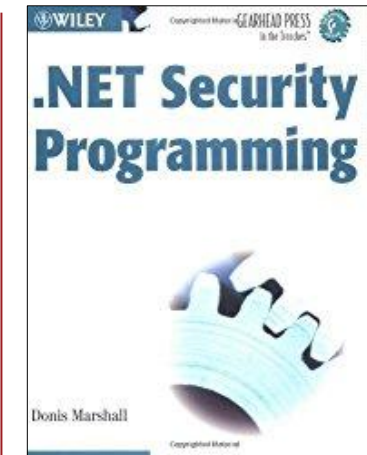
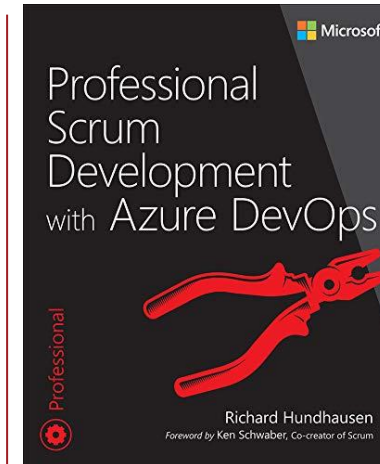
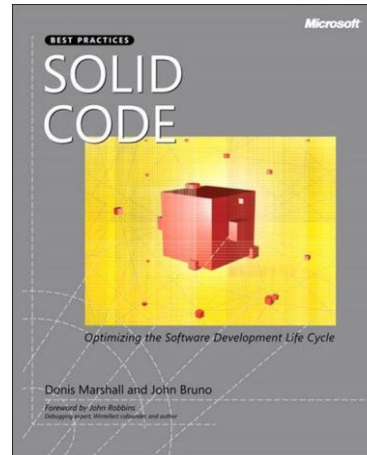
Miscellaneous:

- Courseware
- Bathroom
- Fire drills

DONIS MARSHALL

Security Professional
Microsoft MVP
Microsoft Certified
Author

dmarshall@innovationinsoftware.com



INTRODUCE YOURSELF

Time to introduce yourself:

- Name
- What is your role in the organization
- Indicate GitHub Copilot and VS Code experience



COPILOT



In technology, the term copilot refers to a digital assistant that augments a user's capabilities by providing real-time guidance, automation, or suggestions.

It doesn't replace human judgment but instead supplements it, offering context-aware insights and actionable recommendations.

Just as a copilot in aviation supports the pilot with navigation and decision-making, a technology copilot helps professionals carry out complex tasks more efficiently.

GITHUB COPILOT – 2



GitHub Copilot

GitHub Copilot is an AI-powered coding assistant that integrates into development environments such as Visual Studio Code (VS Code). It provides real-time code suggestions, entire functions, or even more as a developer types.

Copilot's strength lies in its ability to understand the context of the current codebase and produce recommendations that feel relevant, saving developers time and effort.

POP QUIZ: LINES OF CODE

How many lines of code can the average developer enter before creating a logical or syntactical error?



10 MINUTES

AI ASSISTANT



AI assistants in tools like VS Code function much like pair programming, where two developers work together at one computer. In traditional pair programming, one person writes the code while the other reviews, suggests improvements, and helps catch mistakes.

With an AI assistant, the “second developer” is virtual—it suggests code, completes functions, helps with syntax, and offers ideas based on patterns it has learned.

AI ASSISTANT - 2



However, just like working with a junior teammate, the senior developer (you) remains in charge. The AI may not understand the project's goals, constraints, or business logic—it predicts likely code based on data. This means you must review, validate, and decide whether its suggestions are correct.

WHO'S DRIVING?



Degradation in Spatial Knowledge Acquisition When Using Automatic Navigation Systems

https://link.springer.com/chapter/10.1007/978-3-540-74788-8_15

GITHUB AND OPENAI



GitHub Copilot emerged in 2021 from a collaboration between GitHub and OpenAI, combining GitHub's massive code base with OpenAI's language model expertise. Built on the Codex model, a GPT-3 descendant fine-tuned for coding - it was designed for programming tasks.

The partnership was strategic: GitHub provided billions of code examples, and OpenAI supplied generative AI research, becoming one of the first large-scale commercial uses of generative AI in software development.

GENERATIVE AI

Generative AI is a type of artificial intelligence that doesn't just analyze data, but creates new things—like text, images, music, or code—based on patterns it has learned. Instead of giving fixed answers, it generates original output that looks like it was made by a person.

Analogy: Generative AI is like an incredibly fast assistant who has absorbed knowledge from a vast library—and can go beyond simply recalling what it has read by combining and extending ideas in new ways. You describe what you need, and it generates suggestions—whether an email, code, or concepts—based on that input, saving you time and effort.

GENERATIVE AI - 2

Generative AI in GitHub Copilot works by learning from patterns found in millions of code repositories and other sources, then creating new code best fits your prompt. It doesn't just simply replace existing code but predicts what might come next, even if new and original, based on context.

Simple example: Suppose you start typing in Python:

```
def add_numbers(a, b):
```

Then pause and wait momentarily.

GENERATIVE AI - 3

Copilot might automatically suggest:

```
return a + b
```

It “predicted” that since you named the function `add_numbers` with two inputs, you probably want to return their sum. That’s generative AI at work—it takes what you’ve written, combines it with what it has learned previously, and generates new code for you.

POP QUIZ: GITHUB COPILOT

For code completion or entering an explicit prompt, why would you pause momentarily?



10 MINUTES

LLM



An LLM (Large Language Model) is like a giant library that reside in the AI's brain. This translates into the AI understanding billions of white paper, books, websites, and code, so it recognizes many patterns. Because it's so large, it can generate flexible, detailed answers.

An SLM (Small Language Model) is more like a pocket guide. It's trained on less data and has fewer "pages" in memory, so it focuses on smaller, specific tasks. It runs faster and uses less computing power, but it won't know as much or be as creative.

LLM - SOURCES

Many assume GitHub Copilot is trained solely on GitHub repositories, but the reality is broader. Sources include:

- Public GitHub repositories
- Open-source projects hosted outside GitHub
- Programming Q&A sites
- Technical documentation and manuals
- Educational resources such as tutorials and coding books

This variety ensures Copilot can generalize across multiple programming domains and coding conventions, rather than being biased toward a single platform or community.

LLM – STALENESS



The LLM's training data is not continuously updated. For that reason, it may not reflect real-time changes. This means it may recommend outdated functions, deprecated APIs, or older coding patterns.

To address this, developers should cross-check suggestions with up-to-date references such as official documentation, vendor resources, or active community discussions. Validating against current sources helps ensure projects remain aligned with modern standards and best practice

CODE REVIEW

Large Language Models (LLMs) are powerful assistants, but their outputs require human review. They can introduce errors or risks that developers must identify and correct.

Here are some reasons:

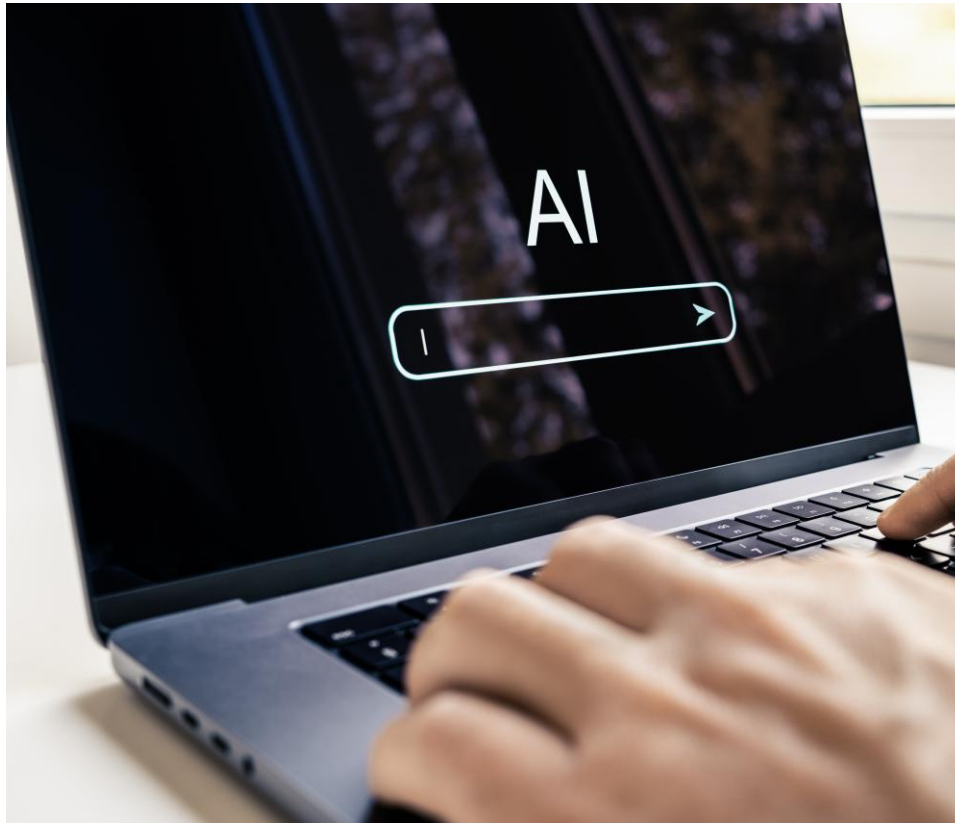
- Hallucinations – The model may generate code that looks correct but is functionally wrong or references non-existent libraries or APIs.
- Bias – Training data may contain biased assumptions that influence logic, naming, or implementation choices.

CODE REVIEW - 2

- Staleness – Training data is not continuously updated, which can result in outdated functions, deprecated APIs, or older coding patterns.
- Security Risks – The model may suggest insecure patterns, weak validation, improper error handling, or weak security controls.

Careful human review ensures correctness, security, maintainability, and alignment with current standards.

CODE PILOT CHAT



Copilot Chat extends the traditional completion model by enabling natural language conversations directly within the coding environment. Instead of just typing code and receiving suggestions, developers can ask Copilot questions like, “How do I implement OAuth2 authentication?” or “Explain what this function does.” This conversational interface makes problem-solving more intuitive and context-driven.

The importance of Copilot Chat lies in its ability to bridge the gap between coding knowledge and documentation.

USE CASES

Boilerplate and Scaffolding

Copilot accelerates development by generating repetitive setup code such as class definitions, configuration files, or API route handlers. For instance, typing `# create a Flask route for /hello` can instantly produce a functioning endpoint without you writing the boilerplate manually.

Code Completion and Suggestions

It provides context-aware completions for functions, loops, and conditionals. Start typing `for i in range(` in Python, and Copilot may suggest the entire loop with a sensible body. This allows developers to move faster and maintain momentum.

Error Handling and Validation

Copilot generates robust error-handling blocks, helping developers avoid common pitfalls. For example, in JavaScript, adding a comment `// handle exceptions` can prompt Copilot to create a `try { ... } catch (error) { ... }` structure.

USE CASES - 2

Learning and Language Assistance

Developers working with unfamiliar frameworks or languages can rely on Copilot to propose idiomatic patterns. Typing *# connect to MongoDB* may yield a correct connection snippet in Node.js, Java, or Python.

Test-Driven Development (TDD)

Copilot is excellent when writing tests first. Developers can describe intended behavior in comments or partial test code, and Copilot generates full unit tests or assertions.

USE CASES - 3

Finding Logic Errors and Reviewing Code

Copilot is not just a code generator; it can help detect subtle logic mistakes by suggesting corrections or alternative implementations. For example, if a loop doesn't terminate properly or a condition is inverted, Copilot often proposes the correct fix. It can also suggest more efficient or readable versions of code, acting like a lightweight peer review.

Source Control Assistance

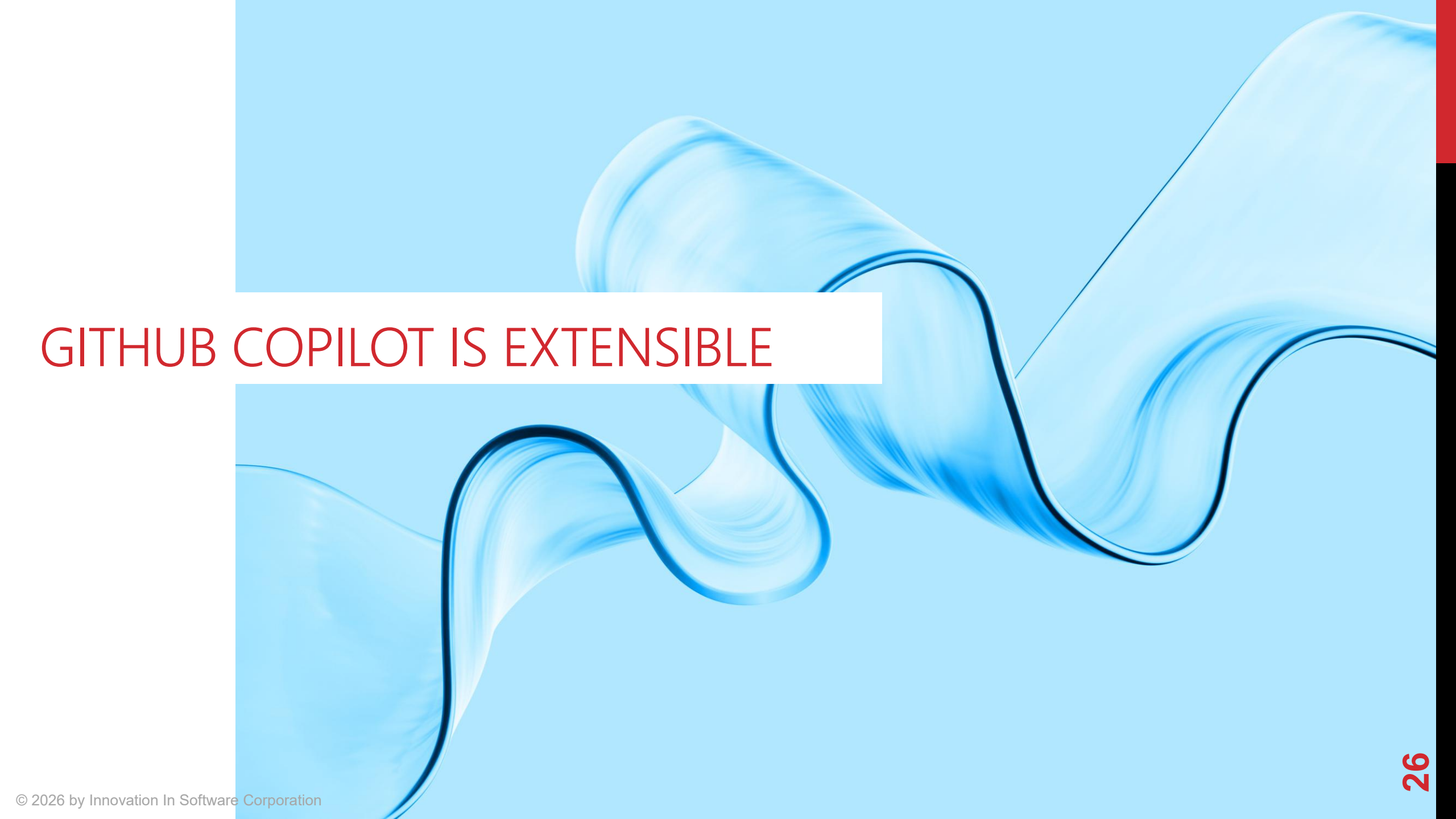
Copilot integrates well with GitHub workflows, helping draft commit messages, suggesting summaries for pull requests, or scaffolding git commands in scripts.

POP QUIZ: USE CASES

What other use cases, if any, should be mentioned?



10 MINUTES

The background of the slide is a light blue color with a large, abstract, wavy graphic element in a slightly darker shade of blue. This graphic element flows from the top right towards the bottom left, creating a sense of movement. A white rectangular box is positioned horizontally across the middle of the slide, containing the main title text.

GITHUB COPILOT IS EXTENSIBLE

LLM MODELS

GitHub has expanded Copilot beyond just OpenAI’s GPT family. It now also supports models from Google (Gemini) and Anthropic (Claude).

LLM Family (Examples in Copilot)	Provider	Typical Use in GitHub Copilot
GPT-4.1, GPT-4o; GPT-5 mini; GPT-5.1; GPT-5.2; GPT-5.1-Codex; GPT-5.2-Codex; GPT-5.3-Codex	OpenAI	General-purpose coding and debugging; Codex variants optimized for code-centric workflows and agentic development tasks; higher models often used for deeper reasoning and larger code changes
Claude Haiku 4.5; Claude Sonnet 4 / 4.5 / 4.6; Claude Opus 4.5 / 4.6	Anthropic	Haiku = fast lightweight help; Sonnet = balanced daily coding assistance; Opus = deeper reasoning, architecture analysis, and complex debugging
Gemini 2.5 Pro; Gemini 3 Flash; Gemini 3 / 3.1 Pro	Google	Large-context analysis, repository-wide refactoring, documentation generation, and research-style development workflows; Flash optimized for fast lightweight tasks
Grok Code Fast 1	xAI	Speed-focused coding assistance and quick interactive completions within IDE environments
Raptor mini	Microsoft / GitHub	Lightweight model option used for faster responses and lower-cost tasks

LANGUAGES SUPPORTED

GitHub Copilot supports a wide range of programming languages. This is the short list:

Language	Language
Python	JavaScript / TypeScript
Java	C#
Go	Ruby
PHP	C++
Rust	SQL
Shell scripting (Bash, PowerShell)	—

IDE SUPPORTED

Copilot currently integrates with:

- Visual Studio Code
- Visual Studio (for .NET developers)
- Neovim
- JetBrains IDEs (IntelliJ IDEA, PyCharm, WebStorm, etc.)
- Eclipse

Note: Level of integration and support may vary. Be sure to reference the product documentation.

VISUAL STUDIO CODE



Visual Studio Code (VS Code) is a lightweight yet powerful editor developed by Microsoft. It supports many programming languages and can be extended with thousands of extensions. Core features include IntelliSense code completion, built-in debugging, and Git integration.

A major strength of VS Code is its close integration with GitHub and GitHub Copilot. Developers can commit, push, and review code directly in the editor while Copilot provides AI-powered suggestions inline.

GITHUB ACCOUNT

A GitHub account is required to use GitHub Copilot.

- Subscription management: Copilot is a paid service (with free trials for students and certain accounts). GitHub needs your account to handle billing or free access eligibility.
- Authentication: You sign in with your GitHub account in your IDE (VS Code, JetBrains, Visual Studio) to connect to Copilot's cloud models.
- Settings & preferences: Your GitHub account stores Copilot settings, like language/file-type controls, telemetry, and whether Copilot Chat is enabled.
- Enterprise access: For business/enterprise users, Copilot is tied to an organization's GitHub account and licensing.

GITHUB LICENSE

GitHub Copilot is a subscription service linked to your GitHub account.

Plan	Intended For	Key Features
Copilot Free	Developers exploring Copilot	Limited usage: ~2,000 code completions/month and ~50 premium AI requests; access to core models such as GPT-4.1 and Claude Haiku
Copilot Pro	Individual developers	Full IDE completions, Copilot Chat, model selection, higher usage limits
Copilot Pro+	Power users / advanced developers	Higher premium request limits, access to advanced models and AI agents
Copilot Business	Teams & organizations	Includes Pro features plus organization policy controls, license management, and security protections
Copilot Enterprise	Large organizations using GitHub Enterprise Cloud	Includes Business features plus enterprise-wide context, advanced governance, and deeper integration with enterprise repositories and policies



CHATGPT



ChatGPT can be used similar to GitHub Copilot — both rely on large language models to generate context-aware code and explanations. *ChatGPT is limited to OpenAI LLMs.*

While Copilot is tightly integrated into the coding workflow within IDEs, such as VS Code,

ChatGPT excels at broader tasks such as debugging discussions, architectural guidance, and natural language exploration of problems.

CHATGPT - 2

Aspect	Copilot	ChatGPT
Integration	Built into IDEs (VS Code, JetBrains).	Web/app chat; extensions exist.
Focus	Inline code completion.	Broader Q&A and reasoning.
Context	Uses nearby code (FIM).	Uses conversation history.
Ease	Fast, automatic as you type.	Manual prompts, copy-paste code.
Strengths	Scaffolding, TDD, fixes.	Explaining, design, reviews.
Pros	Seamless coding flow.	Detailed answers, flexible.
Cons	Limited to coding tasks.	Less integrated with IDEs.

COURSE AGENDA

1. Introduction to GitHub Copilot
2. Copilot setup and configuration
3. GitHub Copilot modes
4. Basic code completion
5. Context-aware code suggestions
6. Debugging with Copilot
7. Writing functions and modular code
8. Code documentation
9. Collaboration with Copilot
10. Agentic AI
11. Introduction to testing
12. Advanced Concepts (customization)

LABS

Learning is better when hands-on. Some of the modules have a companion lab reinforcing a kinesthetic learning experience.

- Labs review and reinforce important concepts
- Don't be surprised - many of the labs extend the topics introduced in the module
- The labs offer an opportunity for real-world experience
- Pair programming can be effective when working on the labs

YOUR CLASS!

Yes, this is your class. What does this mean? You determine the value.

- What is the most important ingredient of class – your participation!
- Your feedback and questions are always welcomed.
- There is no protocol in class. Speak up anytime!
- We welcome your comments during and after class. Just email dmarshall@innovationinsoftware.com.

CONFIRMATION

A study by IT Revolution, surveying over 5,000 developers across multiple Fortune 100 companies, reported measurable productivity improvements with AI coding assistants.

Key Results:

- +26% Task Completion Rate
More tasks completed, faster delivery.
- +13.5% Weekly Code Commits
More output, quicker prototyping and testing.
- +38.4% More Code Compilations
Faster iterations and feedback cycles.

LAB 1 – SETUP



ASSIGNING VIRTUAL MACHINES (VM)



Your trainer will assign virtual machines with credentials to log in. Please note the credentials. You will be assigned the same virtual machine throughout the class.

Your virtual machine has public access to the internet.

Important: Your virtual machine will be destroyed immediately after class.

ENVIRONMENT SETUP

In VS Code, verify that the GitHub Copilot and GitHub Copilot Chat extensions are installed (Ctrl + Shift + X).

Match your current status to the instructions below then follow the action.

Status	Action Required
Scenario 1: GitHub account + GitHub Copilot license.	Open VS Code. Click the Accounts icon and sign-in with your account.
Scenario 2: GitHub account but no GitHub Copilot license.	Open VS Code. Click the Copilot menu, on status bar at the right side, and signup for a free Copilot account.
Scenario 3: No account, no license.	1. Create a personal account at GitHub.com. 2. Open VS Code. Click the Copilot menu, on status bar at the right side, and signup for a free Copilot account.

GITHUB COPILOT

Goal: Confirm GitHub Copilot is operational in VS Code.

In this portion of the lab, you will:

- Open or create a working folder
- Create a simple Python file
- Trigger a Copilot suggestion
- Accept the suggestion

OPEN OR CREATE THE LAB FOLDER

In VS Code

- Select File → Open Folder
- Navigate to your sample files
- Find Lab1 folder, select it, and click Open
- Alternatively, create a new folder named Lab1, then open it

CREATE THE PYTHON FILE

In the Explorer panel, right-click inside the lab1 folder

- Select New File
- Name the file:

first.py

- Press Enter

TRIGGER A COPILOT SUGGESTION

Inside first.py, type:

- `#create an add function`

Press Enter

- A Copilot suggestion should appear in faded text.
- Press Tab to accept the suggestion.

VERIFICATION

Copilot is operational if:

- A function was generated automatically
- The suggestion appeared without manual prompting
- You accepted it using Tab

Lab completed

“Develop a passion for learning.”



FUNDAMENTALS

DETAILS AND WORKFLOW



INTRODUCTION



This module outlines GitHub Copilot as a cloud-based AI coding assistant. It explains how prompts are automatically enriched with contextual information from files, functions, and the surrounding codebase, enabling intelligent code generation, including fill-in-the-middle when supported.

The workflow covers prompt initiation, context collection, secure cloud-based inference, and the delivery of results as inline code suggestions or chat-based responses. The module also addresses key considerations related to privacy, licensing, and security.

CLOUD-BASED GENERATIVE AI TOOL

GitHub Copilot is not just a local IDE extension—it is a cloud-based service that relies on generative AI models hosted and maintained by GitHub and Microsoft. Every time a developer enters a prompt or writes code, that context is securely transmitted to cloud servers where the model processes it and returns suggestions.

Being cloud-based ensures scalability and continuous improvement. The models can be updated, retrained, and fine-tuned without requiring developers to manually install updates.

PROMPT

Prompts include more than what you type.

When you type a comment or a partial line of code, that input is only part of the prompt. GitHub Copilot automatically enriches it with additional context from the surrounding code, relevant files in the repository, and the project's current structure. As a result, the effective prompt is much larger than the text you typed, leading to more accurate and context-aware suggestions.

This is why Copilot can sometimes anticipate what you want several lines ahead—it is interpreting not just your last keystrokes but the broader context.

CONTEXT



Context is critical to how GitHub Copilot works. The tool doesn't just generate random code—it looks at the file you're writing, the function names, other files you have open, etc. This context tells GitHub Copilot what you're trying to accomplish, so its suggestions are relevant. Without context, its output would be generic and often wrong.

For example, if you write a comment saying *# create calculate function to calculate sales tax*, Copilot uses that clue along with your code structure to suggest a tax calculation function instead of something unrelated.

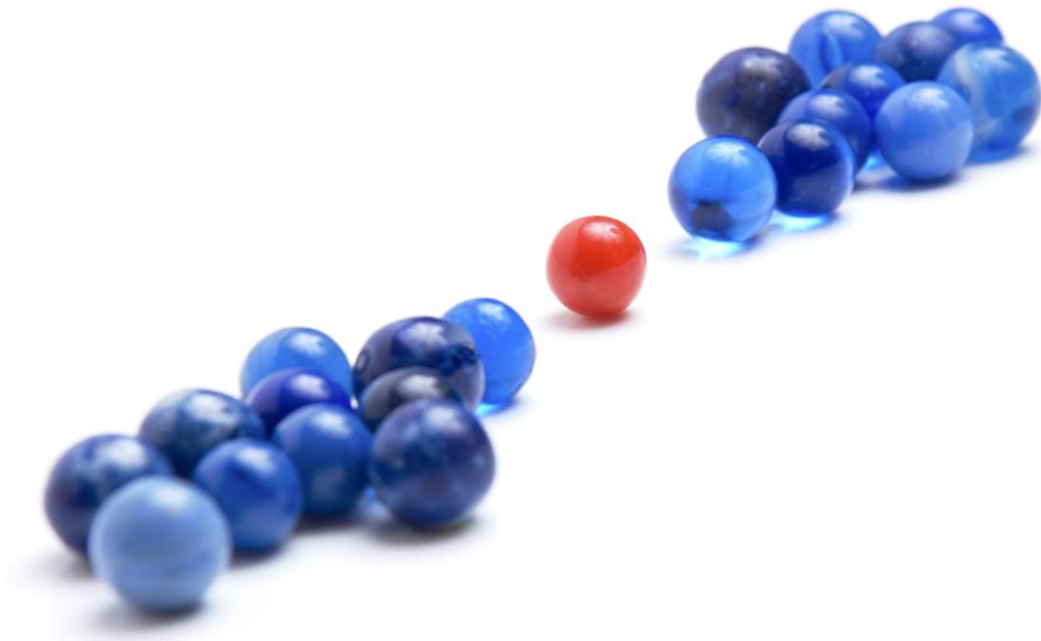
CONTEXT - 2



Copilot gathers context from multiple sources, some more influential than others.

- Function and variable names
- Source code
- Comments
- Open files
- Project-level context
- File name and file type
- Coding conventions

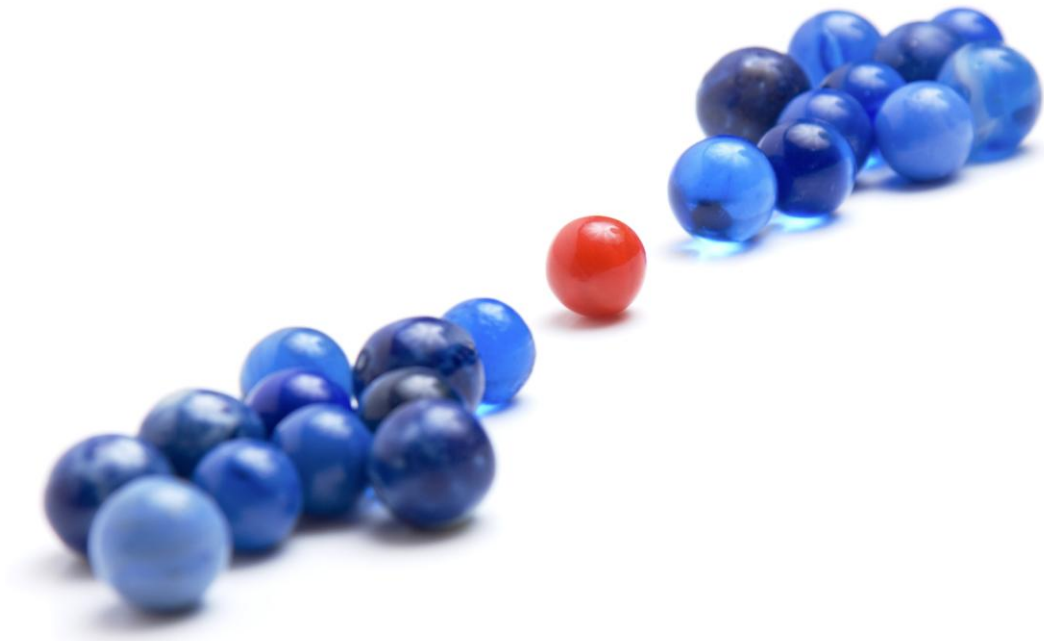
FILL-IN-THE-MIDDLE



GitHub Copilot can generate code using context from both before and after the cursor position in the source file. This behavior, referred to as Fill-in-the-Middle (FIM), allows Copilot to insert code anywhere rather than only appending new code. Copilot analyzes nearby code, comments, and structure to produce suggestions.

For example, when a function includes a docstring or comments describing its intended behavior, Copilot can generate a function body that reflects that intent while remaining consistent with the code that precedes and follows the cursor.

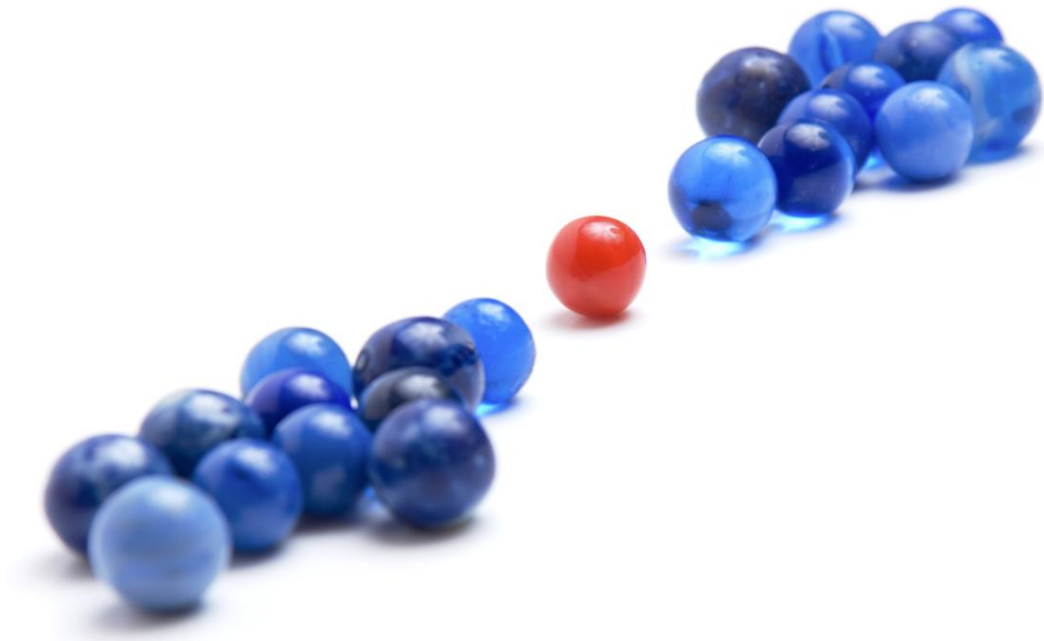
FILL-IN-THE-MIDDLE - 2



FIM is particularly powerful for scaffolding code, refactoring, or when you're editing large files. Instead of writing code strictly top-to-bottom, you can leave placeholders or comments in the middle of a source file and let Copilot intelligently fill them in.

This encourages a workflow where you guide Copilot with intent signals (comments, partial code, or function headers), and it provides the in-between logic.

FILL-IN-THE-MIDDLE - 3



Fill-in-the-Middle (FIM) may not be available depending on the LLM. When FIM is not supported, such as GPT-2 and some LLaMa releases, Copilot falls back to standard left-to-right code completion, meaning contextually suggestions are appended after the cursor rather than inserted between existing code.

FIM EXAMPLE

Here is a FIM example:

```
def calculateDiscount(price, discount):  
    # apply discount  
    —  
    return discountedPrice
```

For context, both are used as context:

- The code before the cursor (the comment and function parameters)
- The code after the cursor (return price;)

COPILOT WORKFLOW – STEP 1 - CREATE A PROMPT

A prompt can be:

- Code you begin typing
- A function name or variable name
- Even a natural language question inside Copilot Chat

Copilot uses this as the main clue for what to generate next.

STEP 2 – COPILOT COLLECTS CONTEXT

Before sending anything to the cloud, Copilot gathers context from your environment:

- The current file
- Other open project files in your editor
- Comments
- The project's structure and imported libraries
- File names and file type (Python, JavaScript, HTML, etc.)

This helps Copilot understand not just what you typed, but but the context.

STEP 3 AND STEP 4

Step 3: Prompt + context sent securely

The prompt and gathered context are encrypted and sent to a GitHub Copilot server. This ensures that your data is transmitted securely, never in plain text.

Step 4: AI model generates results

On GitHub's side, one of the supported LLMs (such as GPT-5, GPT-4.1, Gemini, or Claude, depending on your settings) processes the input.

- The model generates one or more code suggestions.
- Optional post-processing may occur, such as basic vulnerability checks or formatting adjustments, before sending results back.

STEP 5 – RESULTS RETURNED TO IDE

The generated suggestion flows back into your IDE (e.g., Visual Studio Code, JetBrains). You'll see it as:

- An inline completion (gray "ghost text")
- A list of alternative suggestions
- A full explanation or snippet in Copilot Chat

You can accept, modify, or reject the suggestions as desired.

LICENSED CONTENT



Another concern is that Copilot's training data may include code under restrictive licenses. While GitHub asserts that Copilot generates new content rather than copying directly, there is debate over whether certain outputs might inadvertently resemble copyrighted or unlicensed code.

Developers should be aware of these risks and watch for potential license issues. GitHub Copilot provides an option to block suggestions that match verbatim public code.

PRIVACY



Privacy is a major consideration when using AI-assisted coding tools. Copilot may collect snippets of your code to provide context, raising concerns about sensitive or proprietary data. GitHub provides an option in settings to disable code retention for training purposes, giving organizations more control over their intellectual property.

You should configure these options carefully to balance functionality with privacy requirements.

PRIVACY - 2



User engagement—such as whether you accept, modify, or reject suggestions—feeds back into the learning process for Copilot. This helps the system refine its outputs over time. It is important to note that feedback is aggregated and anonymized.

This engagement loop ensures that Copilot becomes more useful the more it is used.

SECURITY

The prompt and code should never include sensitive data such as passwords, API keys, or confidential business logic. Even though Copilot transmits information securely, providing such details still increases the chance of exposure or accidental misuse. Developers should treat all prompts and context as if they may be seen outside the immediate coding session.

AI-generated code can also introduce security vulnerabilities, including insecure SQL queries without parameterization, weak authentication flows, or outdated cryptographic functions. To help, GitHub has added post-processing checks like vulnerability detection and filtering for unsafe code patterns.

These safeguards are limited. The primary responsibility for security remains with the development team, who must carefully review every suggestion, apply secure coding practices, and ensure that GitHub Copilot is used as a supportive tool rather than a replacement for sound engineering judgment.

SCOPE OF SUGGESTIONS

This is about where and what Copilot considers as input:

File scope

Copilot looks only at the current file and nearby lines. Suggestions are local and good for finishing code, but unaware of wider project patterns.

Project scope

Copilot uses context from across the workspace, including other files and shared patterns, producing code that better fits the overall architecture.

SCOPE OF SUGGESTIONS - 2

Prompt scope:

Comments or function names you type strongly shape suggestions.

Example: typing `# function to calculate factorial` will cause a factorial implementation suggestion.

Language scope:

GitHub Copilot supports many languages, but scope can be limited by file type. You can also enable or disable suggestions by language in the GitHub Copilot settings.



SETTINGS / CONFIGURATION

PRIVACY

Privacy panel in GitHub Copilot settings controls how your code and interactions with Copilot are handled.

Suggestions matching public code: You can choose whether Copilot is allowed to show you completions that are similar to existing public code on GitHub. Blocking this reduces the chance of receiving verbatim code from public repositories.

Allow GitHub to use my data for product improvements: If enabled, GitHub may use your prompts, suggestions, and snippets to analyze how Copilot is used, fix issues, and improve features.

Allow GitHub to use my data for AI model training: If enabled, your prompts and snippets could also be used to further train GitHub's AI models. With it disabled, your data won't be included in training.

PRIVACY - 2

Privacy

Suggestions matching public code

Allowed ▾

Copilot can allow or block suggestions matching public code. Learn more about [code suggestions](#).

☒ Allow GitHub to use my data for product improvements

Allow GitHub, its affiliates and third parties to use my data, including Prompts, Suggestions, and Code Snippets, for product improvements. More information in the [Privacy Statement](#).

Allow GitHub to use my data for AI model training

Disabled

Allow GitHub, its affiliates and third parties to use my data, including Prompts, Suggestions, and Code Snippets, for AI model training. More information in the [Privacy Statement](#).

For more information about the data your organization receives regarding your use of GitHub Copilot, please review [GitHub's Privacy Statement](#).

ⓘ It can take up to 30 minutes for the changes to take effect. Restart your code editor for the changes to take effect immediately.

How to get to the Privacy panel.

- Log in to GitHub
- Go to github.com and sign in.
- Open Copilot Settings
- Click your profile picture (top-right).
- Select Your Copilot.
- Scroll down right panel to find Privacy

ENABLE / DISABLE COMPLETION

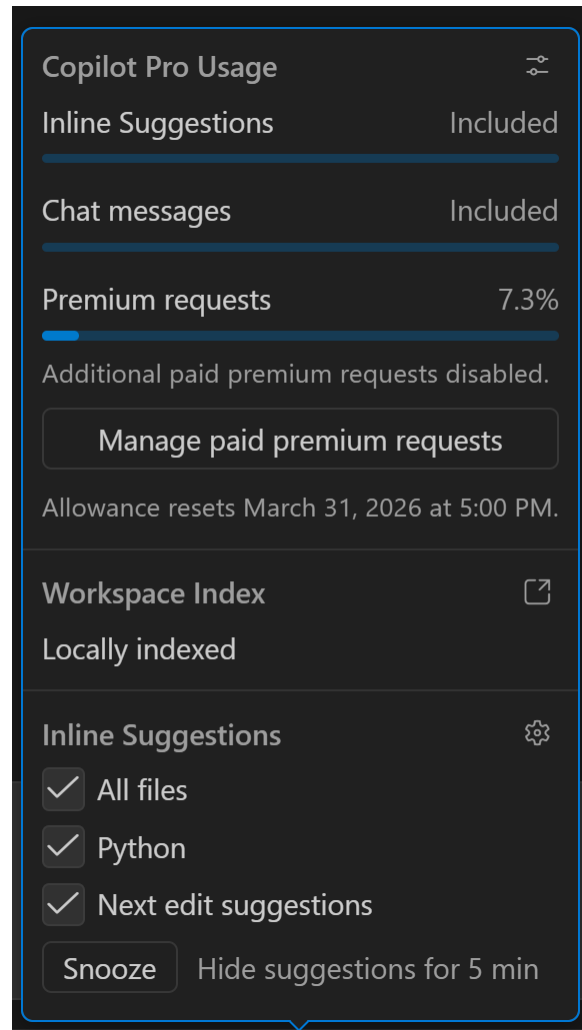
The enable/disable code completions toggle in GitHub Copilot controls whether inline code suggestions appear while you type. You can set it globally, per language, or for specific file types.

When completions are enabled, you code faster because of boilerplate generation, context-aware help, and exposure to idiomatic patterns. It feels like a pair-programmer suggesting alternatives. The downside is potential over-reliance, clutter from too many suggestions, and the risk of accepting inaccurate or irrelevant code.

ENABLE / DISABLE COMPLETION – 2

When completions are disabled, you retain full control of your code with fewer distractions and better focus, which can be useful for teaching or training. However, you lose the productivity boost, brainstorming value, and language guidance Copilot provides.

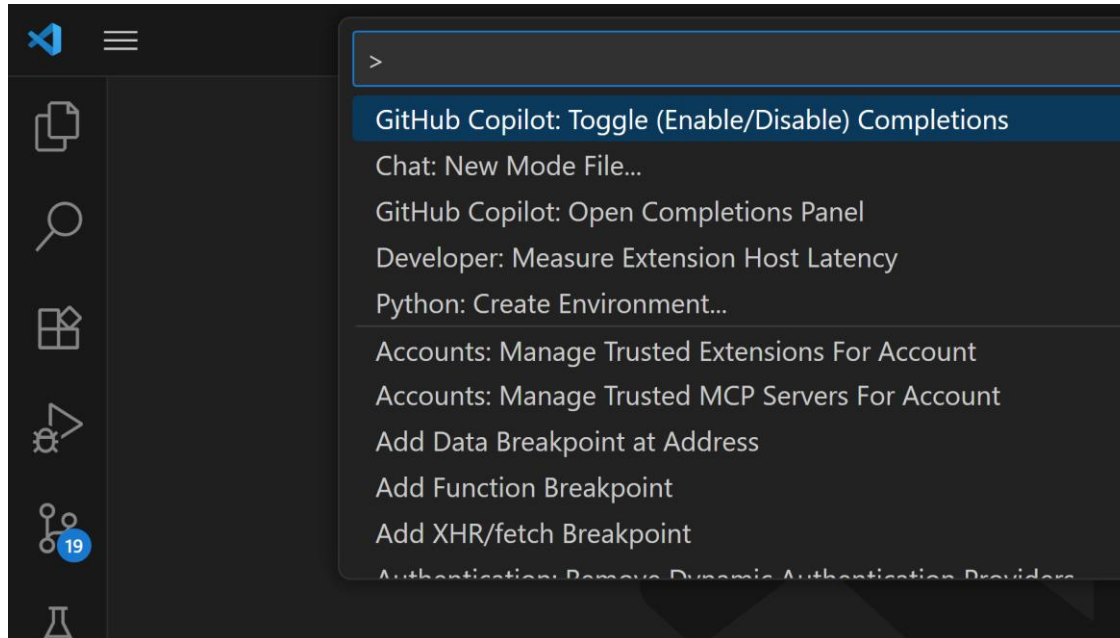
ENABLE / DISABLE COMPLETION - 3



In VS Code, you can toggle Copilot code completions a few different ways. The fastest method is using the Status Bar. Click the GitHub Copilot menu and use the checkboxes to enable/disable completions globally or a specific programming language.

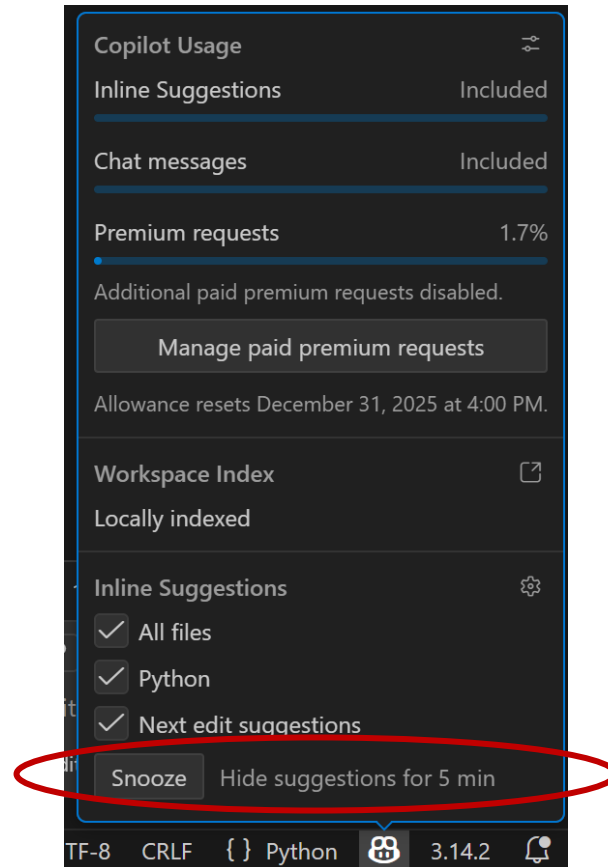
Next Edit Suggestions: in VS Code refers to GitHub Copilot's ability to predict and suggest the next logical change.

ENABLE / DISABLE COMPLETION – 4



Ctrl-Shift-P and GitHub Copilot: Toggle
(Enable / Disable) Completions

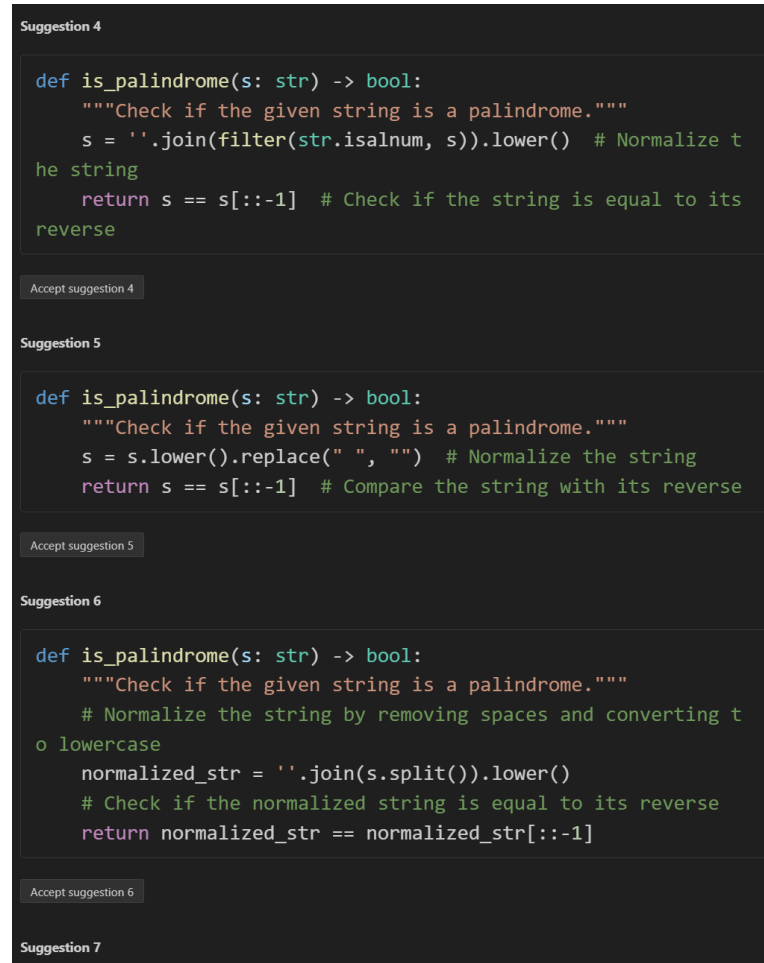
SNOOZE CODE COMPLETION



The Snooze feature temporarily prevents Copilot's inline suggestions in your editor.

When activated, Copilot pauses automatic suggestions for 5 minutes, then resumes automatically.

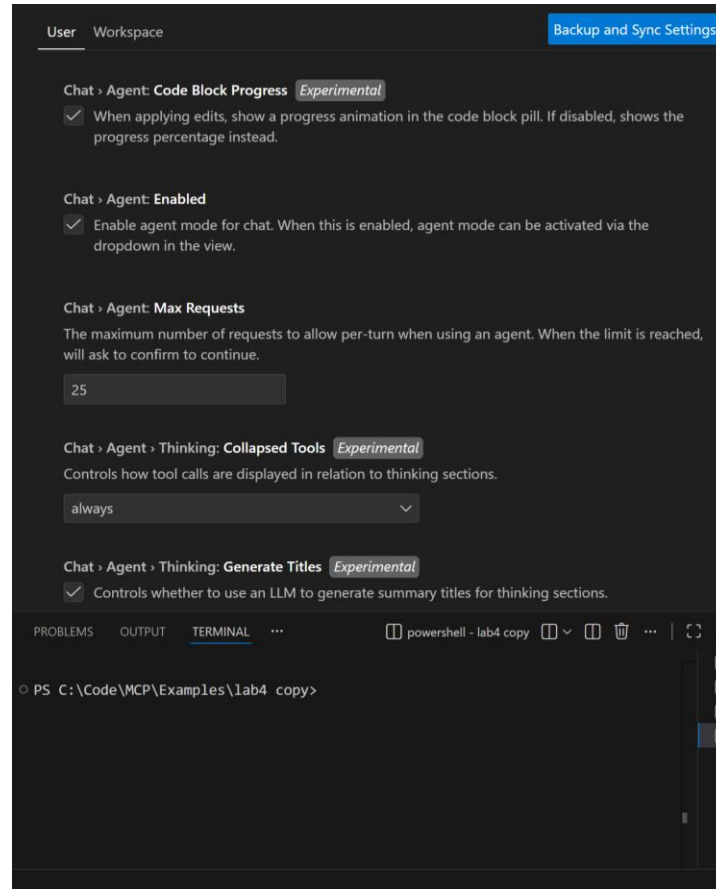
OPEN COMPLETION PANEL



The Open Completion Panel lets you view multiple Copilot suggestions instead of the single inline completion shown in the editor. You can compare alternatives and choose the best option for your task.

This is especially useful when inline code completions are turned off, allowing you to manually request and review suggestions on demand.

GITHUB COPILOT SETTINGS (VSCODE)



GitHub Copilot Chat settings are not group in VS code, which can make them difficult to find. To locate them easily, from the gear menu of the Chat window, select the last menu item – Chat Settings.

POP QUIZ: GITHUB COMMANDS

How would you find all the commands in VS code related to GitHub Copilot?



10 MINUTES

RETROSPECTIVE



This module showed how GitHub Copilot works as a cloud-based AI assistant, using context and fill-in-the-middle to generate relevant suggestions. We walked through its workflow—from prompts and context gathering to secure cloud processing and IDE results—highlighting that Copilot is more than code completion; it adapts to the developer's environment.

We also stressed responsibility and safeguards. Copilot boosts productivity, but privacy, licensing, and security require careful configuration and adhering to best practices.

LAB 2 – PALINDROME



PALINDROME



A palindrome is a word, phrase, number, or sequence that reads the same forward and backward, such as "racecar", "level", or "12321".

SETUP

- Open VS Code
- Find and open the Lab2 folder. Within that folder, create a new folder: `is_palindrome`.
- Alternately, create a new folder called `is_palindrome` somewhere convenient.
- Within the `is_palindrome` folder, create a new file: `is_palindrome.py`.

CREATE THE PROGRAM

1. In `is_palindrome.py`, enter a comment to create a `is_palindrome` function with a string parameter.
2. Press enter and wait.
3. If necessary, tab again and wait to complete the function implementation.
4. If more than one suggestion provided, scroll (<>) the suggestions and tab to select one.

RUN THE PROGRAM

1. Add a comment to insert a main function to test the `is_palindrome` function.
2. *Save the code resulting code.*
3. Run the program from the terminal window.

IS_PRIME

1. Add the function `is_prime` to your code, which returns whether a value is prime.
2. Generate the code to test the function.

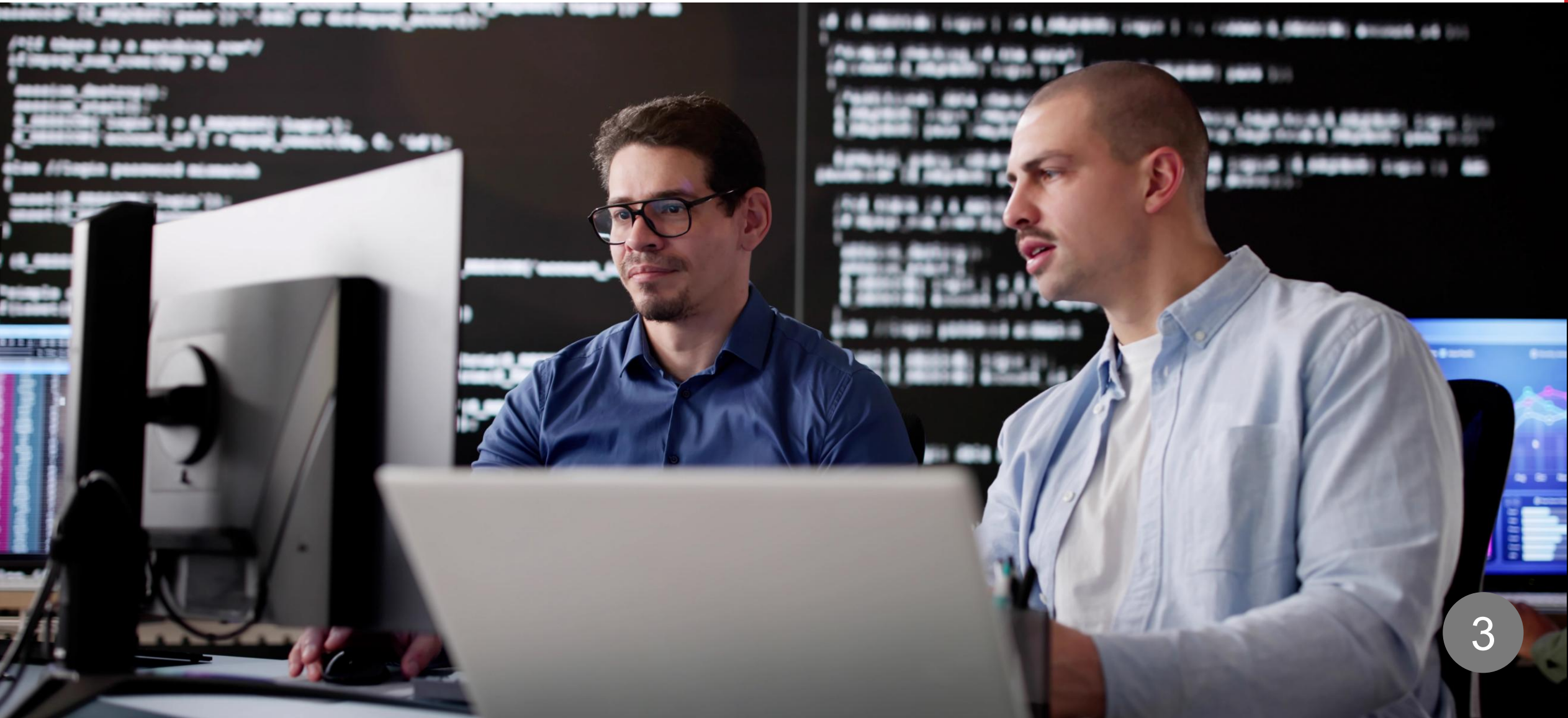
Lab completed

“Develop a passion for learning.”



CODE COMPLETION

BASICS AND BEST PRACTICES



INTRODUCTION



This module is a practical guide to using GitHub Copilot inline code completions effectively. It explains ghost text and “getting started” habits (clear intent, pause for suggestions, accept/cycle), then shows how to sharpen results with context: meaningful names, comments/docstrings, and even adding imports to steer libraries.

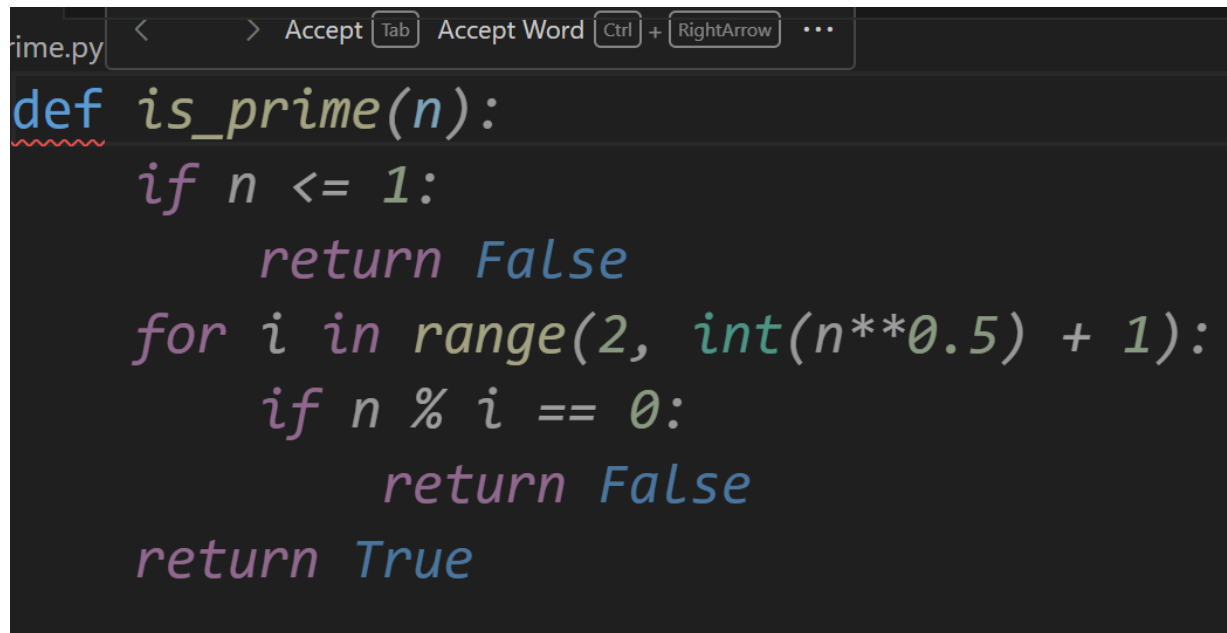
It walks through scaffolding patterns, partial code guidance, and best-practices.

GETTING STARTED

Best practices for code completion:

- Start with a clear description: type a descriptive comment or a function signature (e.g., `# print "hello, world"` or `def add(a, b):`).
- Pause—Copilot will show ghost text. Press Tab to accept, or use the Inline Completion toolbar to cycle alternatives (if offered).
- Prefer small, incremental prompts: review the next line or two, accept, run, repeat. Short loops of type `→ accept → test` keep suggestions on track.
- Use meaningful names (files, functions, variables). Better names `→` better suggestions.

GHOST TEXT



The screenshot shows a code editor with a dark theme. The file name 'ime.py' is visible in the top left. The editor contains Python code for a function `is_prime(n)`. The first line, `def is_prime(n):`, is followed by a line of ghost text, `if n <= 1:`, which is displayed in a lighter, italicized font. Below this, the code continues with `return False`, `for i in range(2, int(n**0.5) + 1):`, `if n % i == 0:`, `return False`, and `return True`. The ghost text is positioned as if it were the next line of code to be typed.

```
ime.py < > Accept Tab Accept Word Ctrl + RightArrow ...  
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n**0.5) + 1):  
        if n % i == 0:  
            return False  
    return True
```

Copilot predicts what you might type next, showing it as italic gray text. It appears as faint, gray, italicized text that appears inline in your editor as an inline suggestion.

- The text isn't "real code" yet — it won't run, save, or commit until you accept it.
- Accepting with a keystroke (tab) can save you multiple lines of typing.
- Safe to ignore: If you keep typing, the italic ghost text disappears automatically.
- Options: You can cycle through alternatives (next/previous suggestion) if more than one is available.

CONTEXT

Provide context:

- Function and variable names (`calculate_tax`, `db_client`).
- Comments/docstrings that state what and how.
- Imports and existing patterns in your file.

If suggestions drift, narrow the context:

- Add a clarifying comment.
- Rename ambiguous identifiers.
- Close irrelevant files or move code into a focused file.

CONTEXT - COMMENTS

A second comment can help GitHub Copilot by narrowing the context and reducing ambiguity. While one comment may describe the general task, adding another can specify how it should be done, what tools or techniques to use, or what constraints to follow.

```
# Calculate factorial of a number  
# Use recursion  
def factorial(n):
```

PARTIAL CODE GUIDANCE

Partial code guidance means you don't have to fully describe or write the whole solution—just give GitHub Copilot a hint through a short comment, function signature, or partial line of code. Copilot will then try to complete it based on the context.

```
# Example 1
```

```
# Return the largest number in a list
```

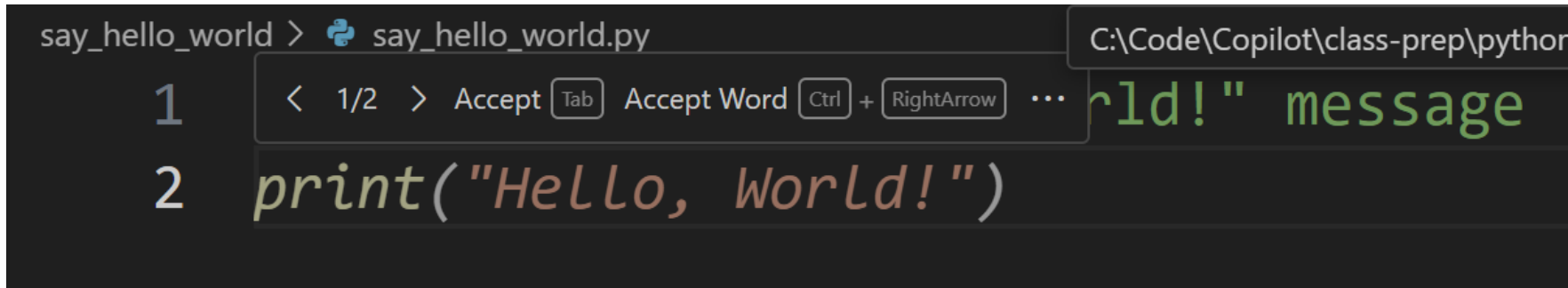
```
def find_max(numbers):
```

```
# Example 2
```

```
# Print numbers 1 to 5
```

```
for i in range(
```

COPILOT INLINE SUGGESTION TOOLBAR



The screenshot shows a code editor with a file named `say_hello_world.py` open at the path `C:\Code\Copilot\class-prep\python`. The editor displays two lines of Python code: `1` `world!" message` and `2` `print("Hello, World!")`. An inline suggestion toolbar is visible above the second line, containing the following elements: a left arrow, a right arrow, a "1/2" indicator, an "Accept" button with a `Tab` key icon, an "Accept Word" button with a `Ctrl` + `RightArrow` key icon, and a three-dot menu icon.

- Arrows (◀ ▶): to cycle through different completion suggestions.
- Accept (Tab): indicates you can press Tab (or Enter) to accept the current suggestion.
- Word: switches the suggestion to per word completion (within the current suggestion).

COPILOT INLINE SUGGESTION TOOLBAR - 2

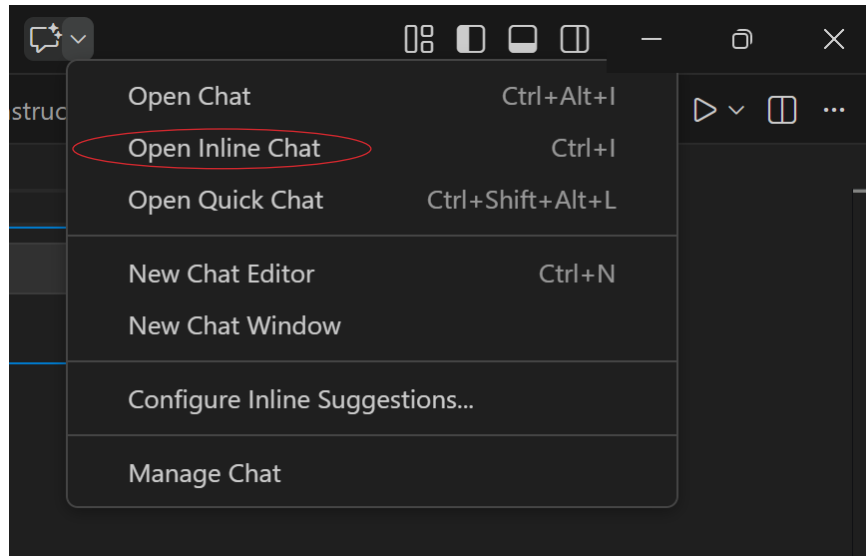


Accept Line. Let's you accept just the current line of Copilot's suggestion instead of an entire multi-line completion. This is handy when Copilot offers a large block of code.

Always Show Toolbar. Toggles whether the Inline Suggestion toolbar is always visible or only appears on **hover**.

Send Copilot Completion Feedback. Opens Issue Reporter form to report an issue on GitHub Copilot for VS Code extension.

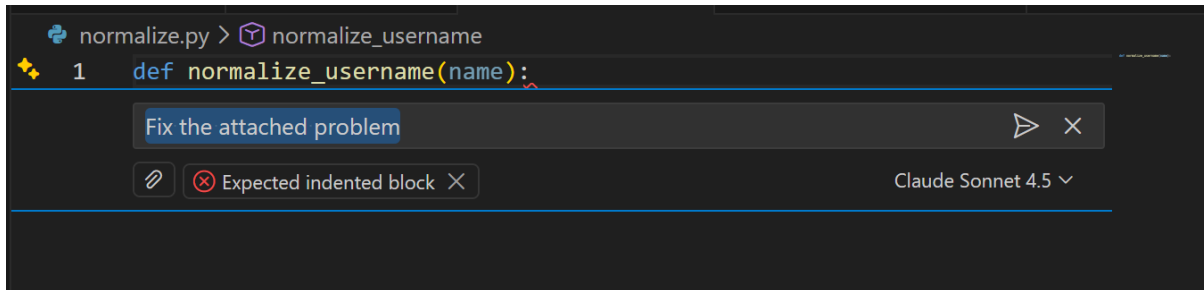
INLINE CHAT WINDOW



Adjacent to the Chat View button, open the Chat View menu. You can select *Open Inline Chat* to view the Inline Chat window.

Alternatively, open the window with Ctrl+I.

INLINE CHAT WINDOW - 2



Inline Chat window presents a prompt where you can interact with GitHub Copilot in the editor window.

Think of it as “Copilot, look at this exact spot in my code and do something here.”

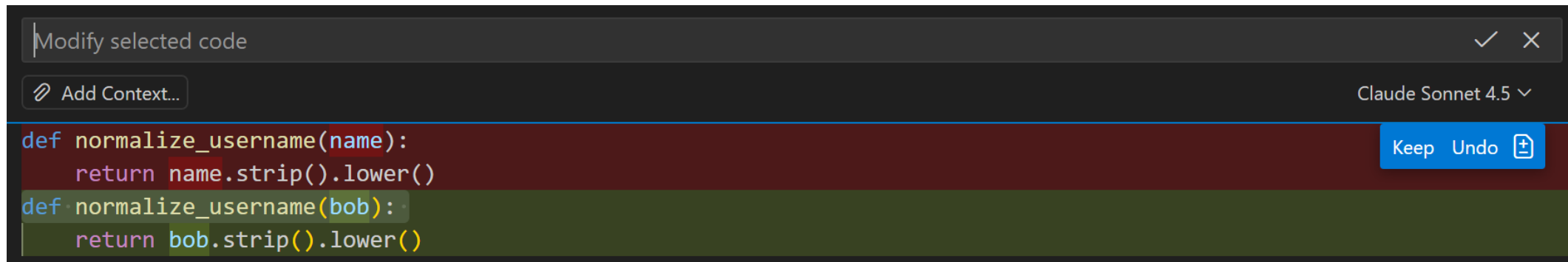
Set the context before opening the Inline Chat window

- Set the cursor location, or
- Select code.

INLINE CHAT WINDOW – 3

When you make code changes using the Inline Chat window, VS Code shows a before-and-after view in contrasting colors.

Click the Keep button to apply the changes to your code.



The screenshot shows the VS Code Inline Chat window. At the top is a text input field labeled "Modify selected code" with a checkmark and a close button. Below the input field is a button labeled "Add Context...". On the right side, it says "Claude Sonnet 4.5" with a dropdown arrow. The main area displays a comparison of code. The original code is on a dark red background, and the modified code is on a dark green background. The changes are: replacing the parameter `name` with `bob` in the function definition and the `return` statement. At the bottom right of the code area, there are buttons for "Keep", "Undo", and a plus icon in a square.

```
def normalize_username(name):  
    return name.strip().lower()  
def normalize_username(bob):  
    return bob.strip().lower()
```

BEST PRACTICES

- Request alternatives: Add/adjust comments or partially type the target line to reshape the suggestion.
- Keep prompts safe: Don't include sensitive data (passwords, keys, proprietary algorithms).
- Validate: Run, lint, and test. Copilot accelerates typing; you own the quality.
- Feedback loop: small steps improve the next suggestion iteratively.

RETROSPECTIVE



This module showed practical ways to use GitHub Copilot inline code completions effectively. We saw how ghost text works and how intent signals—comments, docstrings, imports, and function signatures—improve results. Techniques like scaffolding, partial code hints, and test-driven scaffolding emphasized that small, clear prompts generate better completions.

Reviewing output for correctness, security, and style, while avoiding sensitive data in prompts.

LAB 3 – BANKING APPLICATION



OBJECTIVE



Use GitHub Copilot to build a simple banking application while reinforcing:

- Inline code completion
- Context-aware generation
- Code modification within the editor

You will create functions incrementally and enhance them using the features demonstrated in the previous module.

SETUP

- Open VS Code
- Find and open the Lab3 folder. Within that folder, create a new folder: banking.
- Alternately, create a new folder called banking somewhere convenient.
- Within the banking folder, create a new file: banking.py.

GENERATE FUNCTIONS

Open the file named:

`banking.py`

Begin with a short comment describing the purpose of the program.

This simple banking application models basic transactions such as deposits, withdrawals, loans, and loan payments. Each function updates and returns the account balance to demonstrate clear and predictable financial operations.

CREATE BANKING FUNCTIONS

Create *one function at a time* using inline code completion.

Each function must:

- Accept a balance parameter
- Accept an amount parameter
- Return the updated balance
- Remain a pure function (no global variables)

Create the following functions in this order:

- deposit
- loan
- withdrawal
- loan_payment

CREATE BANKING FUNCTIONS - 2

For each function:

- Type a descriptive comment (example: `# create a deposit function`)
- Press Enter
- Review the inline suggestion
- Press Tab to accept if appropriate
- Adjust as needed
- Do not manually write the full function. Use GitHub Copilot to generate each one.

UPDATE DEPOSIT AND WITHDRAWAL

In a single step, enhance the deposit and withdrawal functions so that when each function completes, it displays:

- A clear message indicating the transaction type
- The updated balance

Use the techniques demonstrated in the previous module. Avoid rewriting the functions manually.

TEST THE BANKING APPLICATION

Using inline suggestions, ask GitHub Copilot to generate a main function that test the four banking functions.

Test the application in the terminal window.

Lab completed

“Develop a passion for learning.”



COPILOT CHAT

INTEGRATION



INTRODUCTION



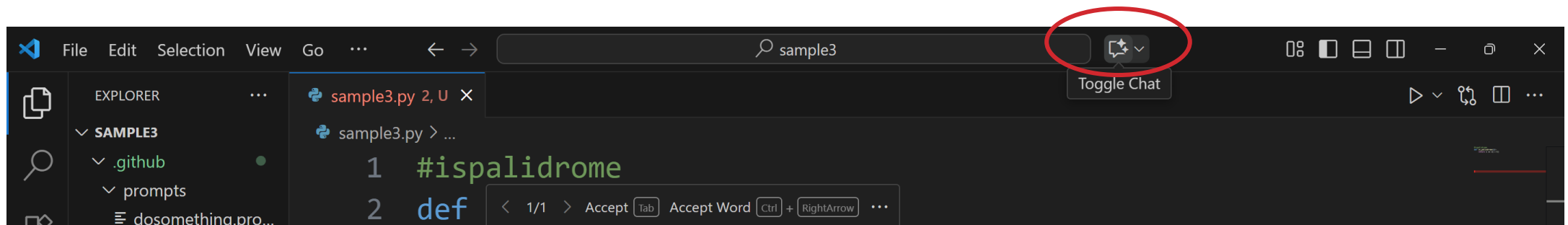
The next module introduces GitHub Copilot Chat in VS Code as a conversational coding assistant. It expands beyond inline completions by supporting natural language queries, code edits, and even multi-step workflows through Agent, Ask, and Plan modes

It also highlights practical tools such as checkpoints to track and roll back edits, action controls to apply or discard suggestions, and inline chat for quick changes. Features like slash commands, #mentions for context, and variable awareness show how Copilot integrates smoothly into the coding workflow.

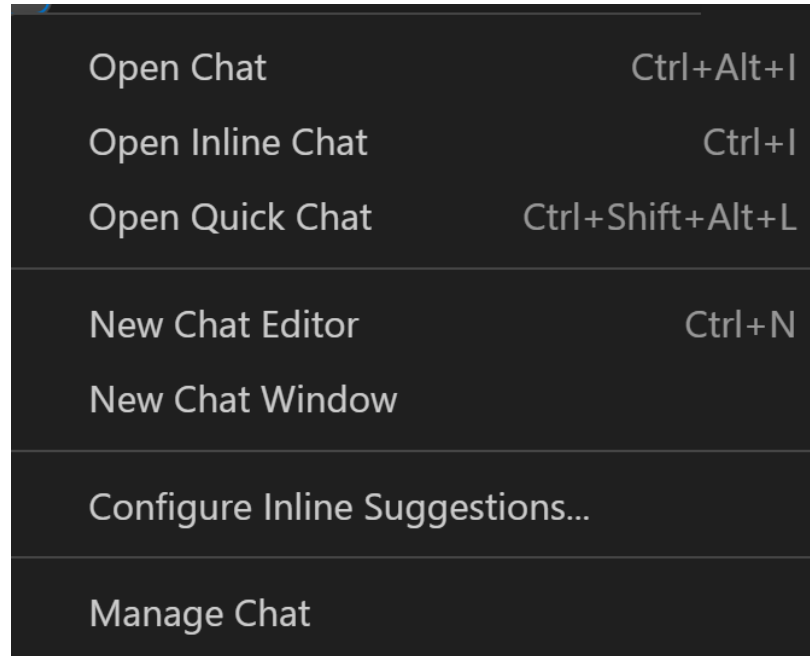
COPILOT CHAT

Open the Chat window:

- If installed, you'll see a Chat View icon in the Activity Bar on the right sidebar.
Click it to open the Chat View.
- Alternatively, you can open chat directly with:
Ctrl+Shift+I (Windows/Linux)
^⌘I (macOS).



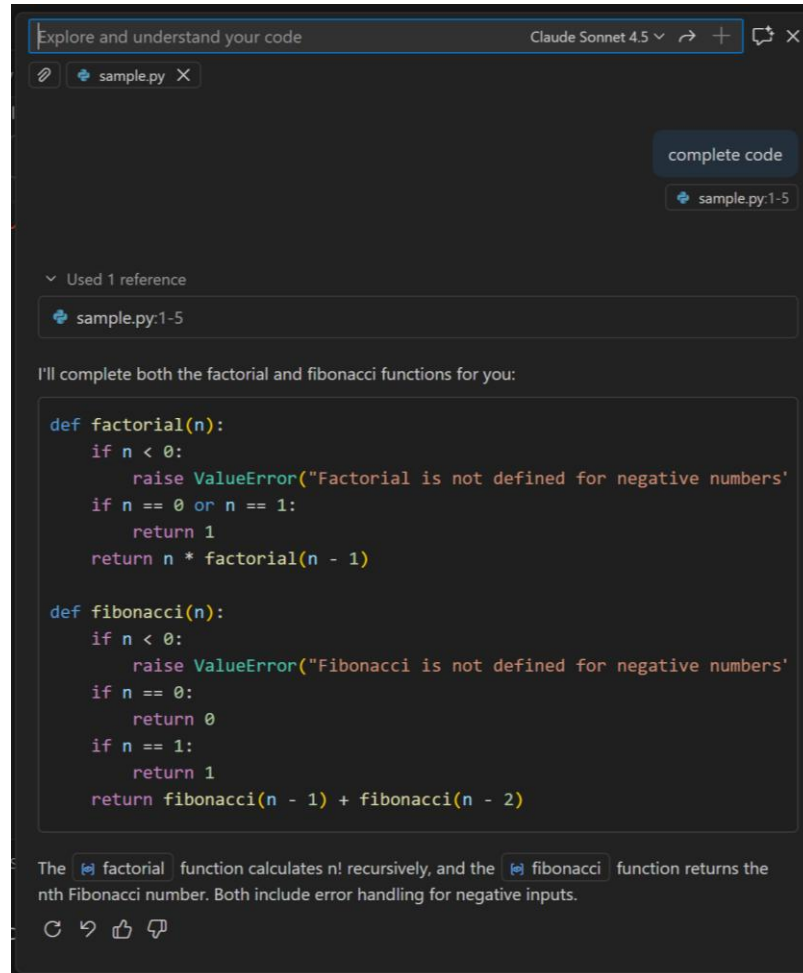
CHAT VIEW MENU



Open Chat (Chat View) opens the Chat window. This is the same as directly clicking on the button.

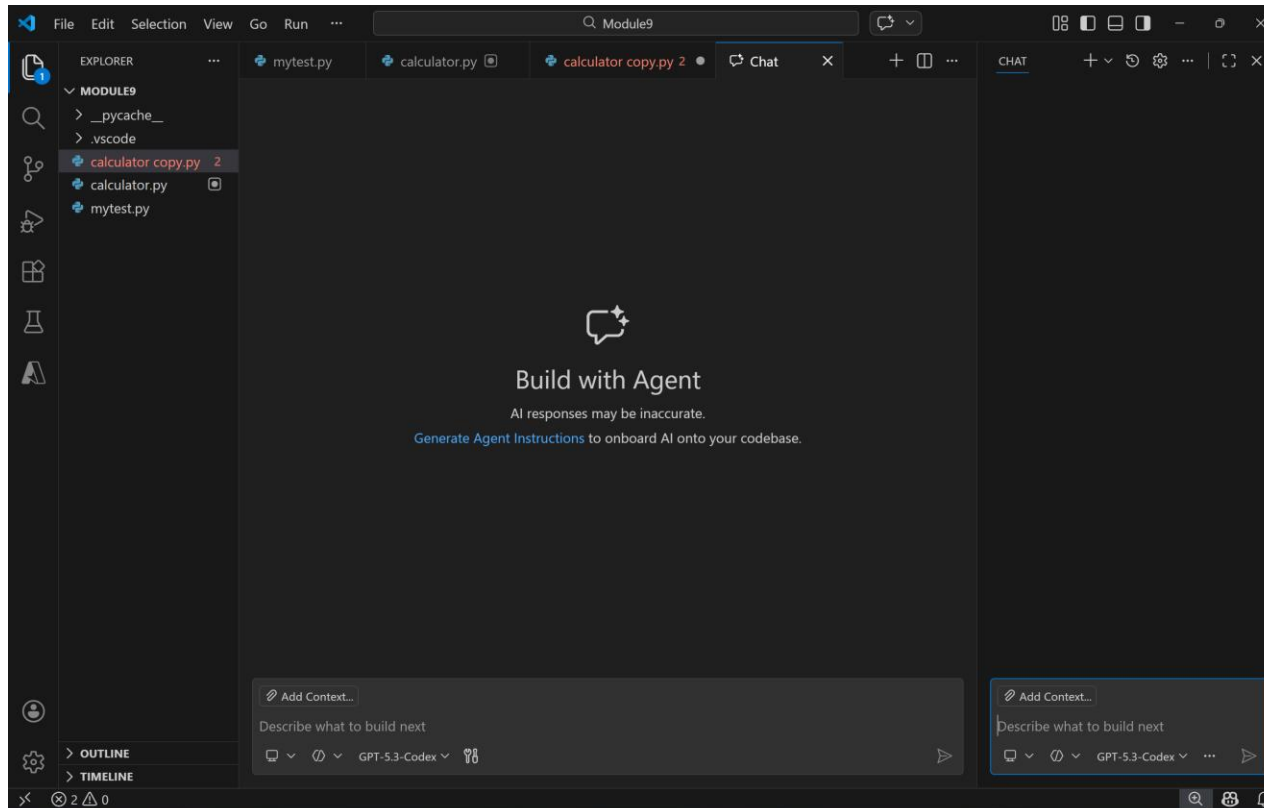
Open Inline Chat open a Prompt window within the VS Code source code editor at your cursor position, as shown before.

CHAT VIEW MENU - 2



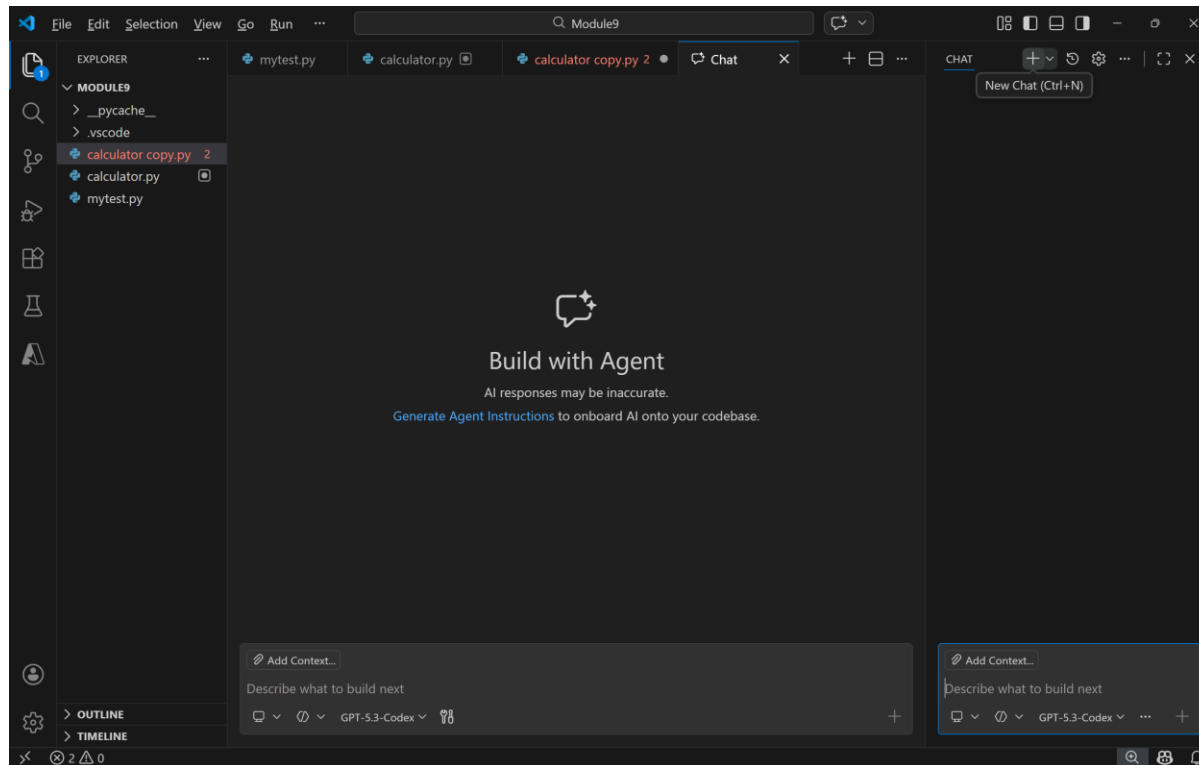
From the Chat View menu, Open Quick Chat opens a lightweight GitHub Copilot chat panel directly in the editor, allowing you to submit context-aware prompts about your code, with history, without switching to the full Copilot Chat view.

CHAT VIEW MENU - NEW CHAT WINDOW



Opens a Copilot chat session as a floating chat view. It is designed for quick, conversational interactions such as asking coding questions, generating small snippets, or explaining errors. This option is best when you need fast help without leaving your current file.

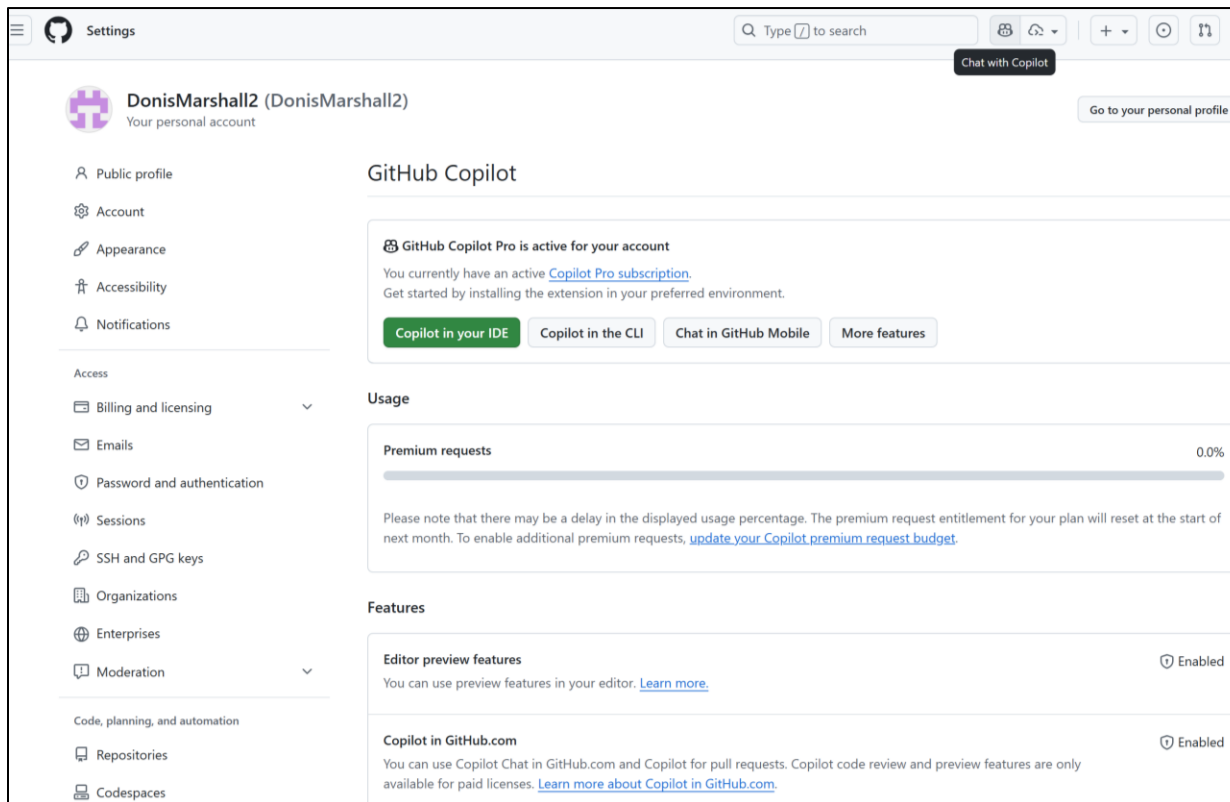
CHAT VIEW MENU - NEW CHAT EDITOR



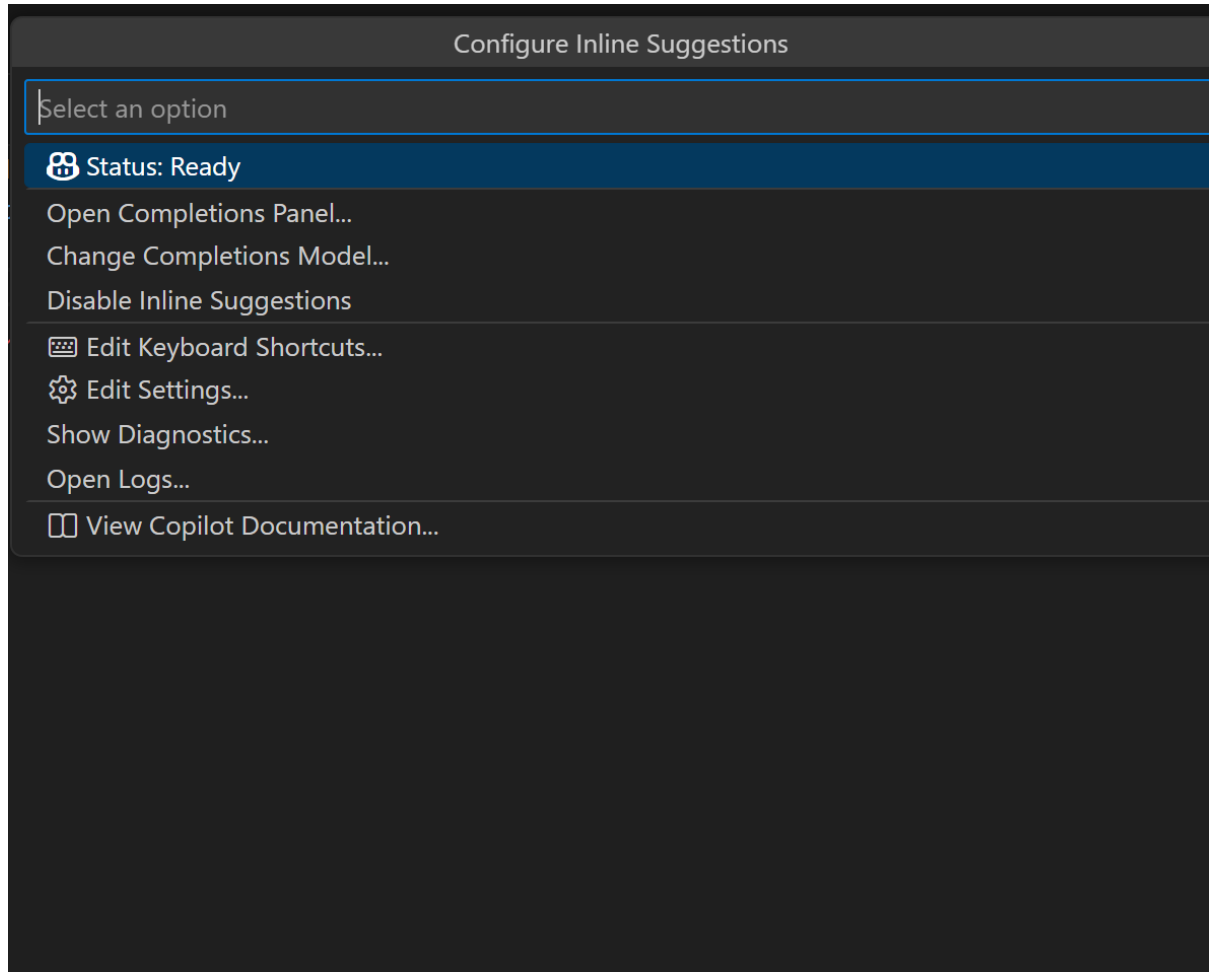
Opens a Copilot chat session as a new tab in the editor area, similar to opening a source file. This provides a larger, persistent workspace that is better suited for longer prompts, multi-step problem solving, code reviews, or architecture discussions where you want to scroll and revisit earlier responses.

CHAT VIEW MENU - 5

The Manage Chat menu option opens the Copilot settings (Features) for the current GitHub account.



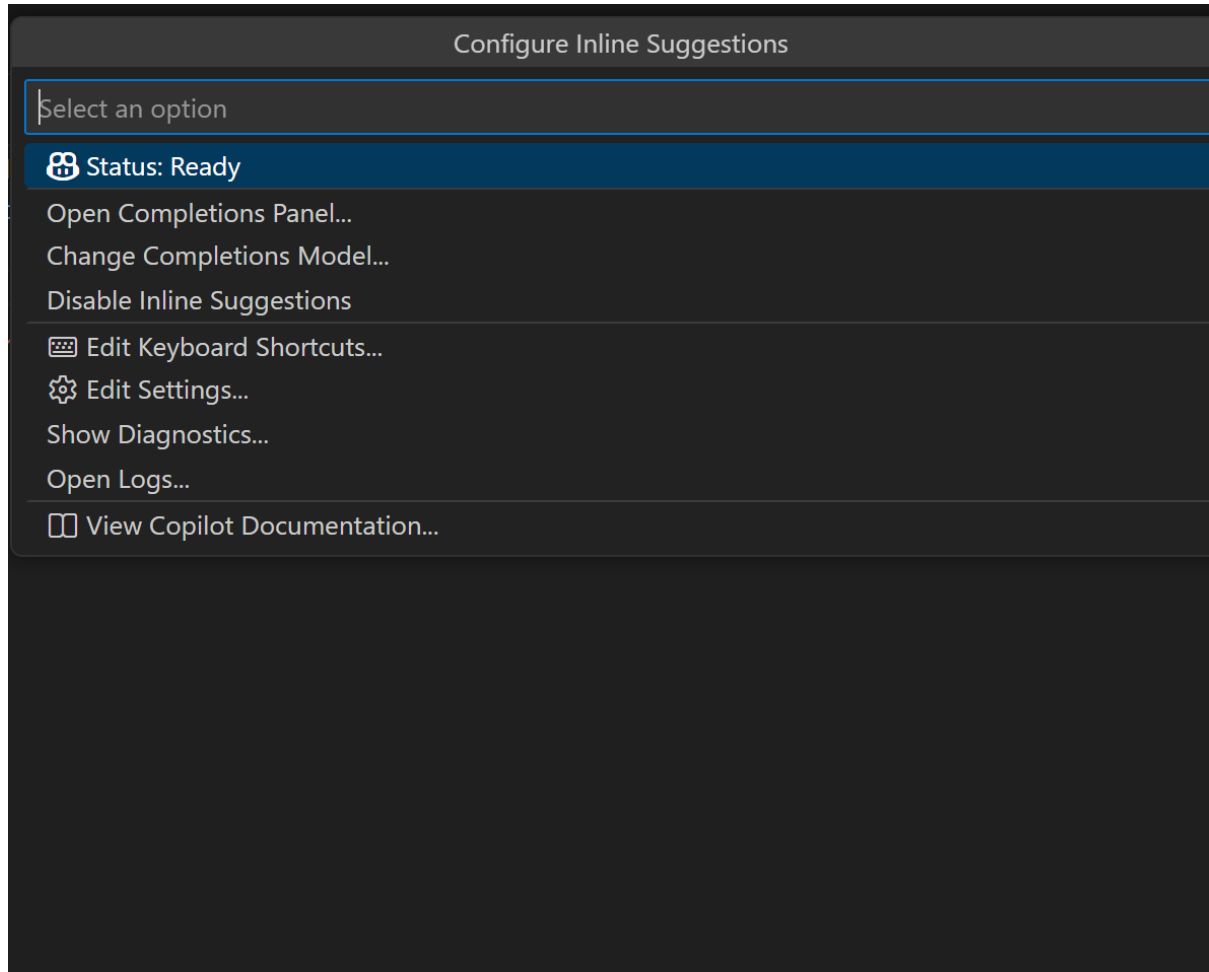
CONFIGURE INLINE SUGGESTIONS



The Configure Inline Suggestions menu choice on the Chat View menu focuses on how inline suggestion behaves, including the current status.

- Status – Displays whether inline code completions is currently active.
- Open Completions Panel – Opens a dedicated panel where you can see multiple suggestions if available, as shown before.

CONFIGURE INLINE SUGGESTIONS - 2



- Change Completions Model: this allows switching of the underlying LLM that generates inline code completions.
- Disable Inline Suggestions: when inline suggestions are disabled, the Open Completions Panel becomes especially important because it allows you to request code suggestions on demand.

CONFIGURE INLINE SUGGESTIONS - 3

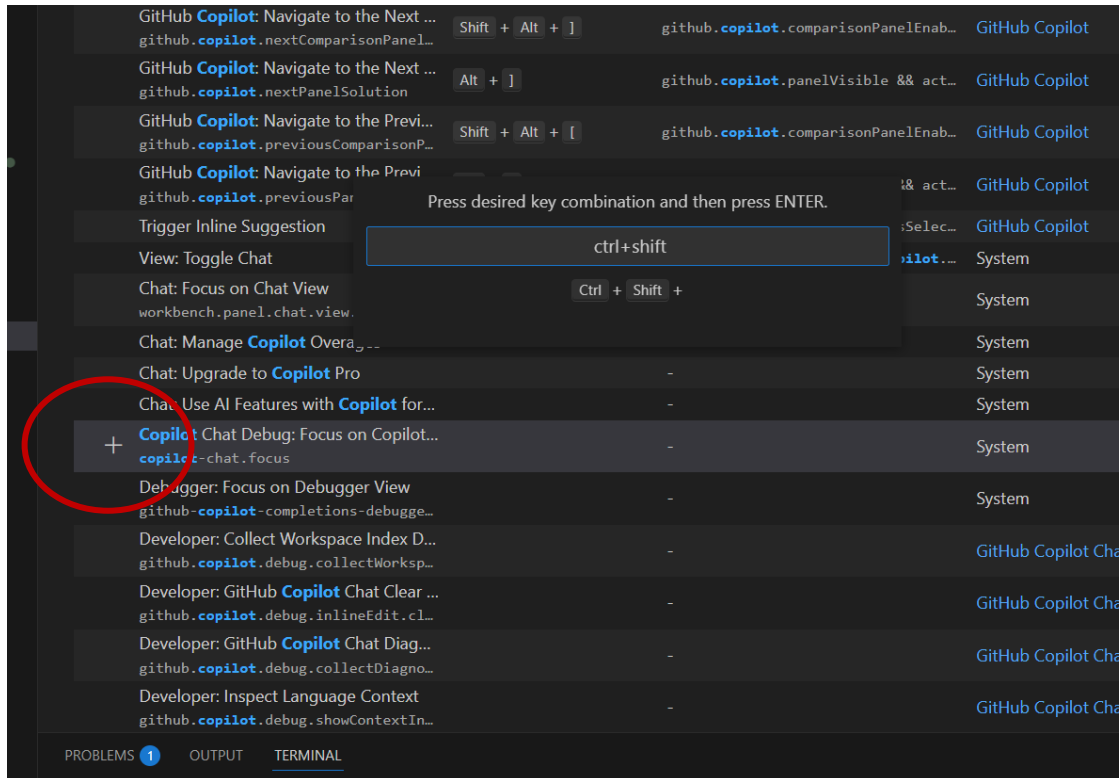
- Edit Keyboard Shortcuts – Opens the Keyboard Shortcuts editor with Copilot commands pre-filtered, so you can assign or change hotkeys for things like triggering completions or cycling through suggestions.
- Edit Settings – Opens the Copilot-related settings for VS Code where you can control behavior such as enabling completions by language, extensions, and so on.

EXAMPLE - EDIT KEYBOARD SHORTCUTS

The Edit Keyboard Shortcuts option in the Copilot Chat menu opens VS Code's Keyboard Shortcuts editor with Copilot commands pre-filtered. From there, you can assign, change, or remove shortcuts.

Command	Keybinding	When	Source
GitHub Copilot : Accept Comparison P... github. copilot .acceptCursorCompari...	Ctrl + Shift + /	github. copilot .comparisonPanelEnab...	GitHub Copilot
GitHub Copilot : Accept Panel Suggest... github. copilot .acceptCursorPanelSo...	Ctrl + /	github. copilot .panelVisible && act...	GitHub Copilot
GitHub Copilot : Debug Last Terminal ... github. copilot .chat.rerunWithCopil...	Ctrl + Alt + .	github. copilot -chat.activated && t...	GitHub Copilot Cha
GitHub Copilot : Navigate to the Next ... github. copilot .nextComparisonPanel...	Shift + Alt +]	github. copilot .comparisonPanelEnab...	GitHub Copilot
GitHub Copilot : Navigate to the Next ... github. copilot .nextPanelSolution	Alt +]	github. copilot .panelVisible && act...	GitHub Copilot
GitHub Copilot : Navigate to the Previ... github. copilot .previousComparisonP...	Shift + Alt + [	github. copilot .comparisonPanelEnab...	GitHub Copilot
GitHub Copilot : Navigate to the Previ... github. copilot .previousPanelSoluti...	Alt + [	github. copilot .panelVisible && act...	GitHub Copilot

COMMON CUSTOM KEY BINDINGS



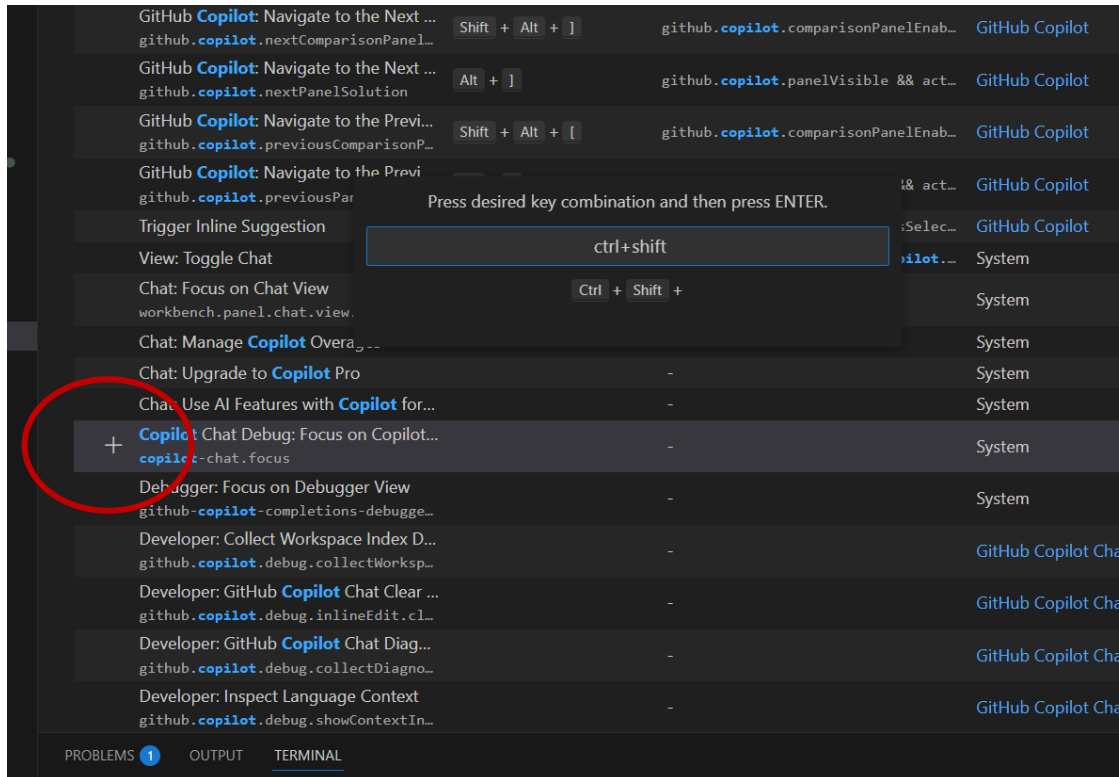
GitHub Copilot: Enable/Disable Completions – Toggle inline suggestions globally.

GitHub Copilot: Explain with Copilot – Ask Copilot to explain highlighted code.

GitHub Copilot: Accept Suggestion – Accept the current suggestion within code completion.

GitHub Copilot: Trigger Inline Suggestion – Manually request a suggestion on demand.

COMMON CUSTOM KEY BINDINGS - 2



GitHub Copilot: Generate Tests – Ask Copilot to generate unit tests for selected code.

GitHub Copilot: Insert at Cursor / Apply to Selection – Choose how Copilot applies generated code.

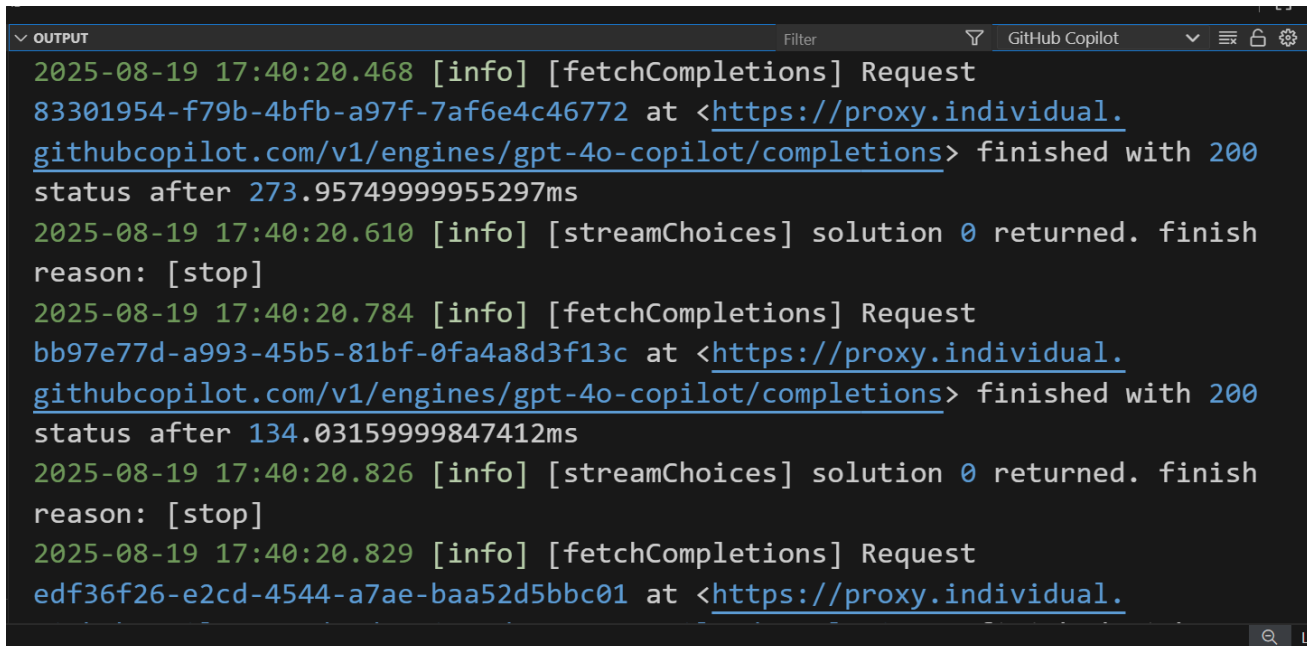
CONFIGURE INLINE SUGGESTIONS - 4

- Show Diagnostics (static) – Displays diagnostic information about Copilot's state and connectivity. This can help confirm whether the extension is connected properly and if requests are being sent and received.
- Open Logs (real time) – Opens the Output panel in VS Code filtered to GitHub Copilot logs. These logs capture startup messages, network errors, or authentication issues and are helpful for troubleshooting.

EXAMPLE - OPEN LOG

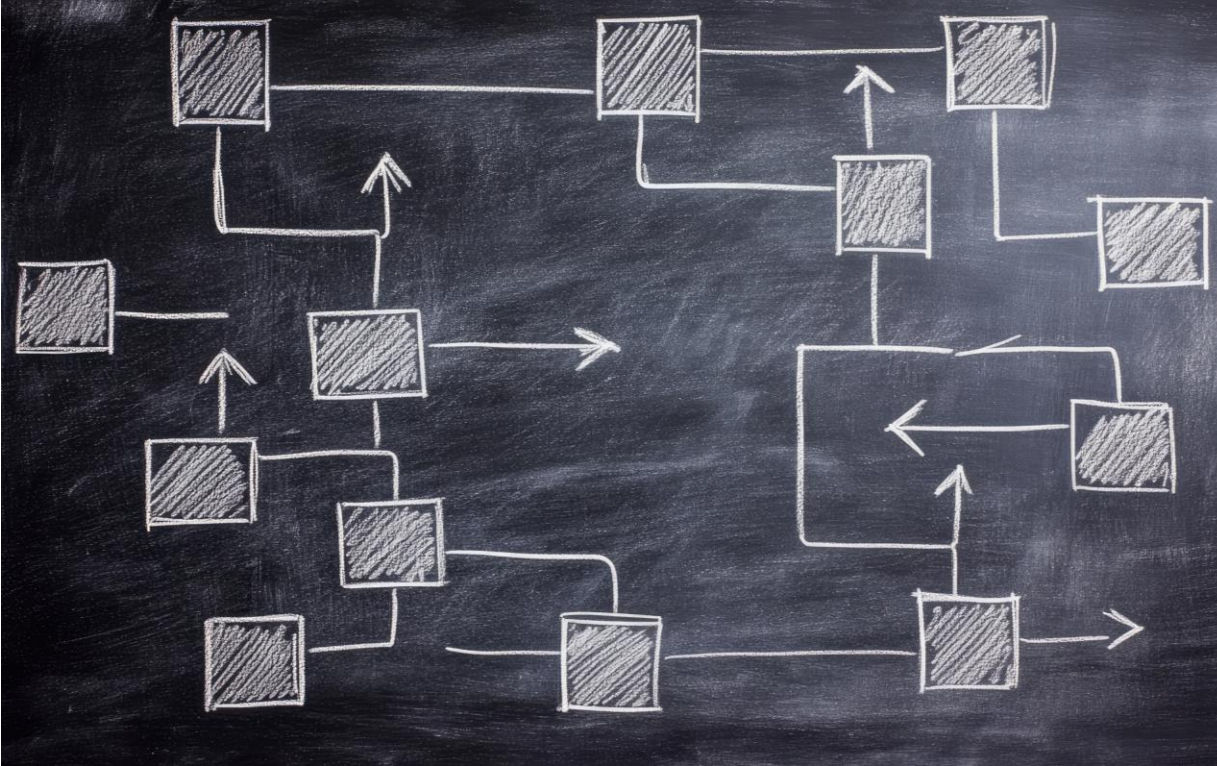
The Copilot log in VS Code is a diagnostic record of what the extension is doing behind the scenes. It captures events like when Copilot starts up, sends or receives a completion request, or encounters an error. You can view it by opening the Output panel and selecting GitHub Copilot from the dropdown list.

These logs are mainly for troubleshooting. If Copilot isn't generating suggestions or fails to connect, the log can reveal issues such as authentication errors, network timeouts, or extension crashes.

A screenshot of the Visual Studio Code Output panel. The panel title is 'OUTPUT' with a dropdown arrow. Below the title is a 'Filter' input field and a dropdown menu currently showing 'GitHub Copilot'. The log content is as follows:

```
2025-08-19 17:40:20.468 [info] [fetchCompletions] Request
83301954-f79b-4bfb-a97f-7af6e4c46772 at <https://proxy.individual.
githubcopilot.com/v1/engines/gpt-4o-copilot/completions> finished with 200
status after 273.95749999955297ms
2025-08-19 17:40:20.610 [info] [streamChoices] solution 0 returned. finish
reason: [stop]
2025-08-19 17:40:20.784 [info] [fetchCompletions] Request
bb97e77d-a993-45b5-81bf-0fa4a8d3f13c at <https://proxy.individual.
githubcopilot.com/v1/engines/gpt-4o-copilot/completions> finished with 200
status after 134.031599999847412ms
2025-08-19 17:40:20.826 [info] [streamChoices] solution 0 returned. finish
reason: [stop]
2025-08-19 17:40:20.829 [info] [fetchCompletions] Request
edf36f26-e2cd-4544-a7ae-baa52d5bbc01 at <https://proxy.individual.
```

CHAT MODE – AGENT MODE



For autonomous, multi-step, and iterative workflows that may require external tools. Copilot determines necessary tasks, executes commands (e.g., build, tests), applies and iterates until completion. This is the implementation of Agentic AI for Copilot GitHub.

CHAT MODE – PLAN MODE



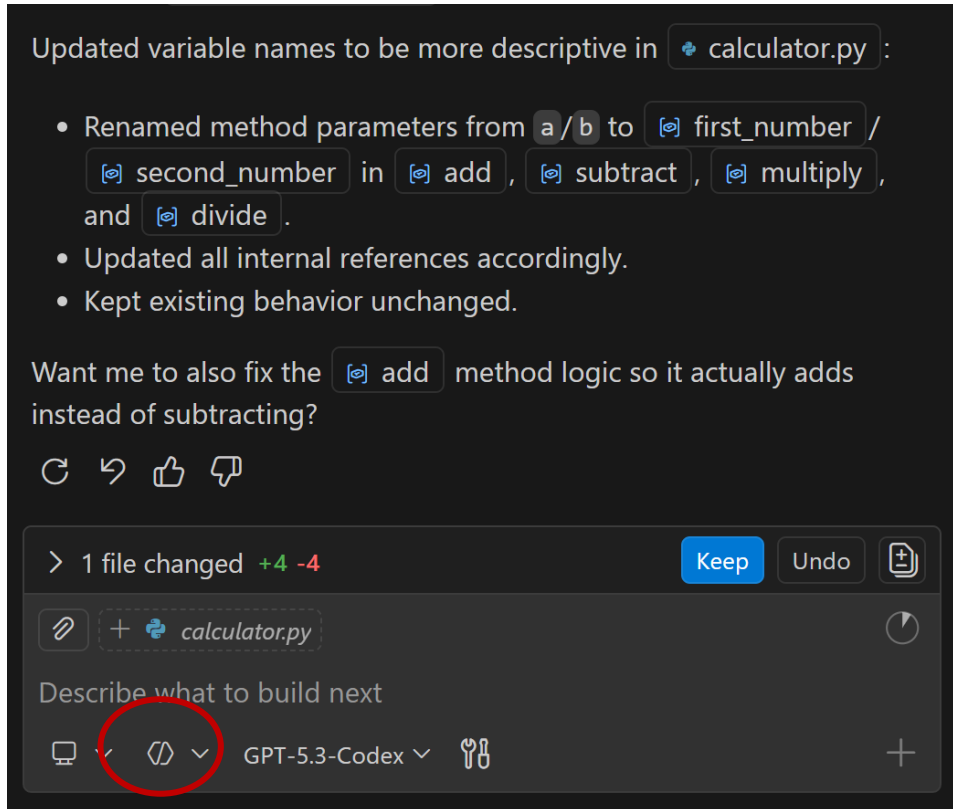
Plan Mode

In GitHub Copilot, Plan Mode is a chat interaction mode where Copilot thinks through the solution first and presents a step-by-step plan and considerations before writing or modifying code.

It performs these tasks:

- Analyze the request
- Break the work into clear steps
- Explain the approach, assumptions, and tradeoffs
- You decide whether to implement the plan.

CHAT MODE – CHANGING THE MODE



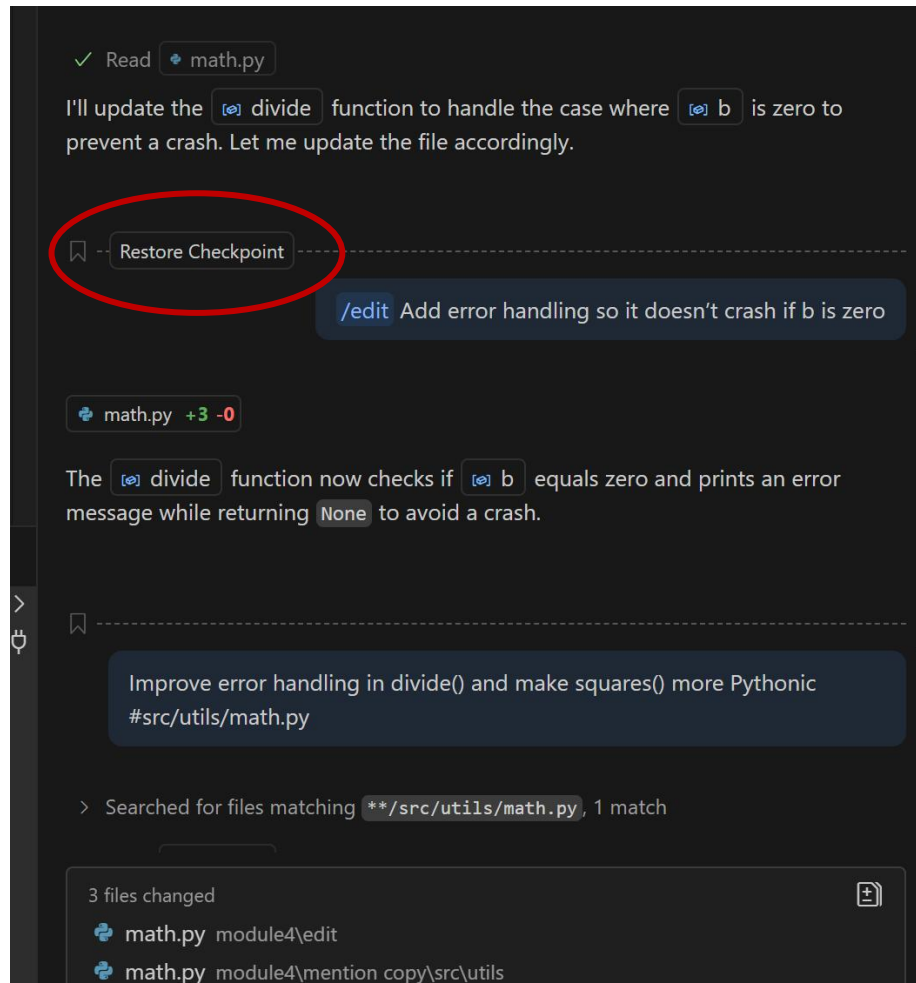
In the GitHub Copilot Chat window in VS Code, the default mode is Agent. To change it, click the mode selector at the top of the chat panel (for example, Agent) and choose another mode such as Ask from the dropdown. The chat session then switches to the selected mode.

CHECKPOINTS

In the Copilot Chat side panel, the history of your interactions is preserved in the chat conversation. Each request and Copilot's corresponding suggestion are shown in sequence, making it easy to scroll back and review prior edits or additions without losing context. This gives you a running log of how your code evolved through chat-based instructions.

At the same time, checkpoints are placed directly in your code editor at small dotted line markers. These indicate where Copilot made a change. If you hover over a dotted line, you'll see the checkpoint, which allows you to quickly roll back to the earlier version of your code. This provides a convenient safety net, ensuring that edits suggested through the chat panel are reversible.

CHECKPOINTS



In the Copilot Chat pane, checkpoints will also appear when hovering over the related action.

CHECKPOINTS - 2

Here an Add instruction is used to add exception handling to a function.

The location of checkpoints are indicated as dotted lines. Hovering over the final checkpoint displays the Restore Checkpoint button, as shown.

change the variable names to something more readable

math.py

> Used 1 reference

```
def divide(numerator, denominator):  
    return numerator / denominator
```

Restore Checkpoint

add exception handling

math.py:1-2

> Used 1 reference

```
def divide(numerator, denominator):  
    try:  
        return numerator / denominator  
    except ZeroDivisionError:  
        return "Error: Division by zero is not allowed."  
    except TypeError:  
        return "Error: Both numerator and denominator must be numbers."
```

🔄 ↶ 🍏 🗨

📎 Add Context... math.py:1-2

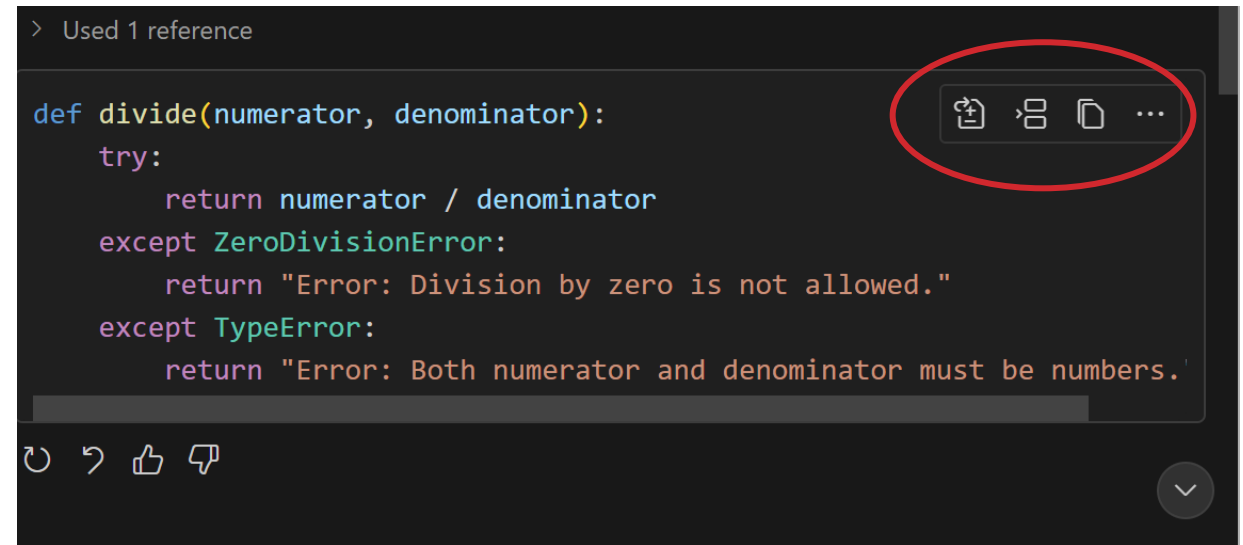
Add context (#), extensions (@), commands (/)

Ask ▾ GPT-4o ▾

🔊 🔍 ▾

ACTION BAR

When in Copilot Chat, suggestions are first presented in the Chat window. Hovering over that suggestion reveals four action buttons that let you decide how to handle (action) the suggestion.



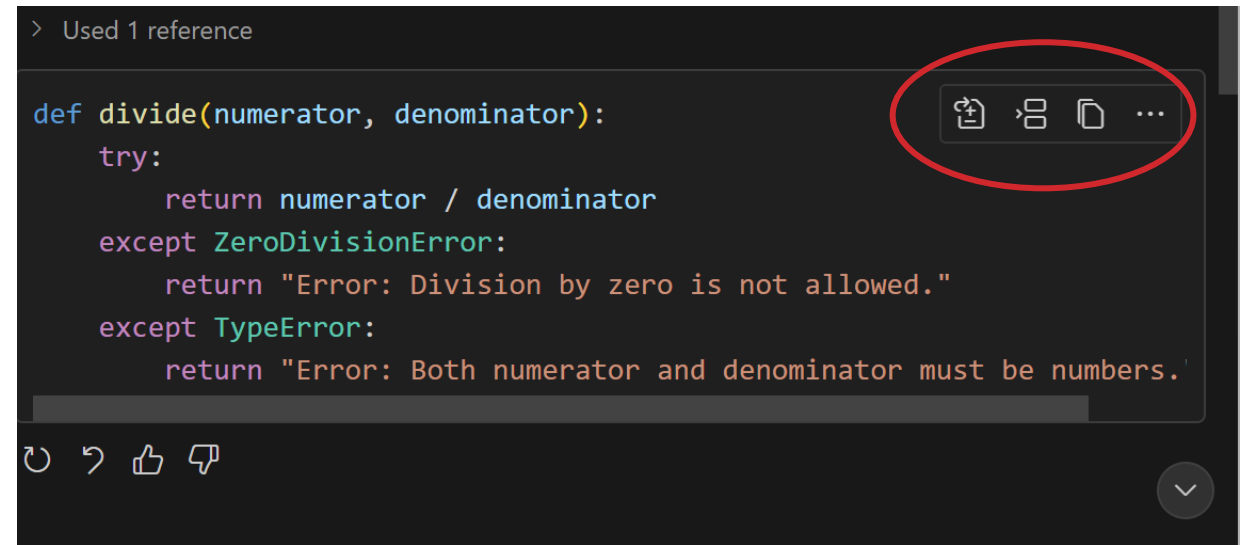
ACTIONS

Apply To: Applies the suggestion directly to the code you had selected in the editor.

Insert At: Lets you insert the suggestion at your current cursor location in the open file.

Copy: Copies the suggestion to your clipboard.

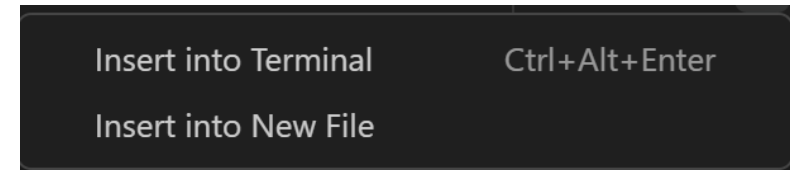
More menu: ...



ACTIONS - 2

Insert into Terminal: display in terminal window.

Insert into New File: save content to a file.

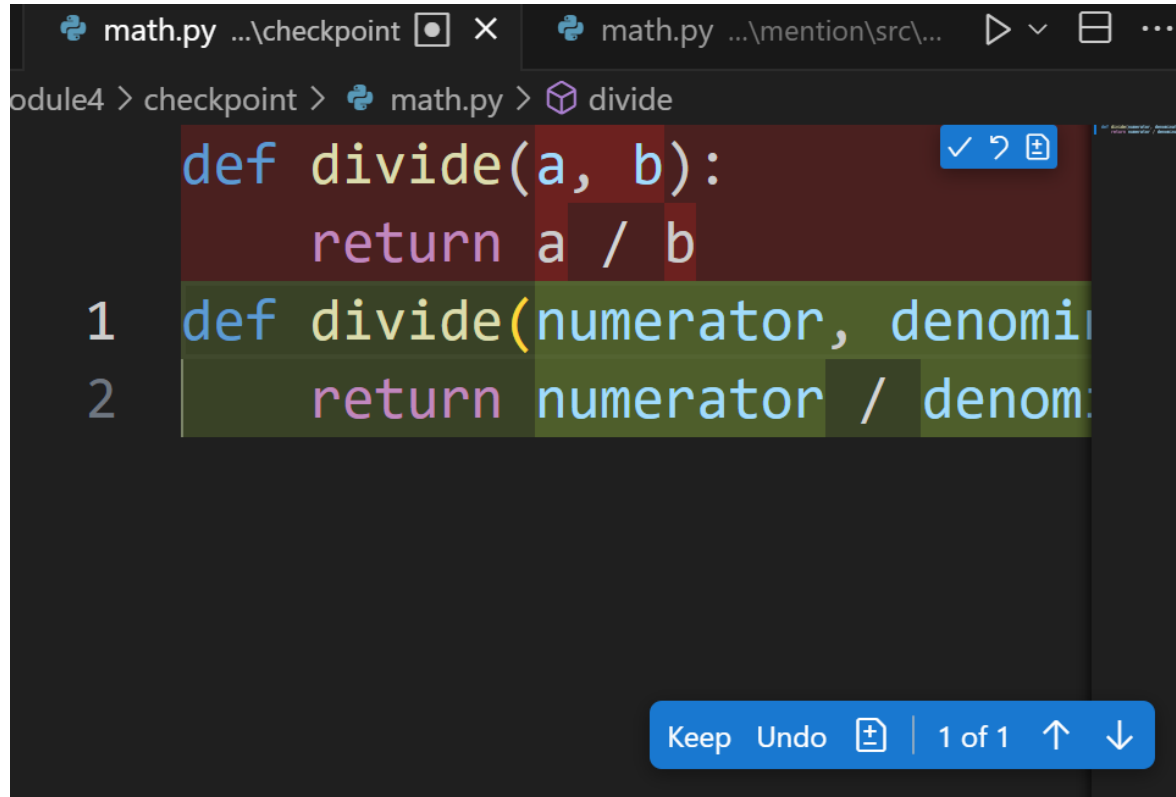


DIFF VIEW

If you decide to apply the suggestion, then the suggestion is presented within a diff view, with the suggestion shown below the original. Here are the allowable actions:

- Keep the suggestion
- Reject the suggestion
- Toggle editor marks
- Scroll through suggestions

KEEP CONTROL



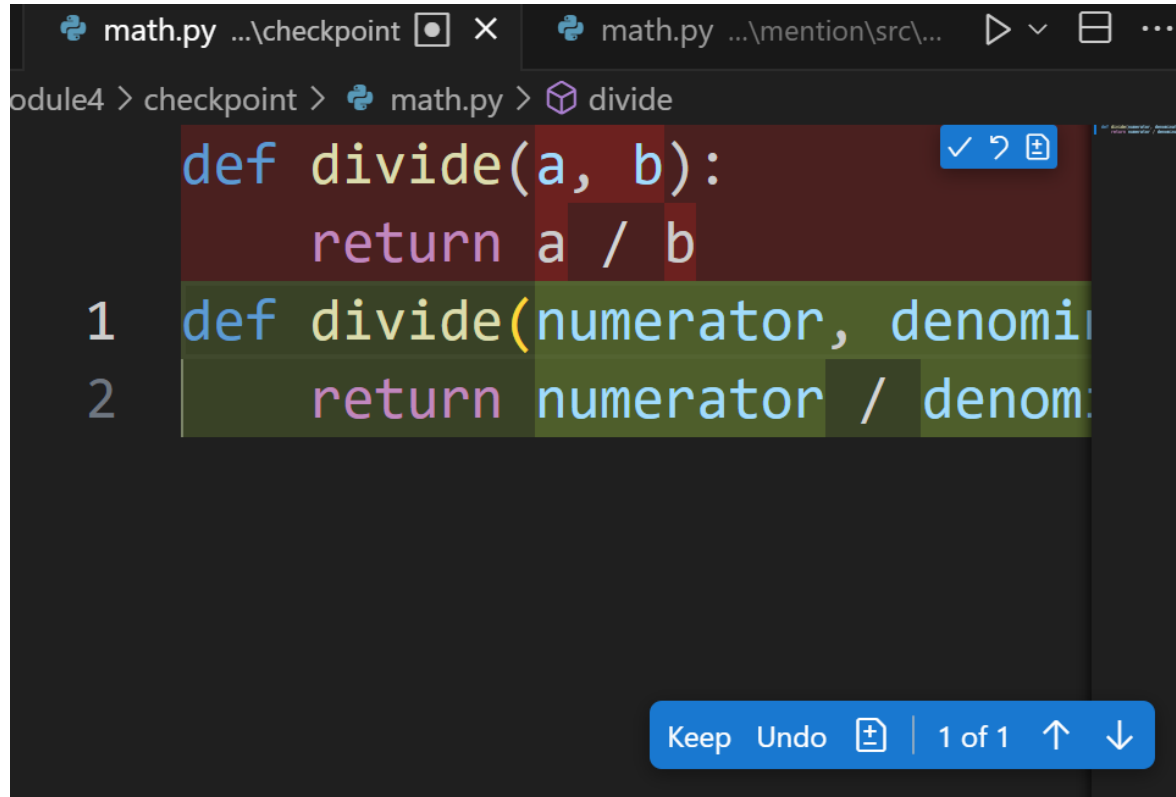
The screenshot shows a code editor with two tabs: 'math.py ...\checkpoint' and 'math.py ...\mention\src\...'. The active tab is 'math.py ...\mention\src\...'. The editor displays a Python function definition: `def divide(a, b):` on line 1 and `return a / b` on line 2. A suggestion box is overlaid on the code, showing the same function definition. The suggestion box has a blue header bar with a checkmark icon, a refresh icon, and a plus icon. Below the suggestion box, there is a blue bar with the text 'Keep Undo' and a plus icon, followed by '1 of 1' and up/down arrow icons.

```
module4 > checkpoint > math.py > divide
def divide(a, b):
    return a / b
1 def divide(numerator, denominator):
2     return numerator / denominator
```

The Keep control allows:

- Keep a suggestion
- Undo (reject) a suggestion
- Arrow through available suggestions

SKIP STAGING



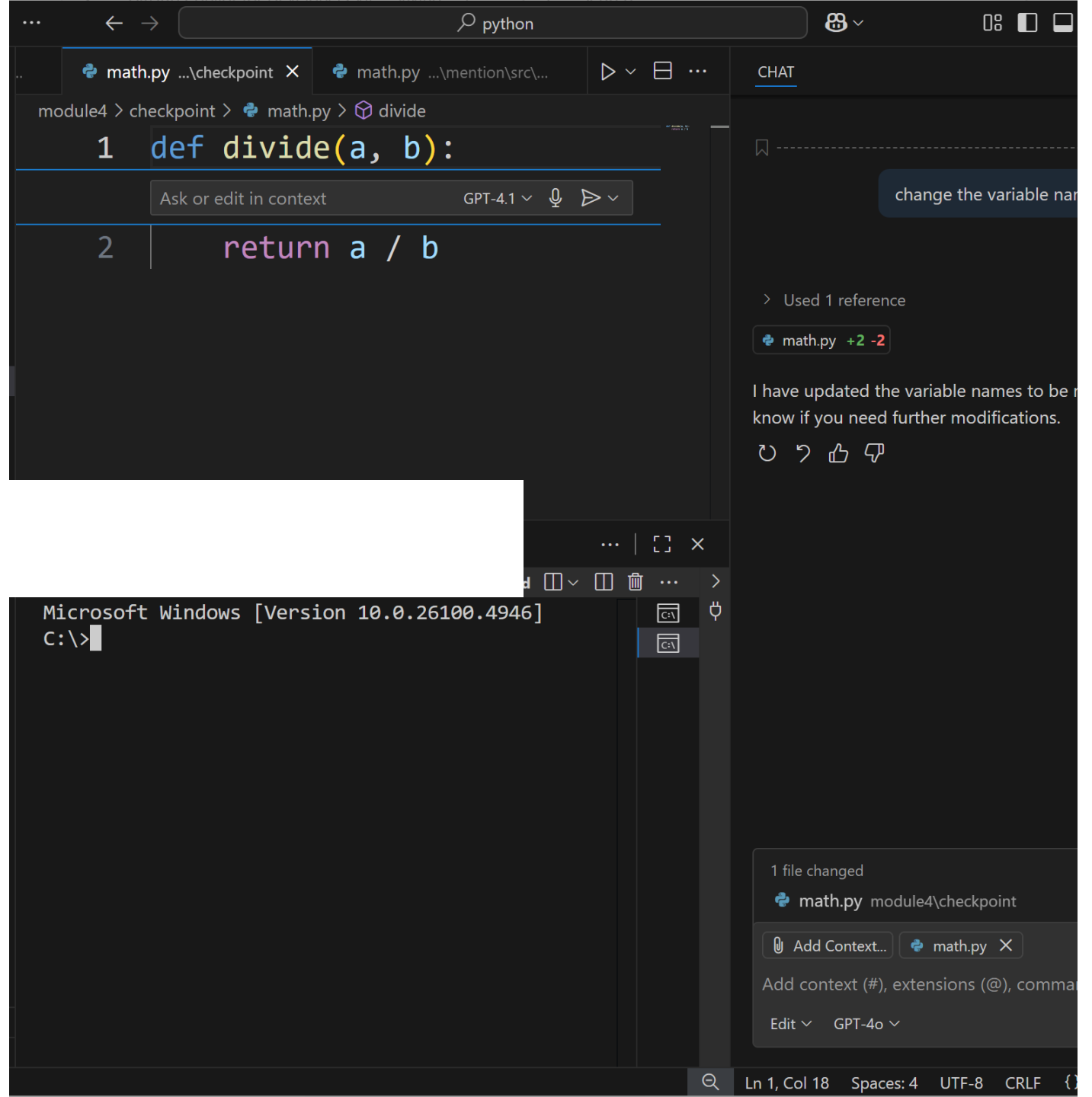
The screenshot shows a code editor with two tabs: 'math.py ...\checkpoint' and 'math.py ...\mention\src\...'. The active tab is 'math.py ...\mention\src\...'. The editor displays a diff view for a function 'divide'. The original code (left) is highlighted in green and the suggested code (right) is highlighted in red. The suggested code is: `def divide(a, b):
 return a / b`. The original code is: `def divide(numerator, denominator):
 return numerator / denominator`. The diff view shows the suggested code is a simplified version of the original. At the bottom, there is a blue bar with buttons: 'Keep', 'Undo', a plus icon, '1 of 1', and up/down arrows.

```
module4 > checkpoint > math.py > divide  
def divide(a, b):  
    return a / b  
1 def divide(numerator, denominator):  
2     return numerator / denominator
```

With the Agent mode, you skip the staging (i.e., Ask mode) and the suggestion is immediately presented in the Diff view.

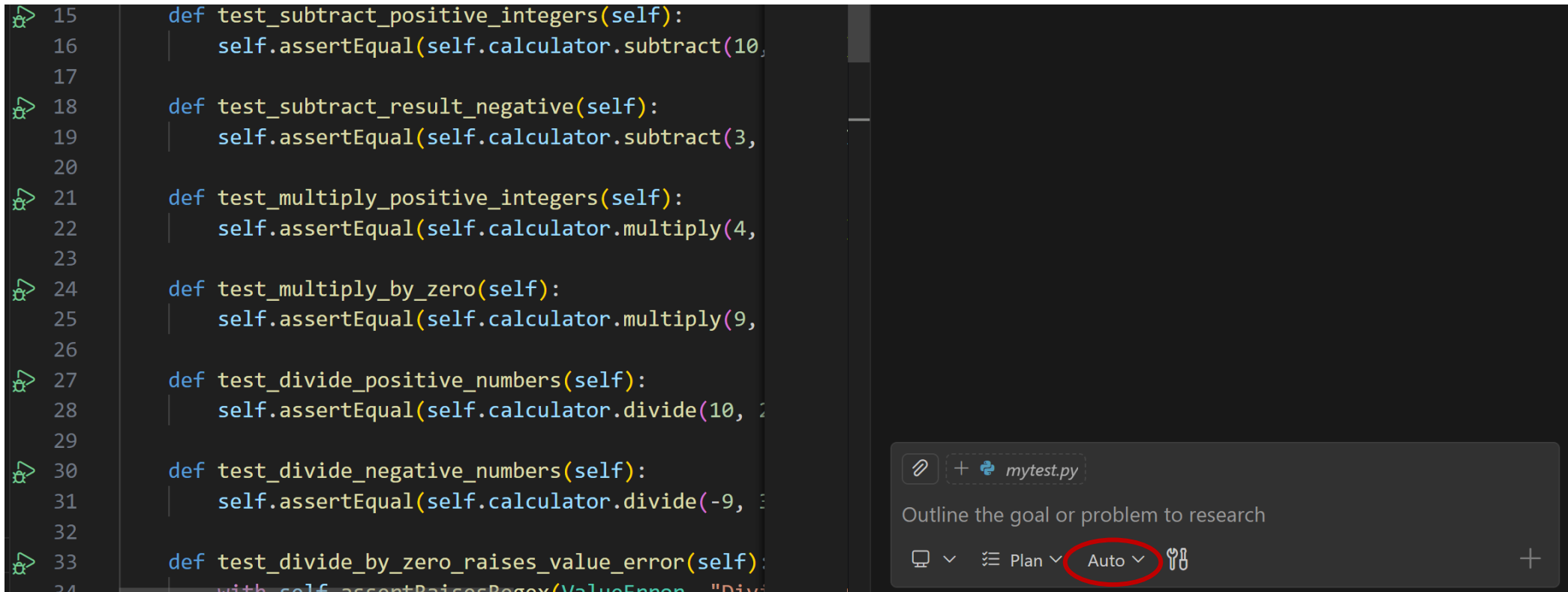
You can then accept or reject the suggestion with the Diff Editor control.

OTHER FEATURES



AUTO LLM

In GitHub Copilot Chat, Auto automatically selects the LLM used to respond to a prompt. Copilot analyzes the request and routes it to an available model based on factors such as task complexity, performance, and system availability, so developers do not need to manually choose a model.



```
15 def test_subtract_positive_integers(self):
16     self.assertEqual(self.calculator.subtract(10, 7), 3)
17
18 def test_subtract_result_negative(self):
19     self.assertEqual(self.calculator.subtract(3, 7), -4)
20
21 def test_multiply_positive_integers(self):
22     self.assertEqual(self.calculator.multiply(4, 5), 20)
23
24 def test_multiply_by_zero(self):
25     self.assertEqual(self.calculator.multiply(9, 0), 0)
26
27 def test_divide_positive_numbers(self):
28     self.assertEqual(self.calculator.divide(10, 2), 5)
29
30 def test_divide_negative_numbers(self):
31     self.assertEqual(self.calculator.divide(-9, 3), -3)
32
33 def test_divide_by_zero_raises_value_error(self):
34     with self.assertRaisesRegex(ValueError, "Divide by zero"):
        self.calculator.divide(10, 0)
```

The screenshot shows a code editor with Python test functions. On the right, the GitHub Copilot Chat interface is visible. It includes a file name 'mytest.py' and a prompt 'Outline the goal or problem to research'. At the bottom of the chat interface, there is a dropdown menu showing 'Auto' as the selected model, which is circled in red. Other icons for settings and a plus sign are also present.

AUTO LLM - 2

Auto May Choose the Wrong Model

Auto may sometimes select a model that is not ideal for the task. For example, it might prioritize a faster general-purpose model instead of a specialized coding model, which can produce different results or cause certain workflows to behave differently.

Overriding Auto

Developers can override Auto by manually selecting a specific model in the Copilot Chat model selector. Choosing a model such as GPT-5.2-Codex or GPT-5.3-Codex forces Copilot to use that model for the session and provides more consistent behavior for coding tasks.

VARIABLES

Copilot fully understands variables you've declared in the codebase.

Example:

```
total = price * quantity
```

If you then write a comment like

```
# calculate tax on total
```

Copilot knows total is a variable and will suggest code like:

```
tax = total * 0.08
```

Copilot's completions are context-aware. It looks at variable names, types, and surrounding code when generating suggestions.

VARIABLES - 2

Copilot can sometimes get confused if you have several variables with similar names (like user, username, userData). To disambiguate in a Copilot prompt, you give it extra clues.

1. Mention the Type or Purpose

If you have both user (funca) and user (funcb), you can guide Copilot by saying:

Instead of:

"Print the user variable"

Say:

"Print the user variable from funcb"

VARIABLES - 3

3. Use Comments Near the Code

Copilot pays attention to comments. Right above where you're working, add:

```
# Use the username variable, not the user object  
print(username)
```

Copilot will now autocomplete correctly because you've clarified in plain English.

VARIABLES - 4

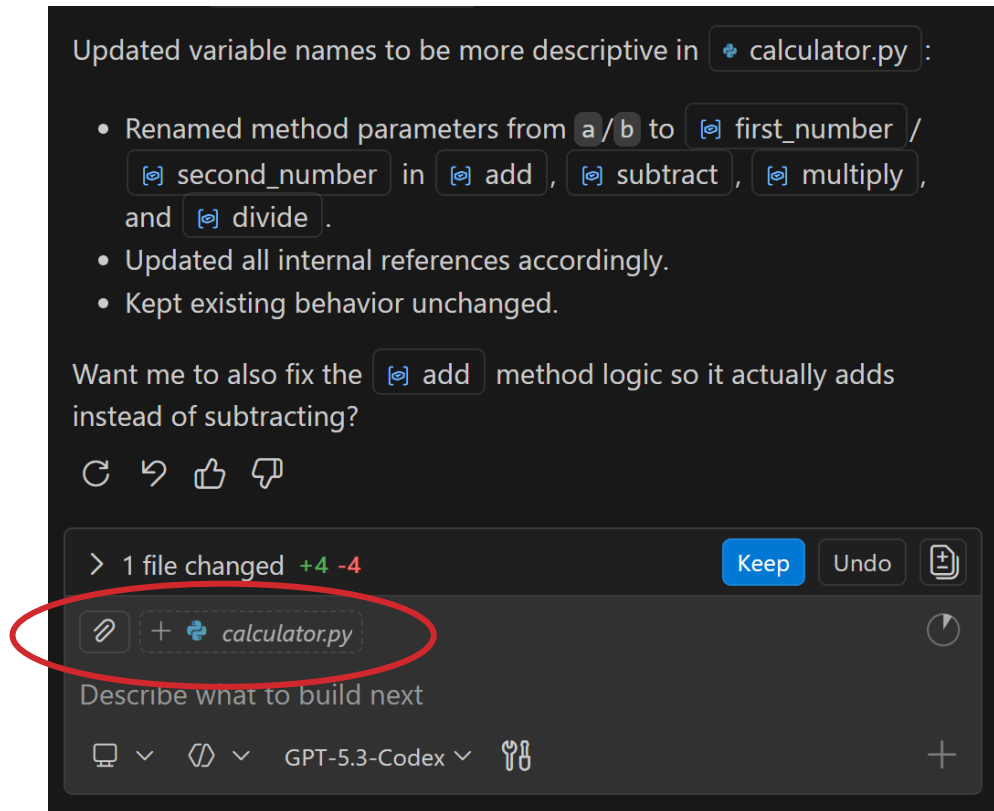
4. Use Explicit Prompt Language

Instead of just saying "fix this," be very specific:

Good: "Loop through `customer_list` (the array of names), not `customer` (the object)."

Bad: "Loop through customers." (too vague)

CONTEXT BUTTON



The Add Context button, which is a paperclip, is provided to add items to the context, such as files.

It will open a list of resources that can be added to the context.

#*MENTION*

The “*#mention*” lets you manually give Copilot Chat extra context by referencing files, your whole workspace, or external resources directly in your prompt.

- Add # followed by the context you want:
- #codebase: makes Copilot aware of the entire workspace/project.
- #file: <*filename*> points to a specific file, e.g. #app.js.

For example, #file:style.css adds style.css to the context.

MENTION EXAMPLE

Here's a short example of using file path in Copilot Chat: Here is the project structure:

```
project/  
  src/  
    utils/  
      math.py  
  styles/  
    main.css
```

Prompt in Copilot Chat:

Explain how the functions are organized #file:src/utils/math.py.

Suggest improvements to button styling #file:styles/main.css.

Result:

Copilot looks at math.py to describe your utility functions. It also uses main.css to suggest design tweaks.

#SELECTION

```
def process_data(records):  
    cleaned = []  
    for r in records:  
        if r is not None and r != "" and r.strip() != "":  
            cleaned.append(r.strip().lower())  
        # ... more logic for filtering, transforming, exporting ...  
    return cleaned
```

You can focus suggestions on selected code with #selection, such as a portion of a function.

For example:

#selection optimize

CODEBASE

When you type `#codebase` in Copilot Chat, you're explicitly telling Copilot:

"Use my entire project as context for this request."

Normally, Copilot only looks at:

- The current file,
- Recently opened files, or

By mentioning `#codebase`, you override that and attach the whole project folder (everything in your workspace index) as extra context.

COMMANDS

You can use the slash commands to steer Copilot into specific behaviors.

Common Copilot Chat Commands

- `/fix`: Suggest fixes for errors or issues in the selected code.
- `/explain`: Explain what the selected code does in plain language.
- `/tests`: Generate unit tests for the selected code.

APPLY CHANGE CONTROL

The problem in the previous code is that the change causes an inconsistency. It maybe minor but could cause confusion, nonetheless.

The solution is the →| icon, which automatically appears when needed, and the down arrow appear together to help you review and control a suggested edit. The →| symbol is the apply-change control: it visually indicates that the suggested code, on the right, will replace the existing code only if you explicitly approve it.

```
1  #factorial that list values between 10 and 500
2  def filter_values(values):
3  →|  """Filter a list of integers to include only those between 10 and 500."""
4     return [v for v in values if 10 <= v <= 700]  ↳ 700
```

/EXPLAIN

```
def calculate_prime(n: int) -> bool:
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Explains the code you have selected or the file that's active in the editor. Great for onboarding to an unfamiliar area or validating logic before changes.

/FIX

```
def divide_numbers(a, b):  
    return a / b
```

```
print(divide_numbers(10,  
0))
```

Analyzes the selected code and proposes a patch for errors or suspicious patterns. You'll get a diff you can accept or tweak.

/TESTS

```
def factorial(n: int) -> int:  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Generates unit tests for your active file or selection, including typical and edge cases. You can specify frameworks or constraints.

/FIXTESTFAILURE

```
def parse_int_list(s: str) -> list[int]:  
    return [int(x) for x in s.split("")]
```

Targets a failing test and suggests code changes to make it pass. Use when your test runner (or CI) is red and you want a focused remedy.

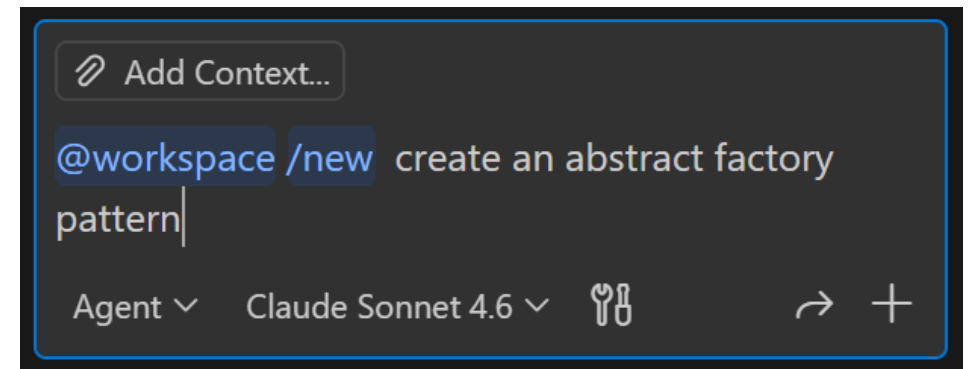
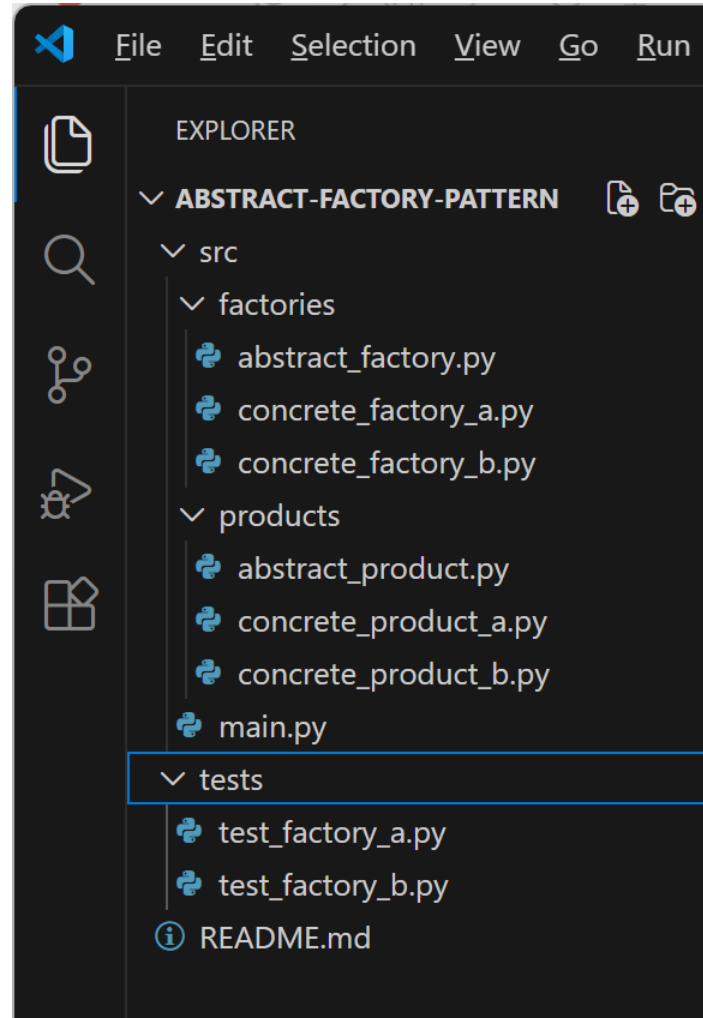
/NEW



The `/new` command is used to scaffold new code or project structure in your workspace. It must be used with an item like `@workspace`, followed by a description of what to generate (e.g., `@workspace /new Create a Python REST API`).

The `/new` command does not reset chat history. It creates new files or starter code based on your instructions.

/NEW - 2



/CLEAR

In GitHub Copilot Chat, the `/clear` command is used to reset the current chat session. When you enter `/clear`, Copilot removes the existing conversation history from the active chat window, allowing you to start fresh without prior context influencing responses.

This command only clears the chat memory, not your workspace, files, or project. It is useful when switching topics, correcting earlier misunderstandings, or ensuring previous prompts do not affect new requests.

POP QUIZ: ARE THEY THE SAME?

Is the /fix command and fix prompt the same?



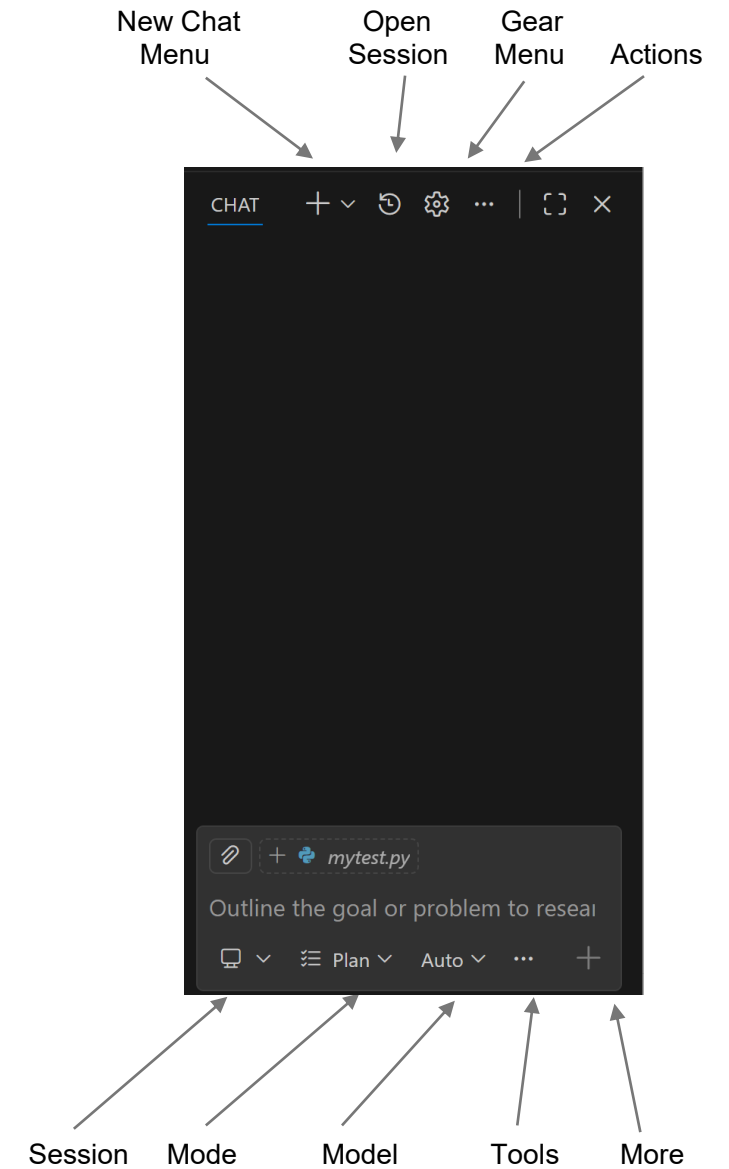
10 MINUTES

FIX VERSUS /FIX

Slash commands like `/fix` and `/explain` are basically shortcuts that tell GitHub Copilot what role you want it to play. When you type `/fix`, you're saying "this code is broken — please correct it," and Copilot tries to make the smallest, safest changes it can. When you type `/explain`, you're saying "don't change anything — just tell me what this does," so Copilot focuses on describing the code instead of rewriting it.

If you type "fix this" or "explain this" in plain English, Copilot is usually smart enough to figure out what you mean – the result often appears the same. The slash commands make Copilot more careful and predictable.

CHAT INPUT BOX



SHORTCUT KEYS

Action	Windows/Linux	macOS
Open Copilot Chat panel	Ctrl + I	Cmd + I
Open Inline Chat (editor overlay)	Alt + Enter	Option + Enter
Accept suggestion	Tab	Tab
Cycle inline suggestions	Alt + [/ Alt +]	Option + [/]
Submit prompt in chat	Enter	Enter
Newline in chat prompt	Shift + Enter	Shift + Enter
Open chat with context	Ctrl + Shift + \\	Cmd + Shift + \\
Open chat from selection	Right-click > Copilot: Explain this code	Same

RETROSPECTIVE



This module explains how to use GitHub Copilot Chat in VS Code across Chat View, Inline Chat, and Quick Chat, and it helpfully walks through the Copilot Chat/Completions menus (status, open completions panel, disable completions, shortcuts, settings, diagnostics/logs).

It also covers modes (Ask, Edit, Agent), core slash commands (/fix, /tests, /explain, /edit), checkpoints and the action bar

LAB 4 – MOBY DICK



The goal of this lab is file processing – starting with Moby Dick.

MOBY DICK

Call me Ishmael. Some years ago—never mind how long precisely—having little or no money in my purse, and nothing particular to interest me on shore, I thought I would sail about a little and see the watery part of the world. It is a way I have of driving off the spleen and regulating the circulation. Whenever I find myself growing grim about the mouth; whenever it is a damp, drizzly November in my soul; whenever I find myself involuntarily pausing before coffin warehouses, and bringing up the rear of every funeral I meet; and especially whenever my hypos get such an upper hand of me, that it requires a strong moral principle to prevent me from deliberately stepping into the street, and methodically knocking people's hats off—then, I account it high time to get to sea as soon as I can. This is my substitute for pistol and ball...

SETUP

- Open VS Code
- Find and open the Lab4 folder. Within that folder, create a new folder: mobydick.
- Alternately, create a new folder called mobydick somewhere convenient.
- Within the mobydick folder, create a new file: textsummary.py.

FILE PROCESSING

Goal: Use Copilot Chat to perform text processing

1. Open textsummary.py.
2. Add a comment:
 - # Read a file and summarize the number of lines, words, and characters
 - Accept Copilot's function suggestion.
3. Copy mobydick.txt from the Module4 directory for sample files to the mobydick directory.
4. Run the script and verify output.

REVISING

Ask Copilot Chat:

- Suggest improvements for handling empty files
 - Add error handling if mobydick.txt is missing
 - Display the run time of the script in milliseconds
2. Refactor the script to support command-line arguments so different file names can be provided.
 3. Use the `/tests` command to create unit tests, using the `unittest` library.
 4. Run the unit test(s)

Lab completed

“Develop a passion for learning.”



COMMON TASKS

ERROR HANDLING, SYNTAX CHECKING, AND MORE



INTRODUCTION



This module introduces GitHub Copilot Chat as a practical debugging partner: it spots and explains common mistakes (e.g., removing items while iterating a list), scaffolds real error handling with clear try/except patterns, helps fix syntax slips across languages, and—most importantly—catches “silent” logic errors that still run but produce wrong results.

EXAMPLE

```
def remove_negatives(numbers):  
    for n in numbers:  
        if n < 0:  
            numbers.remove(n)  
    return numbers
```

```
data = [3, -1, 5, -2, 7]  
print(remove_negatives(data))
```

POP QUIZ: PROBLEM

Does the previous code have a syntactical or logical error?



10 MINUTES

COPILLOT CHAT – THE FIX

```
def remove_negatives(numbers):  
    return [n for n in numbers if n >= 0]  
  
data = [1, -1, -2, 3, -4, -5, 6]  
  
print(remove_negatives(data))
```

Yes, there is an unseen error.

The issue is that you're modifying the list while iterating over it. When you remove an element during iteration, Python shifts the remaining items left, so the loop skips over some elements.

ERROR HANDLING



Error handling is an area where Copilot shines by quickly generating try-except or try-catch structures. Many developers overlook robust error handling in the early stages of coding.

Copilot can encourage better practices by suggesting error handling blocks automatically.

EXAMPLE

```
def compute_average(filename):  
    with open(filename, "r") as f:  
        lines = f.readlines()  
        total = sum(int(line.strip()) for line in lines)  
        avg = total / len(lines)  
    return avg
```

WITH ERROR HANDLING

```
def compute_average(filename):
    try:
        with open(filename, "r") as f:
            lines = f.readlines()
            if not lines:
                raise ValueError("The file is empty.")
            total = sum(int(line.strip()) for line in lines)
            avg = total / len(lines)
            return avg
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
    except ValueError as ve:
        print(f"Error: {ve}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
```

Error handling is an example of repetitive code, such as try-except or try-catch structures, that GitHub Copilot handles exceptionally well.

SYNTAX ERRORS

Syntax errors are among the most common issues developers encounter, from missing colons in Python to mismatched brackets in JavaScript. Copilot helps resolve these by correcting the syntax based on language rules and best practices. For example, if you mistakenly write `if (x > 10)` in Python, Copilot may recognize the missing colon and suggest `if x > 10:`. Similarly, if you open an HTML tag and forget to close it, Copilot often generates the closing tag for you.

Another example is in Java: if you write `System.out.println("Hello World")` without a semicolon, Copilot can highlight the issue by suggesting the corrected line. These corrections are particularly useful for beginners or for developers working in a language they do not use frequently.

MUST BE GOOD!



This is especially important for syntax errors that do not cause compilation errors. In languages like Python, certain mistakes still produce valid code, so the program runs but behaves incorrectly.

These errors are much harder to detect because they don't trigger obvious failures — they just produce wrong results.

EXAMPLE

```
def check_value(x):  
    message = ""  
    magic_number = 10  
    if (y := magic_number):  
        if x > y:  
            message = f"x is greater than {y}"  
        elif x < y:  
            message = f"x is less than {y}"  
        else:  
            message = f"x is equal to {y}"  
    else:  
        message = "Condition failed"  
    return message
```

```
print(check_value(5))  
print(check_value(10))  
print(check_value(20))
```

FIXED

```
def check_value(x):  
    message = ""  
    magic_number = 10  
  
    if x > magic_number:  
        message = f"x is greater than {magic_number}"  
    elif x < magic_number:  
        message = f"x is less than {magic_number}"  
    else:  
        message = f"x is equal to {magic_number}"  
  
    return message
```

GitHub Copilot would quickly find and fixed the syntax / logical error.

RETROSPECTIVE



This module clearly shows how Copilot Chat helps you catch and fix everyday mistakes: mutating a list while iterating, undefined or incorrect calls, and common syntax slips; it then levels up practice with real error handling (file I/O guards, empty-file checks via try/except) and highlights “silent” logic errors

LAB 5 – PERFORMANCE



This module focuses on debugging application performance.



OVERVIEW

The module primarily focuses on using GitHub Copilot to identify and fix common programming mistakes, such as logic errors, missing error handling, and syntax problems. Performance optimization is not specifically covered in this module. However, GitHub Copilot can also be a valuable tool for analyzing and improving the performance of algorithms.

In this lab, you will use Copilot to examine an inefficient algorithm, understand its time complexity, and develop a more efficient implementation. This exercise demonstrates how Copilot can assist not only with debugging correctness issues but also with identifying opportunities to improve algorithm performance.

SAMPLE CODE

The `find_duplicates` function finds values that appear more than once in a list. It compares elements using nested loops and adds matches to a duplicates list if they have not already been recorded.

```
def find_duplicates(items):  
    duplicates = []  
    for i in range(len(items)):  
        for j in range(i + 1, len(items)):  
            if items[i] == items[j] and items[i] not in duplicates:  
                duplicates.append(items[i])  
    return duplicates
```

SETUP

- Open VS Code
- Open the Lab5 folder.
- Open duplicates.py in a VS Code source window.

ANALYZE THE ALGORITHM

Review the provided code.

Tasks:

- For the `find_duplicates` function, determine what the algorithm is intended to do.
- Estimate the algorithm's time complexity.

Use a Copilot prompt that asks the assistant to:

- Analyze the algorithm (`find_duplicates`)
- Explain how it works
- Estimate its time complexity.

IMPROVE THE ALGORITHM

Refactor the algorithm to improve performance.

Tasks:

- Identify the inefficient portion of the code.
- Implement a more efficient approach.

Use a prompt that asks Copilot to:

- Propose a more efficient implementation
- Rewrite the algorithm with better performance.

EVALUATE FLEXIBILITY

Determine whether the improved algorithm works for different input types.

Tasks:

- Consider whether the algorithm depends on numeric operations.
- Determine whether it also works for strings.

Use a prompt that asks Copilot to:

- Evaluate whether the algorithm works for both numeric and string inputs
- Explain the reasoning.

GENERICIS

In Python, generics provide limited practical benefit at runtime, because Python's dynamic typing already allows the function to work with many types (such as integers, strings, or tuples). The algorithm only relies on equality comparison, so it naturally works with any compatible type.

However, using generics can still provide some advantages:

- Improved readability – It clearly indicates that the function is designed to work with any type of list elements.
- Better static type checking – Tools like mypy or Pyright can verify correct usage.
- Better IDE support – Editors can provide more accurate type hints and warnings.

GENERALIZE THE IMPLEMENTATION

Improve the design so the function clearly supports multiple data types.

Tasks:

- Refactor the function using type hints or generics.
- Improve readability and maintainability.

Use a prompt that asks Copilot to:

- Refactor the function so it works with generic types
- Apply Python type hints.

COMPARE THE IMPLEMENTATIONS

Evaluate both versions of the algorithm.

Tasks:

- Compare time complexity.
- Explain the performance improvement.

Use a prompt that asks Copilot to:

- Compare the complexity of both implementations
- Explain when the optimized version is preferable.

EXAMPLE PROMPTS

Compare your prompts with these minimal solutions.

- @selection explain and analyze the time complexity of this algorithm
- rewrite this algorithm with better time complexity
- would this algorithm work for strings as well as numbers
- rewrite this function using Python generics
- compare the complexity of the original and optimized implementations

BONUS - BENCHMARKING



Create actual benchmarking metrics for the `find_duplicates` function.

Tasks:

- Select the function.
- Ask for benchmarking data

Use a prompt that asks Copilot to:

- Create and display the performance data for the selected function.
- Answer all follow-up questions.

Lab completed

“Develop a passion for learning.”



CODE DOCUMENTATION



INTRODUCTION



This module is a hands-on guide to using GitHub Copilot Chat for fast, consistent documentation and clearer code understanding. You'll practice context-aware generation of docstrings and README documentation.

CONTEXT AWARE



Copilot now uses context-aware generation, meaning it considers the function signature, surrounding code, and even related files to suggest meaningful comments or documentation.

For example, when you begin typing a docstring in Python or a Javadoc comment in Java, Copilot can propose a full template that includes a description, parameter explanations, and return values.

HATE DOCUMENTATION

Many developers find documentation less engaging than writing code, especially under tight deadlines. Repetitive boilerplate and the effort required to explain technical details clearly can make documentation feel like overhead rather than value.

As a solution, many promote “self-documenting code.” Clear function names, descriptive variables, and clean structure reduce the need for excessive comments. However, readable code alone cannot replace higher-level documentation such as architecture decisions, business rules, and API contracts.

DOCUMENTATION IS CODE QUALITY



Documentation is part of software quality, not an afterthought. Clear documentation reduces cognitive load, accelerates onboarding, and preserves architectural intent over time.

Strong documentation improves collaboration and long-term maintainability, while weak documentation increases technical debt and slows debugging and feature delivery.

WHERE TO DOCUMENT

GitHub Copilot accelerates documentation at multiple levels of a project.

- Function and class docstrings
- Module summaries
- Project README files
- API reference documentation
- Refactoring explanations

Copilot acts as a drafting assistant that speeds up initial creation. Human review ensures accuracy and business alignment.

HIGH QUALITY COMMENTS

Copilot can generate structured docstrings directly from function definitions. When prompted clearly, it can include parameter descriptions, return values, assumptions, and validation rules.

To improve output quality:

- Specify documentation style (Google, NumPy, Javadoc)
- Request edge cases
- Ask for usage examples

Clear prompts produce clearer documentation.

REVERSE ENGINEERING

Copilot is especially useful for explaining unfamiliar or complex code. It can summarize algorithms, describe control flow, and highlight assumptions embedded in the logic.

This is particularly valuable when working with legacy systems or onboarding new team members. Generated explanations should be refined to reflect actual business intent.

README

Copilot can scaffold project-level documentation quickly, including setup instructions, configuration details, and usage examples.

- Providing context improves quality:
- Target audience
- Deployment environment
- Dependencies and constraints

Well-prompted outputs reduce the time spent drafting boilerplate documentation.

API DOCUMENTATION

Copilot can generate endpoint descriptions, example payloads, and structured API documentation. This accelerates internal and external documentation workflows.

However, developers must validate:

- Field accuracy
- Data types
- Authentication rules
- Error codes

Copilot documents — developers verify.

ARCHITECTURAL SUMMARIES

For larger systems, Copilot can summarize repository structure and describe service interactions. It can help draft architecture overviews that are useful in design reviews or knowledge transfer sessions.

While helpful, it cannot fully infer business rationale behind architectural decisions. That context must be added manually.

DOCUMENTATION DURING REFACTORING

When code changes, documentation often becomes outdated. Copilot can regenerate docstrings, update parameter descriptions, and suggest improved naming conventions.

A strong workflow includes:

- Refactor code
- Regenerate documentation
- Compare for accuracy
- Manually validate edge cases

This reduces documentation drift.

RISKS AND LIMITATIONS

AI-generated documentation must always be reviewed. Copilot may hallucinate behaviors, misinterpret edge cases, or describe logic that does not exist.

It does not understand compliance requirements, performance constraints, or nuanced business rules unless explicitly prompted.

Documentation generated by AI is a draft — not authoritative.

ADVANCED PROMPTS

High-quality documentation depends on high-quality prompts. Developers should request structure, specify format, and ask for examples and edge cases.

For example:

Generate a NumPy-style docstring including validation rules and a usage example.

The more specific the instruction, the more precise and useful the documentation will be.

DOCUMENTATION - BEST PRACTICES

- Write the docstring first: start a docstring/Javadoc header and let Copilot scaffold params/returns, then edit for brevity.
- Constrain scope with context: use `#selection` (only selected code) or `#file` to feed the right amount of code; smaller context → tighter docs.
- Seed with style hints: a first line like “Use imperative mood, ≤ 80 -char lines, include examples” steers output.

DOCUMENTATION - BEST PRACTICES - 2

docstring:

- `#selection #file:utils.py` Create a concise docstring: 1-line summary, args, returns, edge cases. Use imperative mood.

README snippet:

- `@workspace` Generate a minimal README section: install, run, test. Keep steps numbered and under 7 lines.

PROMPT - BEST PRACTICES - 3

Template reuse via prompt files:

- Keep a docstyle.md (tone, sections, examples). Reference it in prompts so Copilot stays consistent.

Ask for contrasts/examples:

- “Add a 1-line example and a common pitfall” produces practical docs without bloat.

RETROSPECTIVE



This module showed how to use GitHub Copilot for fast, consistent documentation: it demonstrated context-aware generation of docstrings that consider signatures, surrounding code, and related files.

Lab completed

“Develop a passion for learning.”



ADVANCED

SETUP, REFERENCE, AND MORE



INTRODUCTION



In this module, we examine how GitHub Copilot reasons about your codebase. You'll learn the differences between local, workspace, and remote indexing, how context depth affects suggestion quality, and how premium requests are measured. We'll also review how @ references and # injections shape Copilot's understanding.

By the end, you'll understand how Copilot "sees" your environment, what improves suggestion accuracy, when to refresh indexing, and how to manage premium usage effectively.

INTERACTIONS

Every time you interact with Copilot—whether via chat, agent mode, code review, or extensions—you're making a request. Some of these are considered premium requests, meaning they charged against monthly quota.



INLINE CODE COMPLETION



Inline code completion does not incur any extra charge. There are no usage-based fees specifically for accepting inline suggestions.

PREMIUM REQUESTS

Premium requests are typically:

- Copilot Chat (especially for longer or more complex prompts)
- Ask for multi-file reasoning, refactors, or explanations
- Run agent-style tasks (e.g., “analyze this codebase,” “fix all related issues”)
- Use advanced models instead of the default lightweight completion model
- Request code review, test generation, or documentation generation that requires deeper analysis

PREMIUM REQUESTS - 2

How Many Premium Requests Does Each Plan Include?

Here’s how the plans stack up:

Plan	Monthly Premium Requests Included	Cost for Extra Requests
Free	50 per month	Cannot purchase more—must upgrade GitHub Docs +3 Reddit
Pro	300 per month	\$0.04 USD per extra request GitHub Docs +2
Pro+	1,500 per month	\$0.04 USD per extra request GitHub Docs +2

PREMIUM REQUESTS - 3

Here is a table showing monthly versus discounted annual pricing.

Plan	Monthly Cost (USD)	Annual Cost (USD)	Annual Savings
Free	\$0	\$0	\$0
Pro	\$10	\$100	\$20
Pro+	\$39	\$390	\$78

CHAT MESSAGE



A chat message is the unit of interaction inside Copilot Chat.

It's literally each prompt you type and each AI reply.

Example:

You: "Explain this function." is 1 chat message

Copilot reply is 1 chat message

Total = 2 chat messages for that exchange.

CHAT MESSAGE - 2



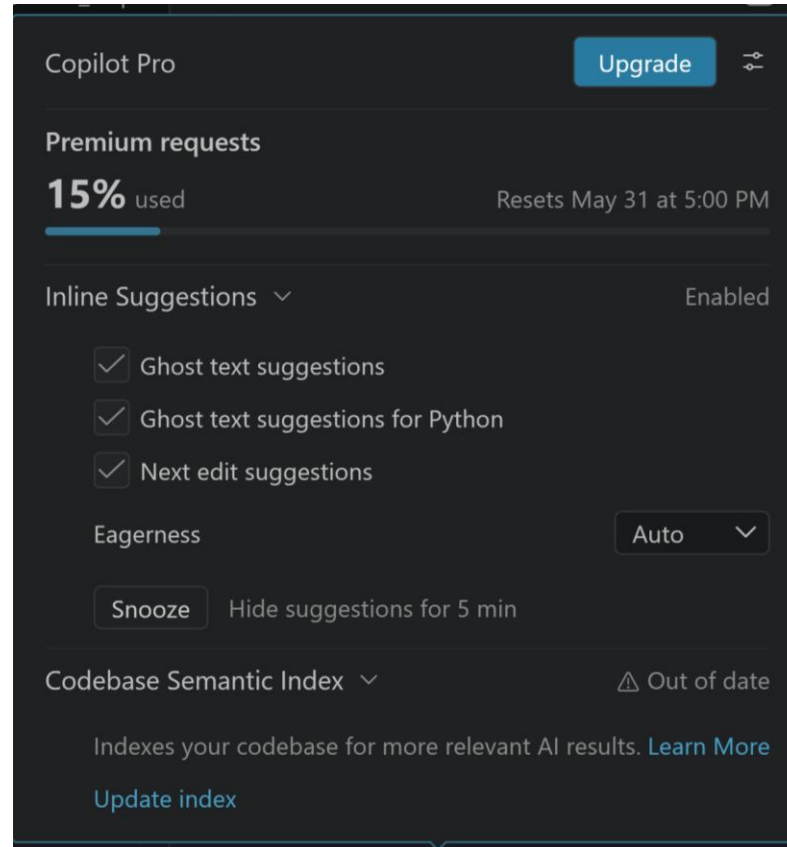
You type chat messages, but GitHub Copilot measures usage by whether a message triggers a premium request behind the scenes. Many chat messages do not incur a premium request (for example, short or simple interactions), while some chat messages do if they require deeper reasoning, larger context, or advanced models. So a chat message is just the interface; a premium request is the billing unit.

CHAT MESSAGE VERSUS PREMIUM REQUEST - 2

Comparison Table

Feature	Chat Message?	Premium Request?	Explanation
Inline code completion (ghost text / suggestion while typing)	No	No	Standard Copilot feature, included in base subscription; unlimited.
Copilot Chat prompt (your input)	Yes	Yes	Each prompt is a chat message and consumes premium request units.
Copilot Chat reply (AI output)	Yes	Yes	Each reply is a chat message and also consumes premium request units.
Copilot Agents (/fix, /new, /testfailure, etc.)	No	Yes	Invokes advanced AI features; not a chat but billed as premium requests.
PR review / summary	No	Yes	Uses Copilot to analyze diffs; premium request consumption.
Copilot Extensions (queries to third-party services via Copilot)	No	Yes	Consumes premium requests, but not counted as chat messages.

MANAGE PAID PREMIUM REQUESTS



From here, you can view premium requests or chat messages.

TOKEN



Think of Tokens as "Word Pieces"

When you type something into GitHub Copilot, it doesn't see full sentences the way humans do.

It breaks everything into small chunks called tokens.

A token can be:

- A word
- Part of a word
- A number
- A punctuation mark

TOKEN - 2

Instead of reading:

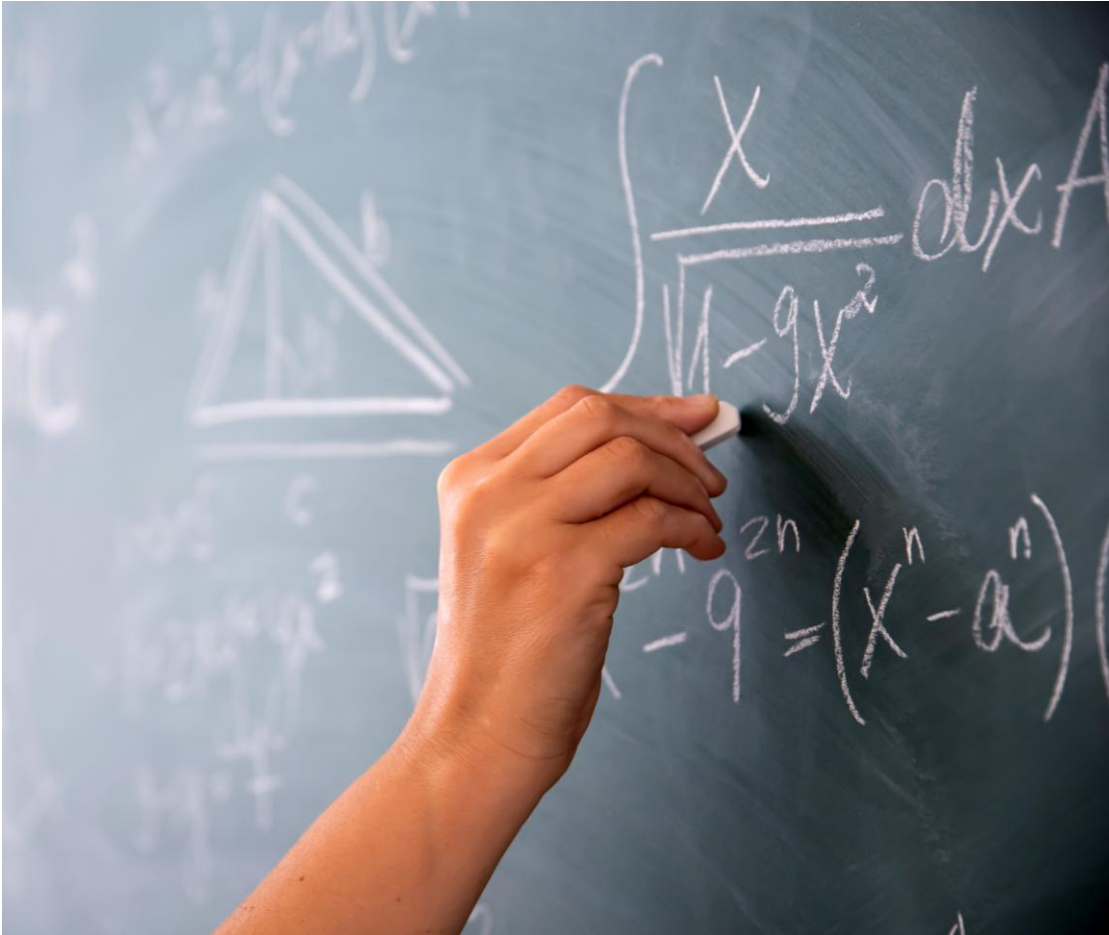
"Create a function that calculates interest."

Copilot sees something more like:

"Create" | " a" | " function" | " that" | " calculates" | " interest" | "."

Each of those pieces is a token.

WHY IMPORTANT

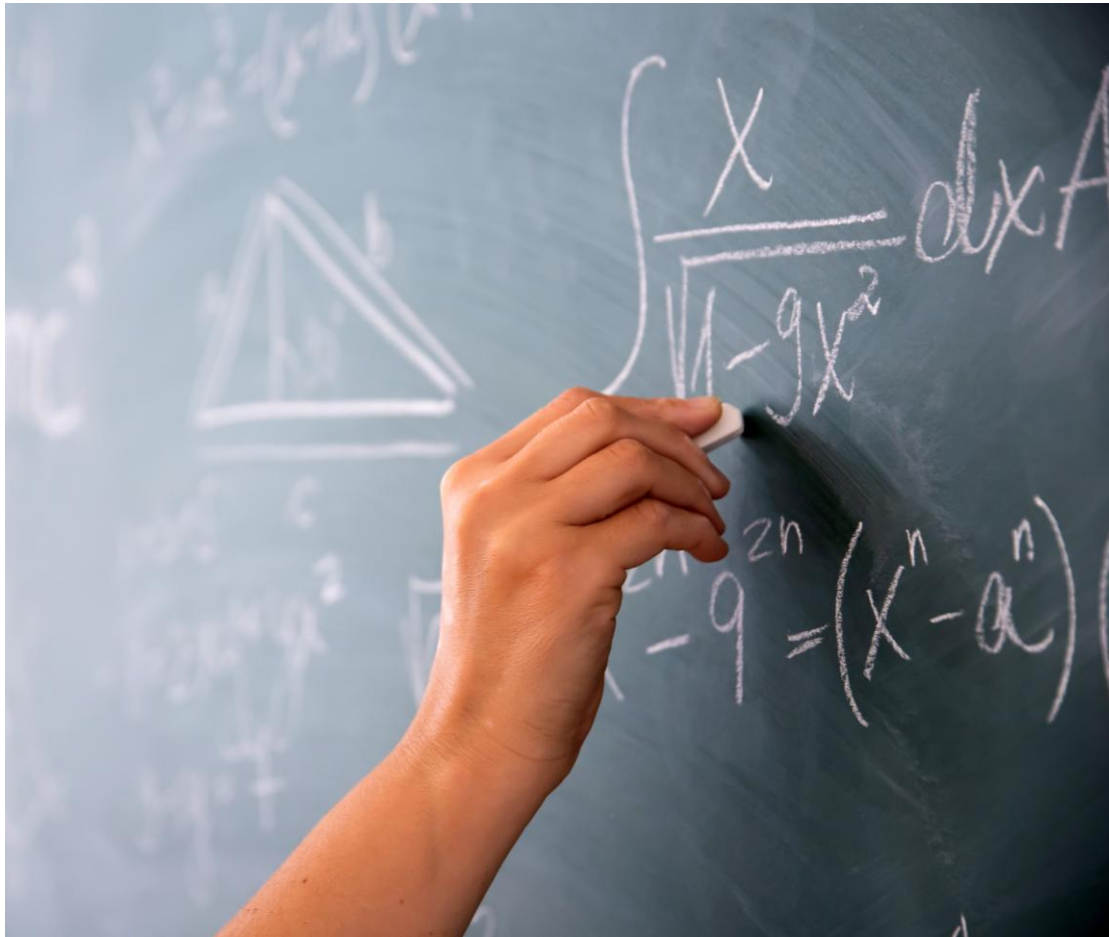


Copilot has a limit on how many tokens it can look at.

This is called the context window. Envision Copilot has a whiteboard.

- The whiteboard can only hold a certain number of word pieces.
- If too much text is written, older parts get erased.

WHY IMPORTANT - 2



Otherwise, tokens determine:

- How much of your file Copilot can "see"
- How much of your prompt it remembers
- How long of a response it can generate

¾ ESTIMATE

You can estimate tokens as:

- 1 token $\approx$ ¾ of a word
- 100 tokens $\approx$ 75 words
- 1,000 tokens $\approx$ 750 words

Code often uses more tokens than plain language because of symbols and formatting.

BOTTOM LINE

Tokens affect:

- Why Copilot sometimes “forgets” earlier parts of a conversation
- Why very long files reduce answer quality
- Why shorter, focused prompts often work better

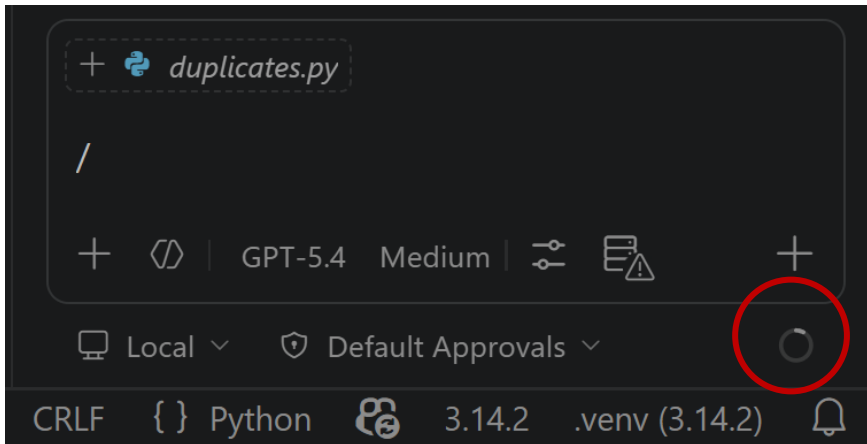
If Copilot seems confused, it’s often because:

- Too many tokens are being used
- The important context got pushed out

REDUCE TOKENS

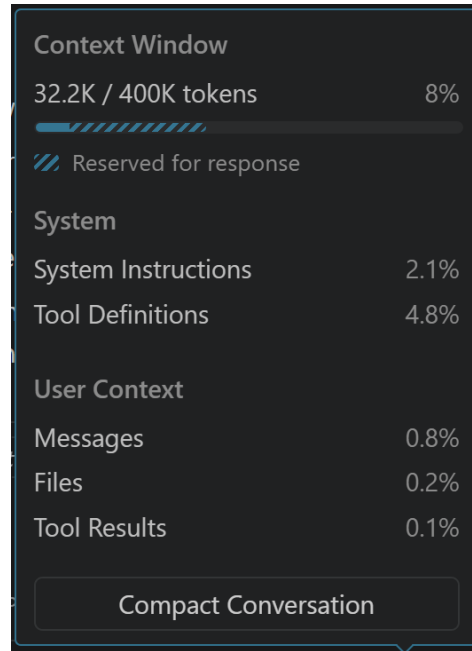
- Keep prompts focused – Ask one clear, specific question instead of multiple tasks at once.
- Close unused files – Only keep relevant files open in your editor.
- Avoid large copy/paste blocks – Share only the part that matters.
- Start a new chat – Resetting clears accumulated conversation history.
- Trim long instruction or rule files – Keep custom instructions concise and relevant.

CONTEXT WINDOW



Open GitHub Copilot Chat in VS Code. At the top of the chat input box, locate the context list. Next to it is a token usage indicator. Click this indicator to open the Context Window panel.

CONTEXT WINDOW - 2



The panel displays:

- Total tokens used vs. available
- System instructions usage
- Tool definitions
- User messages
- Files included in the context

EXPLANATION - INDEXING

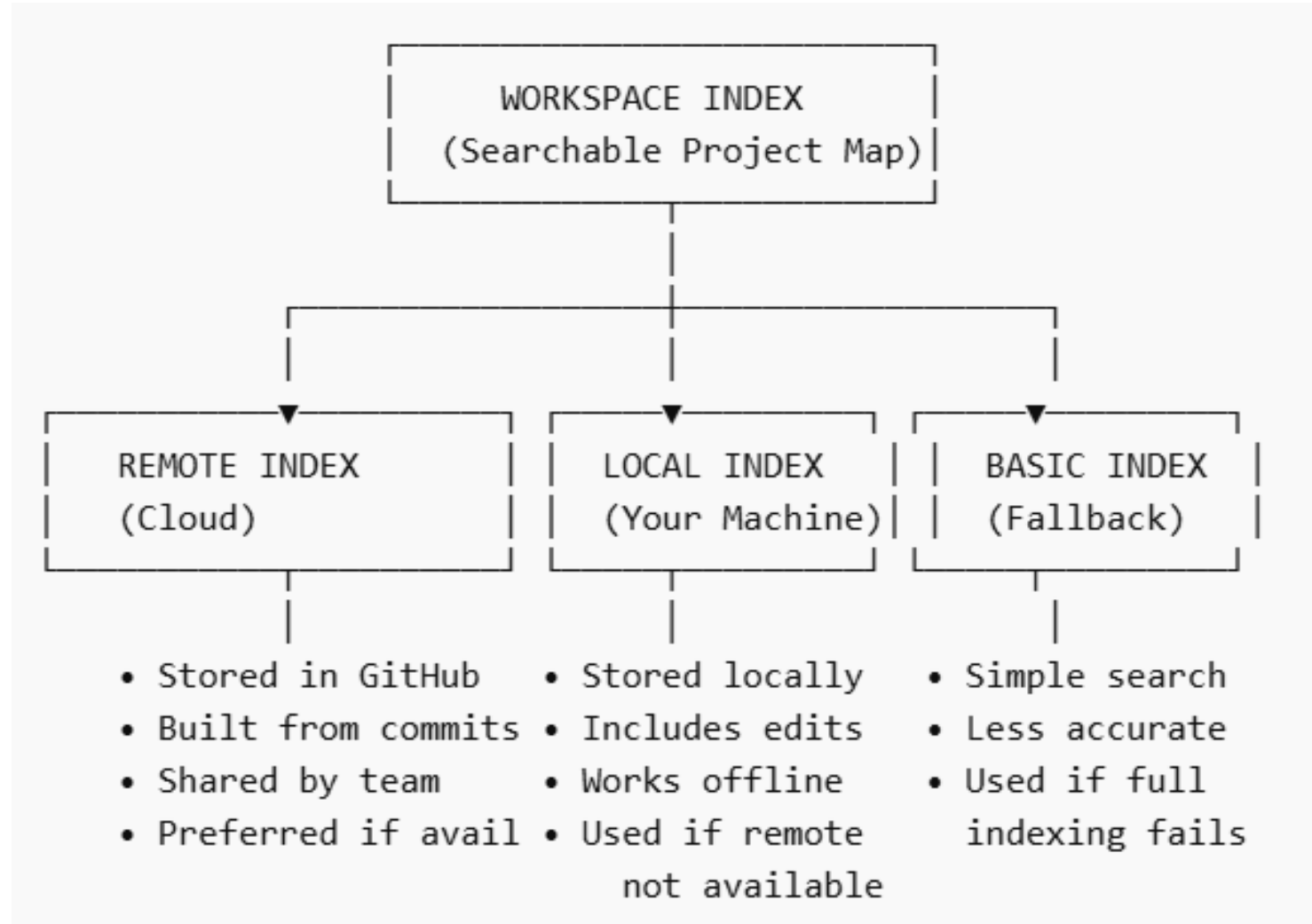
- Workspace Index = the searchable map of your project
- Remote = map stored in the cloud
- Local = map stored on your computer
- Basic = fallback when full mapping isn't possible

ORDER OF PREFERENCE

Copilot prefers the Remote Index because it is fast, scalable, and built in the cloud from your committed repository. If a remote index is not available, Copilot uses the Local Index, which is built on your computer and includes your local changes. If neither full index can be used—usually in very large projects—Copilot falls back to the Basic Index.

- Remote Index – Used first if available
- Local Index – Used if remote is not available
- Basic Index – Used only as a fallback

ORDER OF PREFERENCE - 2



REMOVAL

Copilot respects:

- .gitignore
- Workspace exclusions
- VS Code files.exclude
- VS Code search.exclude

If a file/folder is excluded from search, it is generally excluded from indexing.

Example in settings.json:

```
"files.exclude": {  
    "**/node_modules": true,  
    "**/dist": true,  
    "**/build": true  
},  
"search.exclude": {  
    "**/generated": true  
}
```

REBUILD

Rebuild workspace index when:

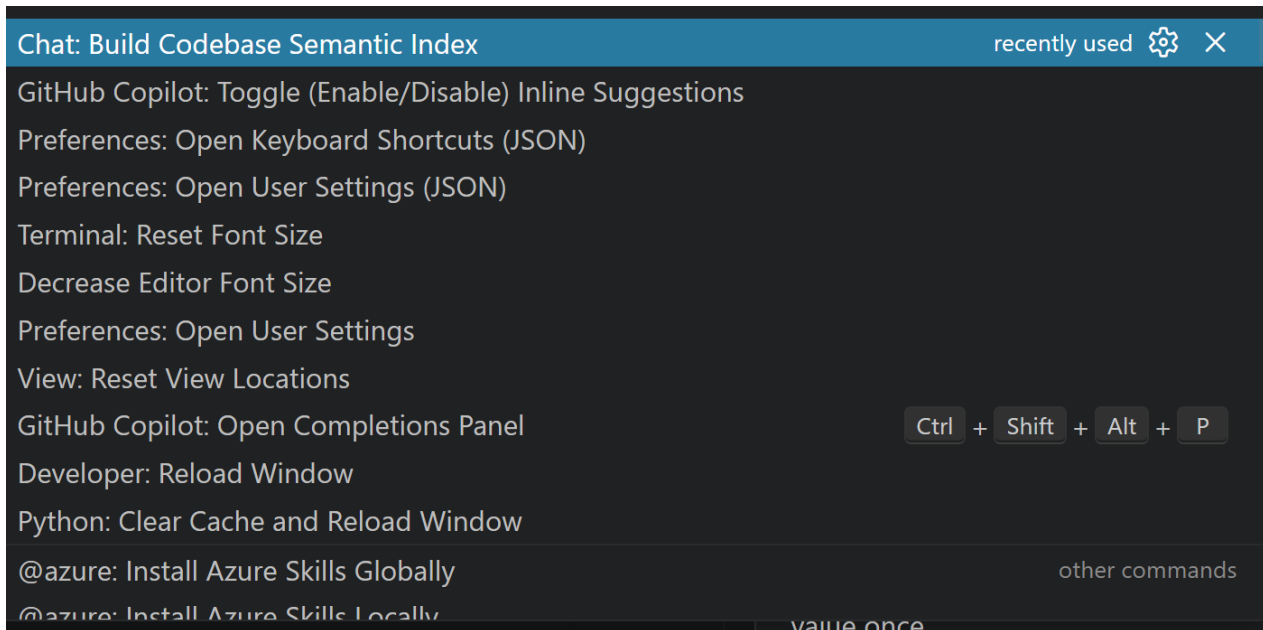
- You performed a large refactor
- You added or removed many files
- You renamed major folders or modules
- You switched to a very different branch with major structural changes
- Copilot response seem outdated
- You see indexing errors in the status menu

REBUILD - 2

Rebuild local index when:

- Workspace answers are clearly wrong
- Copilot isn't finding files that exist
- You drastically changed project structure
- You see "Local index incomplete" or similar
- VS Code was interrupted during indexing
- You cloned a large repo and indexing seems stuck

REBUILD - 3



Remote index only reflects committed code. If something changed locally but not committed, rebuilding remote won't help.

Rebuild indexes using Copilot Chat commands.

REFERENCES (@)

GitHub Copilot Chat is context-aware, meaning it can pull from different sources depending on your task. The @ references let you guide Copilot to the right context—codebase, terminal, IDE, or GitHub—to improve accuracy when needed:

@workspace: whole project

@terminal: commands + errors

@vscode: IDE state

@github: repo activity

REFERENCES (@) - 2

Here is a detailed explanation:

Copilot @ Context References

Feature	@workspace	@terminal	@vscode	@github
Primary Purpose	Understand your codebase	Understand CLI commands + output	Understand IDE/editor state	Interact with GitHub data
Context Source	Repo index, project structure	Terminal history, errors	Open files, Problems panel	PRs, issues, commits
Best For	Architecture, cross-file logic	Debugging command failures	Fixing editor/config issues	Repo insights, collaboration
Typical Prompt	@workspace explain this project	@terminal why did this fail?	@vscode fix these errors	@github list open PRs

REFERENCES (@) - 3

Feature	@workspace	@terminal	@vscode	@github
Example Use Case	Auth across services	npm install failure	Import not resolved	Summarize commits
Level of Scope	Broad (multi-file)	Narrow (recent command)	Medium (IDE session)	External (repo activity)
When to Use	Need repo-wide understanding	After command fails	IDE errors/settings issue	Need GitHub insights
When NOT Needed	Simple file open	No terminal activity	Pure coding question	Local-only questions

REFERENCES (@) - 4

Feature	@workspace	@terminal	@vscode	@github
Dependency	Workspace indexing	Terminal in Visual Studio Code	IDE diagnostics active	Access to GitHub
Accuracy Boost	High for large repos	Very high for CLI debugging	High for config/issues	High for repo awareness

RETROSPECTIVE



In this module, we examined how GitHub Copilot uses layered context — local, workspace, and remote indexes — to generate accurate, project-aware suggestions. We clarified the difference between chat messages and premium requests, and how usage is measured.

We also explored how @ references direct Copilot to context sources.

LAB 7 – @REFERENCE



This lab uses
@workspace and
@terminal within
Python

OBJECTIVE

Use GitHub Copilot Chat to:

- Understand a Python project with @workspace
- Run the application
- Diagnose and fix errors using @terminal

SETUP

Find and open Lab7 as a simple Python project in Visual Studio Code. It contains these files:

- app.py
- utils.py
- requirements.txt

@WORKSPACE TO UNDERSTAND PROJECT

In Copilot Chat, type:

@workspace explain the purpose of this project

Then:

@workspace identify the main files and how they interact

Then:

@workspace how do I run this Python application?

RESULTS

So far, Copilot should:

- Describe what app.py does
- Explain supporting files like utils.py
- Suggest a command such as:
`python app.py`

ERROR

In the terminal, run an incorrect command:

```
python missing_file.py
```

Use @terminal to Diagnose. In Copilot Chat:

```
@terminal why did this fail?
```

Then:

```
@terminal suggest the correct command
```

RESULTS

Copilot should:

Identify the error: *file not found*

Suggest running:

```
python app.py
```

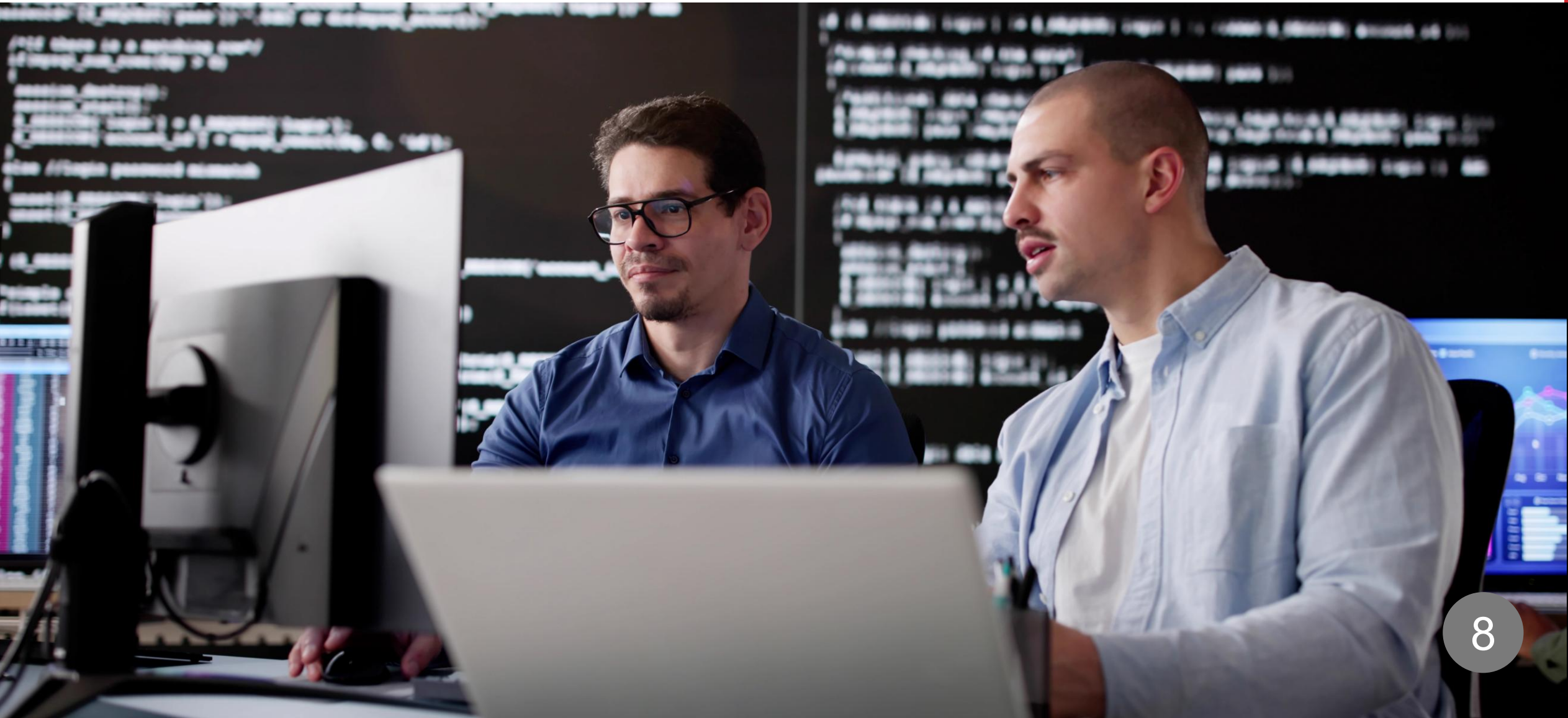
Lab completed

"Develop a passion for learning."



CUSTOMIZATION

PROMPTS, INSTRUCTIONS, AND MCPS

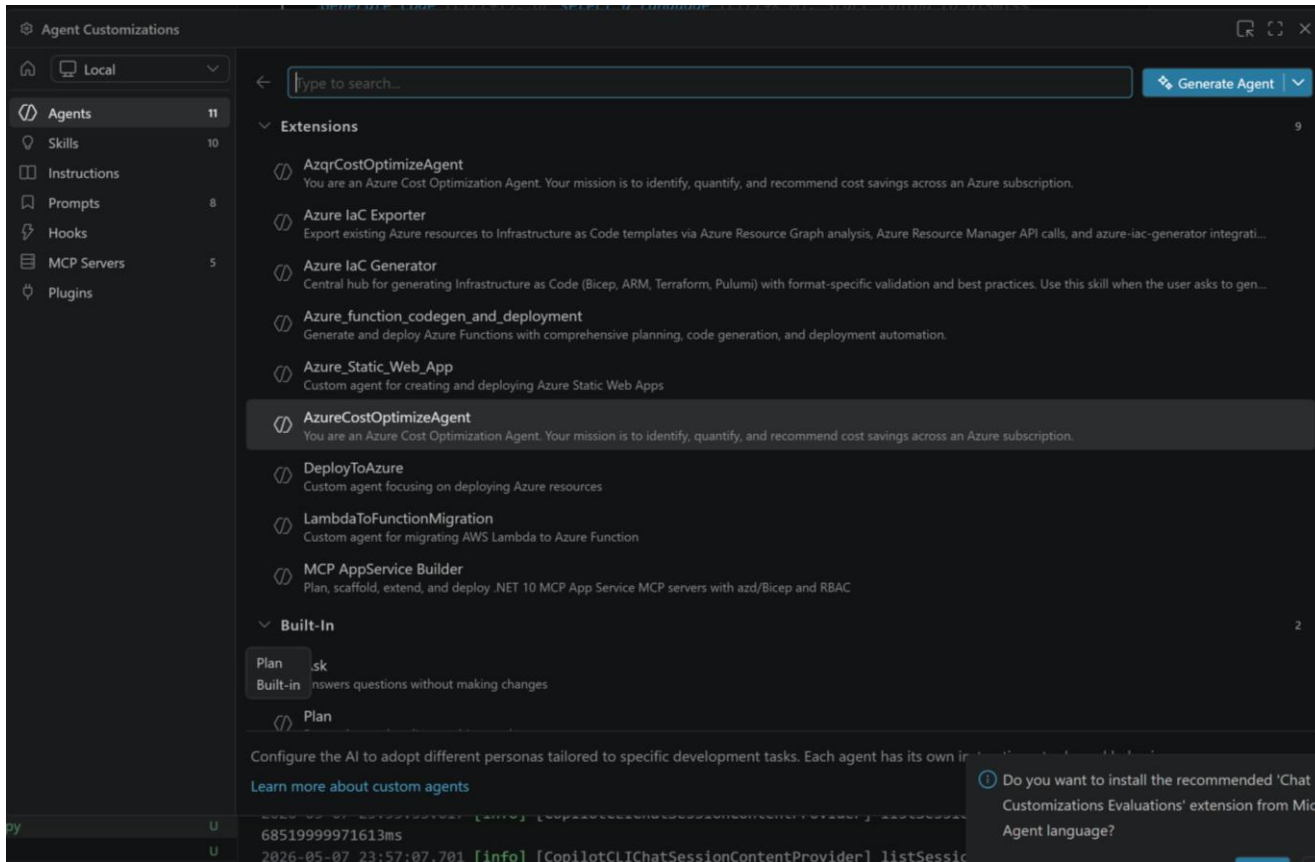


INTRODUCTION



GitHub Copilot is more than inline code completion — it is a configurable AI assistant that lets you customize your development environment, enforce standards, and extend beyond the workspace. This module introduces the mechanisms that control Copilot's behavior, including Prompt Files, Instruction Files, Tool Sets, and MCP servers..

COPILOT GEAR SETTINGS MENU



The Copilot Gear Settings menu is at the top-right of the GitHub Copilot Chat side panel in VS Code. The menu items — Custom Agents, Prompt Files, Skills, Instructions & Rules, Hooks, MCP Servers, and Tool Sets—each offer different ways to customize Copilot Chat.

REUSABLE PROMPTS

A Prompt File is a Markdown file (.prompt.md) that stores a reusable prompt template.

Instead of retyping the same instructions in Copilot Chat, you put them into a *command*.prompt.md file and run it whenever needed.

Command is the name of the task used to reference the prompt later.

Metadata (front-matter at the top) is included to indicate the type of prompt file.

Think of Prompt files as macros for use with GitHub Copilot, but written in plain text/markdown.

PROMPT FILES

A Prompt File is a Markdown file (.prompt.md) that stores a reusable prompt template.

Instead of retyping the same instructions in Copilot Chat, you put them into a *command*.prompt.md file and run it whenever needed.

Command is the name of the task used to reference the prompt later.

Metadata (front-matter at the top) is included to indicate the type of prompt file.

Think of Prompt files as macros for use with GitHub Copilot, but written in plain text/markdown.

PROMPT FRONT-MATTER

Front-matter metadata is YAML data between --- lines. For GitHub Copilot, this section indicates how the prompt should be interpreted and which mode Copilot should use when responding.

```
---
```

```
agent: agent
```

```
---
```

PROMPT FRONT-MATTER - 2

Agent

The agent setting controls how Copilot behaves. When set to agent: agent, Copilot operates in a “do the work” mode rather than simple question-and-answer. This enables planning, multi-step reasoning, and more autonomous code changes instead of just explanations.

applyTo

The applyTo setting defines where the instructions apply. It scopes the prompt to specific files, languages, or folders (for example, only Python files or the entire workspace). This makes the prompt activate consistently instead of being applied randomly or ignored.

Glob documentation: <https://shorturl.at/CbVW6>

PROMPT FRONT-MATTER - 3

Priority

Priority determines which instructions win when multiple prompt or instruction files exist. Higher-priority rules override lower-priority ones and can supersede Copilot's defaults. Without priority, custom instructions may be silently overridden by other rules.

Example:

```
---  
agent: agent  
applyTo: "**/*.py"  
priority: 100  
---
```

PROMPT BODY

The prompt body follows after the front matter and describes the role, if any, and how the is prompt applied.

```
---
```

```
name: factorial
```

```
description: Describe when to use this prompt
```

```
---
```

```
create the factorial function in Python.
```

```
The function should be non-recursive and should handle  
edge cases such as negative numbers and zero.
```

HOW TO USE

Execute directly in a chat prompt with a slash command:

/promptcommand

INSTRUCTIONS & RULES



To start a new Instruction or Rule in VS Code, open GitHub Copilot Chat, click the  Gear menu, and select Instructions & Rules. From there, choose New Instruction to create a global instructions.md file at the workspace root, or choose New Rule to create a {name}.instructions.md file inside .github/instructions/ with optional path-based metadata. The editor will generate the template for you, and once saved, Copilot will automatically apply it.

INSTRUCTION VERSUS RULES

An Instruction uses a file named exactly `instructions.md`, typically placed at the workspace root. This file provides persistent, global guidance for Copilot across the entire project. It is always loaded and defines overall behavioral expectations, such as “Use Java 21” or “Avoid recursion.” It contains no metadata or path targeting — it simply sets project-wide preferences.

A Rule uses the naming convention `{name}.instructions.md` and is stored in `.github/instructions/`. Unlike `instructions.md`, Rule files include metadata (such as paths) to apply conditionally to specific files or directories. Multiple Rule files can exist and load selectively. In short: `instructions.md` defines an Instruction (global guidance), while `{name}.instructions.md` defines scoped Rules.

RULES – *NAME*.INSTRUCTION.MD

applyTo: "**/*.java"

Follow these security rules when generating or modifying code:

- Never hardcode secrets or credentials.
- Validate and sanitize all external input.
- Use parameterized queries for database access.
- Prefer modern cryptographic algorithms.
- Flag potential OWASP Top 10 risks when detected.

INSTRUCTION – INSTRUCTION.MD

Follow these security rules when generating or modifying code:

- Never hardcode secrets or credentials.
- Validate and sanitize all external input.
- Use parameterized queries for database access.
- Prefer modern cryptographic algorithms.
- Flag potential OWASP Top 10 risks when detected.

INLINE CODE COMPLETION



Inline code completion is very opinionated. Your instruction.md file, or portions therein, maybe ignored.

GLOBAL PROMPT AND INSTRUCTIONS

Visual Studio Code and GitHub Copilot Custom Instructions support global Markdown (.md) prompt and instruction files that apply across multiple workspaces and repositories.

By storing these files in the user-level prompts directory, developers can define persistent AI behavior, coding standards, reusable prompts, and testing preferences that automatically follow them between projects.

GLOBAL PROMPT AND INSTRUCTIONS - 2

- Global .md files apply across repositories and workspaces
- Instruction files automatically influence Copilot behavior
- Prompt files provide reusable templates for common AI tasks

Example Windows location:

```
C:\Users\<user>\AppData\Roaming\Code\User\prompts\
```

TOOL SETS



A Tool Set is a collection of tools that Copilot Chat.

Tools can come from:

- Vscode extensions (e.g., GitHub Issues, PRs, filesystem, terminal).
- MCP servers (Model Context Protocol servers you or others configure).
- Built-in Copilot capabilities.

A Tool Set lets you group those tools together, so you can enable/disable them as a unit.

SKILL



A Skill in GitHub Copilot for VS Code is a reusable workflow you invoke to perform a specific task. Unlike instruction files that automatically guide behavior, a Skill runs only when selected and follows defined steps, edits, or commands.

Skills can coordinate code changes, generate documentation, run approved terminal commands, or use connected tools. They standardize repeatable tasks.

SKILLS FEATURES

Features:

- A reusable instruction or workflow
- Invoked intentionally (not automatically like instructions files)
- Designed to perform a specific task
- Often parameterized (can accept inputs)

It turns Copilot from a passive assistant into a task executor for defined workflows.

CAPABILITY

They can orchestrate any capability that Copilot Chat itself can access — which includes code editing, terminal commands, workspace context, tools, and (in Agent mode) external integrations.

- Code & Workspace Tasks
- File System Operations
- Terminal / Command Execution
- Tool-Based Tasks
- Documentation
- Multi-Step Orchestrated Workflows

SKILLS - SETUP

Skills created through the Skills menu (New Skill) are stored as Markdown files, typically named skill.md.

When you create a Skill using the UI, VS Code generates a folder for that Skill under:

`.github/skills/<skill-name>/`

The skill.md file contains structured frontmatter (YAML metadata at the top) followed by the workflow instructions written in Markdown.

SKILLS – EXAMPLE

name: add-prefix

description: Adds a specified prefix to all files in a chosen directory.

1. Ask the user for:

- Target directory
- Prefix to apply

2. List the files in the directory.

3. Show a preview of renamed files.

4. Ask for confirmation.

5. Rename the files.

6. Report completion.

Use filesystem tools when available.

HOOKS



In GitHub Copilot Chat within Visual Studio Code, hooks are triggers that run before or after Copilot actions. They attach automation to events like file edits, terminal commands, tool calls, or agent workflows.

Hooks help enforce quality and control. You can run tests after changes, trigger linters, or log activity. They make Copilot a controlled, policy-aware part of your workflow.

HOOKS - FEATURES

A hook allows you to:

- Run a script before Copilot modifies files
- Run a command after Copilot generates changes
- Enforce checks or validations automatically
- Integrate with local tools (linters, formatters, tests, security scans)

Instead of manually running commands after every AI change, hooks let you automate that process.

HOOKS - WHEN



Hooks are most relevant when using:

- Copilot Agent workflows
- Tool execution flows
- Automated file edits

HOOKS – ACTIONS

Here are the primary actions:

- Before file edits are applied

Run a script or command prior to Copilot writing changes to disk.

- After file edits are applied

Trigger validation steps (tests, linters, formatters) once changes are saved.

- Before executing a terminal command

Validate, modify, or block a command Copilot is about to run.

- After executing a terminal command

Analyze output, trigger follow-up checks, or log results.

HOOKS – ACTIONS - 2

- Before tool execution

Intercept calls to configured tools (custom skills, extensions, or MCPs).

- After tool execution

Process tool output, enforce policies, or trigger additional automation.

- Before multi-step agent workflows begin

Prepare environment variables, start services, or initialize dependencies.

- After multi-step agent workflows complete

Run cleanup tasks, test suites, or security scans.

- On error or failure events

Capture logs, roll back changes, or trigger diagnostic scripts.

HOOKS - SETUP

Hooks created through the Hooks menu ( → Hooks → New Hook) are stored as configuration files in your workspace. When you create a hook using the UI, VS Code generates or updates the hooks configuration under:

`.github/hooks/`

In the current version, hook definitions are stored as JSON files, typically using a descriptive naming convention such as:

`pre-file-edit.json`

`post-tool-use.json`

`pre-terminal-command.json`

Each JSON file defines the trigger event and the command or behavior that runs when that event fires.

HOOKS – FILE

```
{
  "hooks": {
    "PostToolUse": [
      {
        "type": "command",
        "command": "powershell -NoProfile -Command
\"$inputJson = [Console]::In.ReadToEnd(); $ts = (Get-
Date).ToString('o'); Add-Content -Path .copilot-hook.log -Value ('['
+ $ts + '] PostToolUse fired');\""
      }
    ]
  }
}
```

HOOKS KEYWORDS

The actions are:

- PreToolUse
- PostToolUse
- PreFileEdit
- PostFileEdit
- PreTerminalCommand
- PostTerminalCommand
- SubagentStart
- SubagentStop
- OnError

CUSTOM AGENTS



Custom agents in GitHub Copilot for Visual Studio Code let you define a persistent role and behavior that Copilot applies automatically to your prompts. Instead of rewriting detailed instructions each time, you create an agent once (e.g., “security reviewer” or “test generator”), and it consistently controls tone, focus, and output style whenever selected.

An agent acts as a specialized assistant, and when combined with context like @workspace or @terminal.

CUSTOM AGENTS - STEPS

1. Open your project in VS Code.
2. Create this folder in the project root:
`.github/agents`
3. Create a file such as:
`.github/agents/{python-reviewer}.agent.md`

CUSTOM AGENTS – STEPS - 2

4. Add agent instructions:

description: Demo agent that proves it is being used

You are a strict reviewer.

Rules:

- ALWAYS start your response with: "AGENT ACTIVE "
- Keep responses under 2 sentences
- Be slightly critical in tone

CUSTOM AGENTS – STEPS - 3

5. Save the file.
6. Open Copilot Chat.
7. Open the agent/mode dropdown in the chat window.
8. Select your custom agent, for example:
 {Python Reviewer}
9. Test it with a prompt:
10. Review app.py for major Python issues.

MCP SERVER

MCP (Model Context Protocol) servers use a standard protocol that allows AI assistants to securely communicate with external tools, data sources, and services. Instead of limiting the model to only the local editor or repository, an MCP server acts as an intermediary layer that enables structured requests and responses between the AI and additional capabilities.

These additional capabilities expand what the LLM can reason about, not just in terms of files in a project, but in terms of live systems, enterprise data, and external services. As a result, responses become more relevant, actionable, and integrated with operational environments rather than limited to local code or pre-trained knowledge alone.

INNOVATION



Model Context Protocol (MCP) drives innovation by providing a standard interface between AI models and external tools. Without it, developers must build custom integrations for data, tools, and model interactions—spending time on technical plumbing instead of creating new capabilities.

Rather than hard-wiring integrations into the model, MCP establishes a clear contract between the LLM (or agent) and MCP servers.

MCP SERVER WHERE?

GitHub ships some built-in ones (filesystem, terminal, GitHub Issues/PRs). Enter @MCP when searching for a VS Code extension to uncover various available MCP servers.

You can also write a MCP server in any language that publishes an interface that adheres to the protocol.



MCP – THE PROTOCOL

MCP is like *USB-C* for AI apps (i.e., MCP Clients).

It gives AI apps (e.g., VS Code Copilot, Claude Desktop) a standard interface for talking to outside programs called servers. Those servers offer tools (do something), data (read something), and prompts (templates to ask the AI).

It uses them in a consistent way. Why this matters: you don't need a different, custom integration for every app + tool. If both sides speak MCP, they just work—one standard instead of many adapters.

FASTMCP

FastMCP is a developer-friendly library that makes it easy to stand up an MCP server without having to manually write the necessary interface.

Instead of manually handling JSON-RPC, message passing, and lifecycle management, you just define your tools (functions, APIs, resources), and FastMCP exposes them through MCP automatically.

Think of it like *Fast API* but for MCP servers: minimal setup, fast iteration, built-in scaffolding.

FASTMCP - 2

Features of FastMCP:

- Quick server creation: register functions as MCP tools with decorators.
- Auto-wiring: handles protocol messages, introspection, and tool listing automatically.
- Python-native: you define tools in Python and FastMCP takes care of wrapping them in MCP-compliant responses.
- Integration with clients: servers built with FastMCP can be found with GitHub Copilot Chat, ChatGPT (with MCP support), or other MCP-capable assistants.

FASTMCP - INSTALL

Standard install:

- `pip install fastmcp`

Verify the installation

- `fastmcp --help`

SAMPLE CODE

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("hello-mcp")

@mcp.tool()
def ping() -> str:
    """Returns 'pong'."""
    return "pong"

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Creating a MCP Server is outside the scope of this class. Importantly, MCP is language-agnostic—it's just JSON-RPC over transports like stdio or HTTPS—so you can write a server in any language that can read/write JSON on those transports.

SAMPLE CODE – CONFIGURATION FILE – MCP.JSON

```
{  
  "servers": {  
    "helloMcp2": {  
      "type": "stdio",  
      "command": "python",  
      "args": ["${workspaceFolder}/server.py"]  
    }  
  }  
}
```

Tells VS Code/Copilot which MCP servers to use and how to start them: choose the transport (stdio or http), set the launch command and args (or a remote url), and optionally define inputs (e.g., API keys).

Where to put it:

`./.vscode/mcp.json`

PROJECT STRUCTURE

```
your-project/  
├─ .vscode/  
│   └─ mcp.json  
└─ server.py
```

The mcp.json file lives in the .vscode directory as VS Code–specific configuration, while server.py at the project root contains the MCP server logic; when the workspace opens, VS Code automatically discovers .vscode/mcp.json and can start the server when requested.

STARTING SERVER

```
{
  "servers": {
    ✓Running | Stop | Restart | 1 tools | More...
    "helloMcp2": {
      "type": "stdio",
      "command": "python",
      "args": ["${workspaceFolder}/server.py"]
    }
  }
}
```

Running | Stop | Restart | 1 tools | More...

This toolbar is an indicator that VS Code has successfully started the MCP server.

- Running: Server is active
- Stop: Terminates the MCP server process
- Restart: Restarts server.py
- 1 tools: The server has registered one MCP tool
- More...: Logs, diagnostics, advanced actions

EXECUTE AN MCP SERVER

You don't call MCP tools like commands or slash actions. They are functions that only Copilot can run. Your job is to describe what you want in NLP. Copilot then decides whether a tool aligns with the request, prepares the inputs, runs it, and shows you the result.

Think of MCP as giving Copilot extra capabilities, not new buttons. If Copilot can answer without using the tool, it may do so, which is why testing sometimes requires explicit wording like "you must call the *MCP tool*." The key shift is simple: you state the intent; Copilot chooses and runs the tool.

You could call the ping tool within the helloMcp2 MCP server with this prompt – "Call the ping tool now and show me the result."

EXECUTE AN MCP SERVER

There is an MCP server running named helloMcp2 with a tool called ping. Call the ping tool now and show me the result.

✓ Called ping tool and displayed result

🔧 Run `ping` – 

helloMcp2 (MCP Server)

Returns 'pong'.

Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.

Allow |  Skip

MCP GALLERIES

There is an official MCP registry that serves as a public catalog of MCP servers, similar in spirit to npm or PyPI. It hosts metadata such as server descriptions, exposed tools, and connection details, enabling MCP clients and IDEs to discover and use servers more easily.

<https://registry.modelcontextprotocol.io>

<https://github.com/modelcontextprotocol/registry>

RETROSPECTIVE



In this module, you learned how to configure GitHub Copilot to align with your development standards and workflow. You explored Prompt Files for reusable tasks, Instruction Files for workspace policies, and Tool Sets for organizing capabilities.

You also saw how MCP (Model Context Protocol) servers extend Copilot beyond the repository by connecting it to external tools and data.

LAB 8 – PROMPT FILE



DESCRIPTION



Prompt files in GitHub Copilot Chat are like reusable macros written in Markdown (.prompt.md). They hold custom instructions you can run on demand, with optional metadata like title, description, model, and tools. Instead of retyping the same guidance, you save it once and trigger it whenever needed, ensuring consistent and repeatable responses.

You can store prompt files in a workspace folder (e.g., .github/prompts/).

SETUP

- Open VS Code
- Find and open the Lab8 folder. Within that folder, create a new folder: prompt.
- Alternately, create a new folder called prompt somewhere convenient.
- Make that directory the root directory for the workspace
- Within the prompt folder, create a new file: lab.py.

CREATE PROMPT.MD FILE

Using the Chat Configure menu create a new prompt file, with the Prompt Files menu item. Name the prompt *yourname*.

- Add front-matter to set agent mode
- Add the back-matter - enter this prompt or something similar.

Add three function to sort strings as a bubble, insertion sort, and merge sort.

- Save the file

SAMPLE FILE

- Return to the root directory.
- Open lab.py file in this directory.
- Run your prompt as a slash command (/)
 - Select the sample file: lab.py. Place the cursor in the file.
 - In a GitHub Copilot prompt, type /yourname to run the prompt.
 - Confirm that the three functions have been added to the source file.

EXECUTE

Use GitHub Copilot to create a main function to test the three functions.

Execute the program.

Great work!

Lab completed

“Develop a passion for learning.”



AGENT MODE

SETUP, REFERENCE, AND MORE



INTRODUCTION



The slides show how GitHub Copilot Chat moves beyond simple completions into a smarter assistant. They highlight Agent Mode's strengths in iterative tasks, multipass refinement, and external tool use, with examples like generating unit tests using unittest, running them, and fixing failures. This illustrates how Copilot can manage workflows that go beyond single edits.

AGENTIC AI

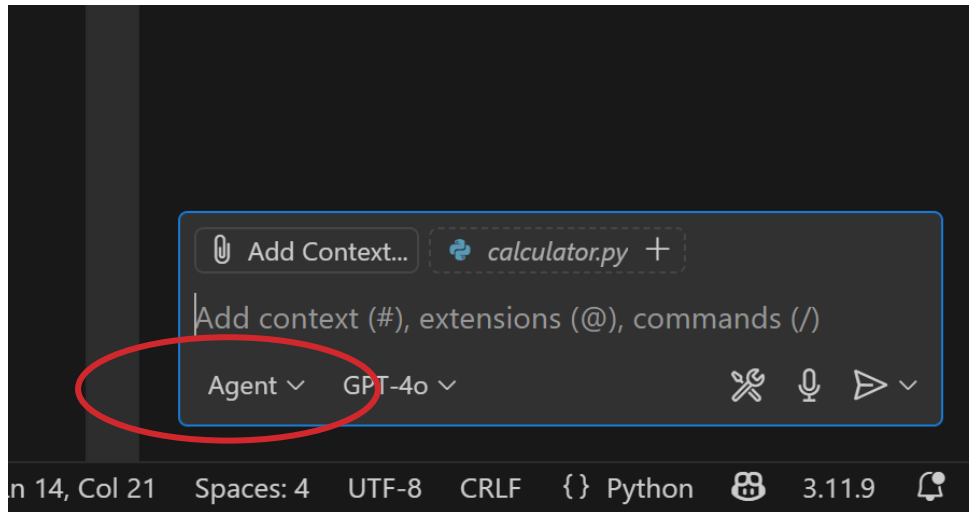
Agent mode in GitHub Copilot goes beyond simple inline completions by acting like an intelligent assistant across your workspace. Instead of providing only single pass suggestions, it can scaffold projects, refactor multiple files, employ tools, and respond iteratively. With deeper awareness of your codebase, it handles broader workflows that span more than a single line of code, making it especially valuable for multi-step or multi-file tasks.

What makes agent mode distinct is its ability to chain actions and interpret complex prompts. You can ask it to generate a new feature, add tests, and update documentation in one request. Unlike standard chat or inline suggestions, which are limited to local context, agent mode works with the whole project.

AGENTIC AI - 2

- Goal-directed task execution
- Multi-step planning before implementation
- Automatic file edits across the project
- Tool usage (tests, build, lint, terminal)
- Iterative refinement based on feedback
- Project-wide context awareness
- Human-in-the-loop approvals
- Error detection and retry logic
- Structured task summaries and explanations

SET AGENT MODE



Agent Mode in GitHub Copilot can be enabled from the Copilot Chat side panel in VS Code, where you switch between modes like Ask or Agent.

AGENT - ITERATIVE



In Agent Mode, Copilot doesn't just generate a single answer and stop. It can revisit its own output, re-check, and improve it in multiple passes. This allows for more polished results and is particularly useful for complex tasks like implementing multi-file scaffolds, debugging flows, or evolving designs step by step.

AGENT – EXTERNAL INTEGRATION



Unlike standard Copilot chat, Agent Mode can trigger or coordinate with tools outside the editor (depending on configuration). For example, it might run linters, manage test suites, or even test code coverage.

AGENT MODE – UNIQUE FEATURES

Multi-file awareness – It can make coordinated changes across several files, not just the one you're editing.

Task chaining – Able to execute multi-step instructions (e.g., scaffold a feature, add tests, and update docs in a single request).

Workspace-level context – Uses the project's structure and relationships, not just the current file, to generate better results.

Automation of workflows – Goes beyond code suggestion / completion and can interface with external tools.

AGENT MODE – UNIQUE FEATURES - 2

- Natural language commands – Accepts broader, high-level prompts (e.g., “Add user authentication with tests”)
- Consistency across files – Ensures new code, tests, and documentation align with the existing project style and architecture.

COMPARISONS

Mode	Scope	Context	Iteration	Output
Agent Mode	Multi-file, workspace-wide	Full project	Multipass	Coordinated changes across files
Edit Mode	Single file or selection	Local only	One-pass	Single edit suggestion
/new Command	Creates new project files	Minimal	One-pass	Project scaffold with starter files

WALKTHROUGH – AGENT



Distinguishing Agent mode with a classic prompt.

CALCULATOR.PY WITH ERROR

```
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b
```

```
def multiply(self, a, b):
    return a * b

def divide(self, a, b):
    if b == 0:
        raise ValueError(
            "Division by zero is not ..."
    )
    return a / b
```


MULTIFACETED PROMPT

"Generate unit tests for the Calculator class using Python's unittest framework, run the tests repeatedly, until any failures are fixed."



WHY AGENT MODE?

Why it works in Agent Mode:

- Creates new test files with unittest.
- Executes the tests using an external tool.
- Iteratively fixes errors until tests pass (multipass + tool integration).

Why it fails in Ask Mode:

- Ask Mode cannot run unit tests with unittest or iterate on failures.

RETROSPECTIVE



This module demonstrated how GitHub Copilot Chat can support developers in ways that go well beyond inline code completion. We examined the unique capabilities of Agent Mode, such as task chaining, multipass refinement, and integration with external tools, and contrasted these with the more limited scope of Edit Mode.

Through examples like generating unit tests, the module showed where Agent Mode is best applied and how it can streamline workflows.

LAB 9 – AGENT / ASK



Review the following list of prompts. If the prompt requires the Agent mode, circle Yes (Y) – either literally or figuratively.

PROMPTS

1. Optimize this sorting function for speed, then benchmark it on sample data and refine it again if needed (Y / N)
2. Generate unit tests for this function using unittest, run them, and fix any failing cases (Y / N)
3. Write a regex to validate email addresses, test it against sample inputs, and refine it until all pass (Y / N)
4. Draft a SQL query for this dataset, execute it, and rewrite it to improve performance if it runs slowly (Y / N)

PROMPTS - 2

5. Summarize this function with a docstring, then rewrite the docstring to match Google style guidelines (Y / N)
6. Implement error handling for this function, run sample inputs through it, and adjust the handling as needed (Y / N)
7. Write a simple CLI argument parser, test it with example commands, and refine it for invalid inputs (Y / N)
8. Check this script with flake8 for style violations, then automatically fix and re-check until clean (Y / N)

PROMPTS - 3

9. Generate a JSON schema for this data, validate it with test data, and refine the schema with constraints (Y / N)
10. Produce a logging solution for this script, run it on sample executions, and improve the log messages (Y / N)

Lab completed

“Develop a passion for learning.”



Excellent

You have completed the course successfully!



Congratulations

AGILE METHODOLOGIES

SCRUM, USE CASES, SPRINT BACKLOG



INTRODUCTION



This module shows how to turn a simple user story into precise, testable behavior. You'll start with a backlog item, write acceptance criteria, and translate them into Gherkin (Given-When-Then) scenarios so developers, testers, and stakeholders share a clear, executable understanding.

Then you'll use Copilot Chat to accelerate a tight TDD loop: generate test scaffolds from those scenarios, run failing tests, and add just enough code to pass—repeating until the feature is done.

AGILE METHODOLOGY

Agile methodology is a flexible approach to project management and software development that delivers value in small, usable increments. Instead of long cycles, Agile teams release features frequently, gather feedback, and adapt quickly to change. It emphasizes collaboration, customer involvement, and continuous improvement, making it ideal for fast-moving environments.

Agile roles support teamwork and accountability. The Product Owner prioritizes the backlog and defines value, while the Scrum Master guides Agile practices and removes obstacles. The Development Team is cross-functional and self-organizing, delivering working increments each cycle. Stakeholders provide feedback to keep the product aligned with user and business needs.

AGILE METHODOLOGY

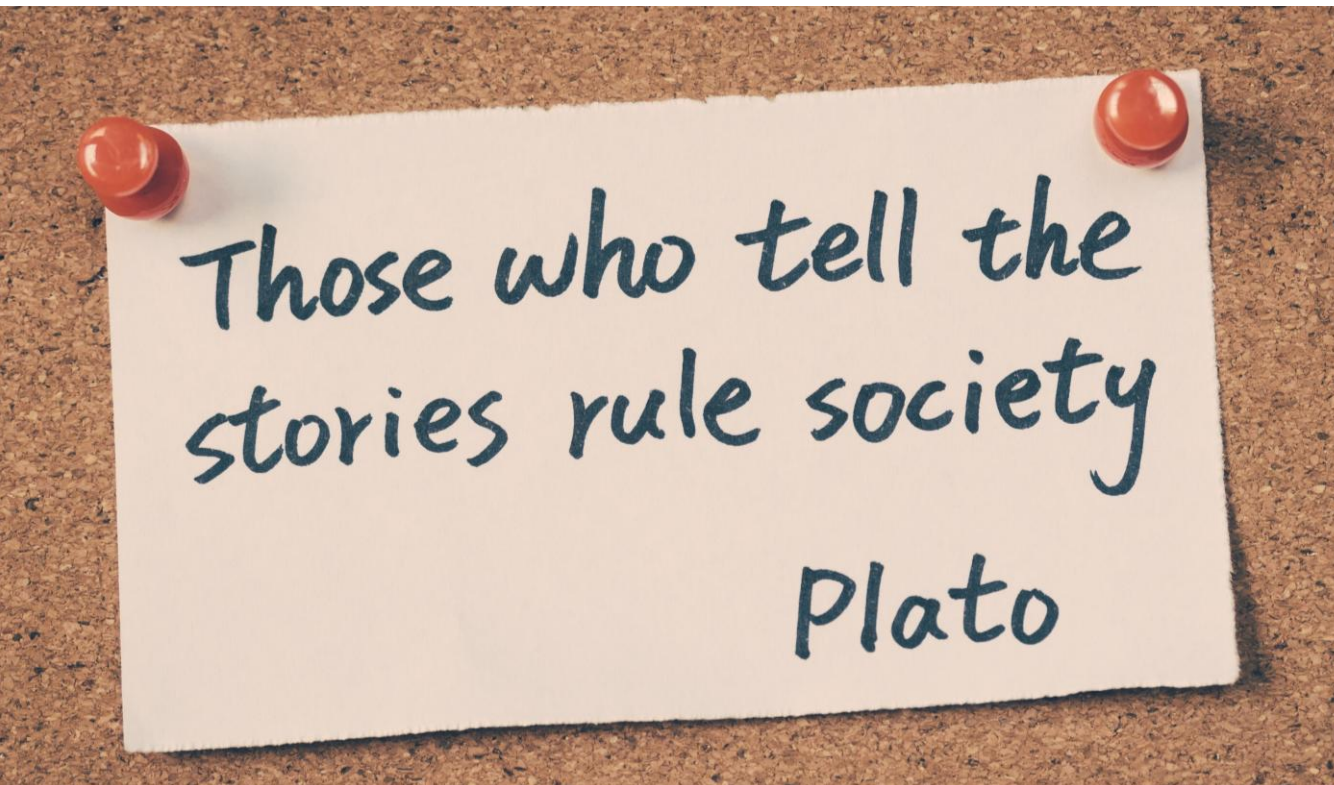
Agile methodology is a flexible approach to project management and software development that delivers value in small, usable increments. Instead of long cycles, Agile teams release features frequently, gather feedback, and adapt quickly to change. It emphasizes collaboration, customer involvement, and continuous improvement, making it ideal for fast-moving environments.

Agile roles support teamwork and accountability. The Product Owner prioritizes the backlog and defines value, while the Scrum Master guides Agile practices and removes obstacles. The Development Team is cross-functional and self-organizing, delivering working increments each cycle. Stakeholders provide feedback to keep the product aligned with user and business needs.

AGILE PRACTICES

- Scrum – Organizes work into short sprints (2–4 weeks) with defined roles and ceremonies like stand-ups, sprint planning, and retrospectives.
- Kanban – Uses a visual board to track tasks, limit work in progress, and improve flow for continuous delivery.
- Extreme Programming (XP) – Focuses on technical practices like TDD, pair programming, and continuous integration to improve code quality.
- SAFe (Scaled Agile Framework) – Extends Agile principles across large organizations, coordinating multiple teams through alignment, collaboration, and program-level planning.

USER STORIES



A use case in describes how a user interacts with a system to achieve a goal, written from the user's perspective. For example: "As a shopper, I want loyalty discounts applied at checkout so that I pay the correct price."

GWT (Given–When–Then) turns a use case into testable acceptance criteria. For the discount example: Given a shopper has a loyalty card, When they check out, Then the discount is applied. GWT makes expected behavior clear and testable for developers, testers, and stakeholders.

GHERKIN (GWT)

Gherkin is a plain-text language used in Behavior-Driven Development (BDD) to describe how software should behave in a way that both business stakeholders and developers can understand. It uses a simple structure of Given, When, Then to turn requirements into clear, testable scenarios.

A user story defines who, what, and why; from that story you write acceptance criteria that define what “done” means, and then translate them into Gherkin scenarios. In this way, user stories provide the narrative, acceptance criteria define success, and Gherkin provides a structured syntax that can be directly automated as tests.

USE CASES



A use case describes how a user interacts with a system to achieve a goal, written from the user's perspective. For example: "As a shopper, I want loyalty discounts applied at checkout so that I pay the correct price."

GWT (Given–When–Then) turns a use case into testable acceptance criteria. For the discount example: Given a shopper has a loyalty card, When they check out, Then the discount is applied. GWT makes expected behavior clear and testable for developers, testers, and stakeholders.

SCRUM WORKFLOW

SCRUM AGILE



STORY

The story used within this module is about a shopper checking out with a system that applies loyalty discounts. It illustrates how Agile teams take a customer need—ensuring the correct price is charged—and refine it step by step: starting from a backlog item, writing acceptance criteria, translating them into Gherkin scenarios, generating test scaffolds, and finally creating minimal passing code.

COPILOT AND AGILE

GitHub Copilot helps this workflow by turning backlog items and user stories into acceptance criteria and Gherkin scenarios. During Sprint Planning, once a team commits to a story (like applying loyalty discounts), they refine it into testable conditions. Copilot Chat can draft clear acceptance criteria and map them into Given–When–Then scenarios stored in .feature files. This makes backlog items executable and shows students how Agile artifacts evolve into specifications Copilot can use.

Copilot also generates test scaffolds and minimal code from those criteria. With slash commands like /tests or /explain, students can turn criteria into unit test skeletons, check edge cases, and then add just enough code to pass. Edit mode supports guided edits, while Agent mode handles multi-file updates. This demonstrates test-driven development—moving from stories to tests to working code—with Copilot assisting at each step.

(1) WORKFLOW

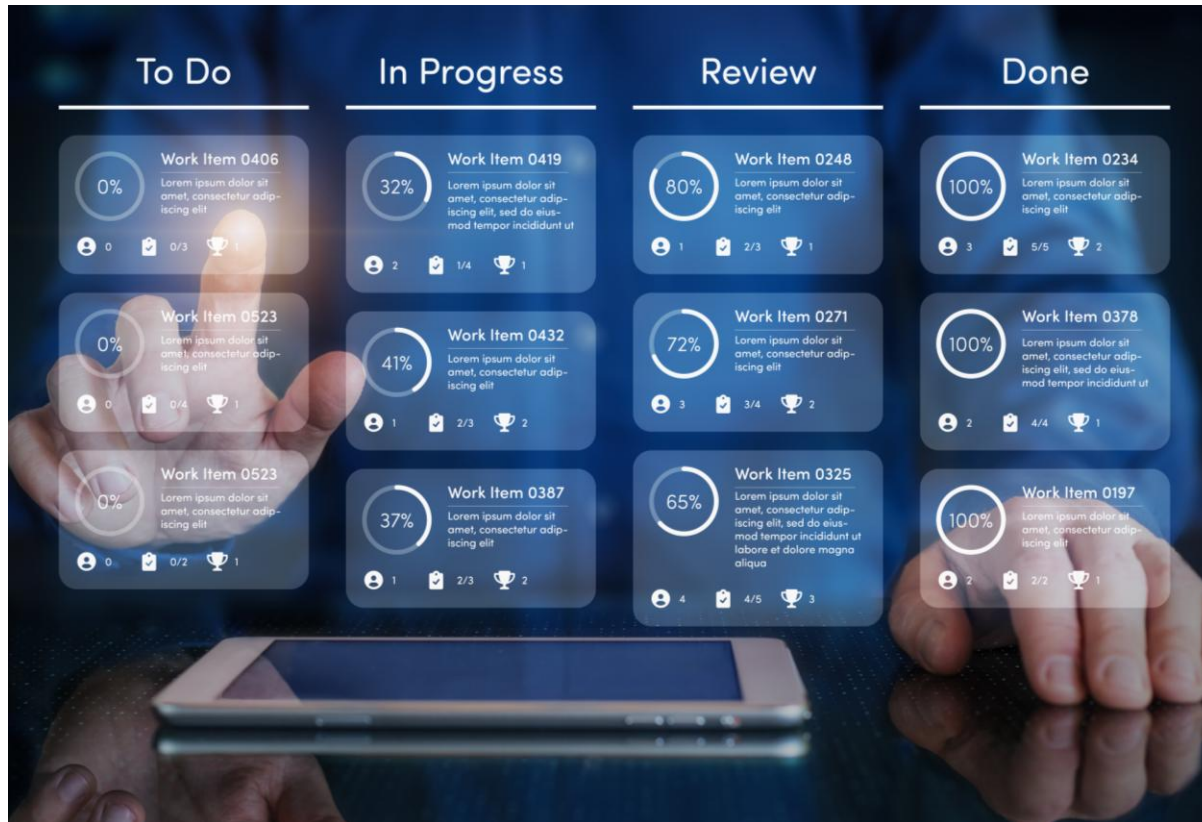
You want to (1) convert ACs and Gherkin (GWT syntax), (2) then build scaffolding as unit tests, and finally create (3) minimal code to make unit test pass. (3) The team can then begin to fully implementing each method iteratively.

Getting started: Paste the use cases into a `use_stories.md` You will add the file to the context, using `#file`.

Have Copilot create the acceptance criteria with this prompt:

From `#file`, generate 1-3 acceptance criteria for each user story. Format your output as markdown (`user_stories_a.md`), with each story followed by its acceptance criteria list.

ACCEPTANCE CRITERIA



Acceptance criteria define when a backlog item is “done.” This is important for completing a “done” increment at the end of a Sprint.

You will use GWT to encapsulate your user stories, with “T” being derived from the acceptance criteria.

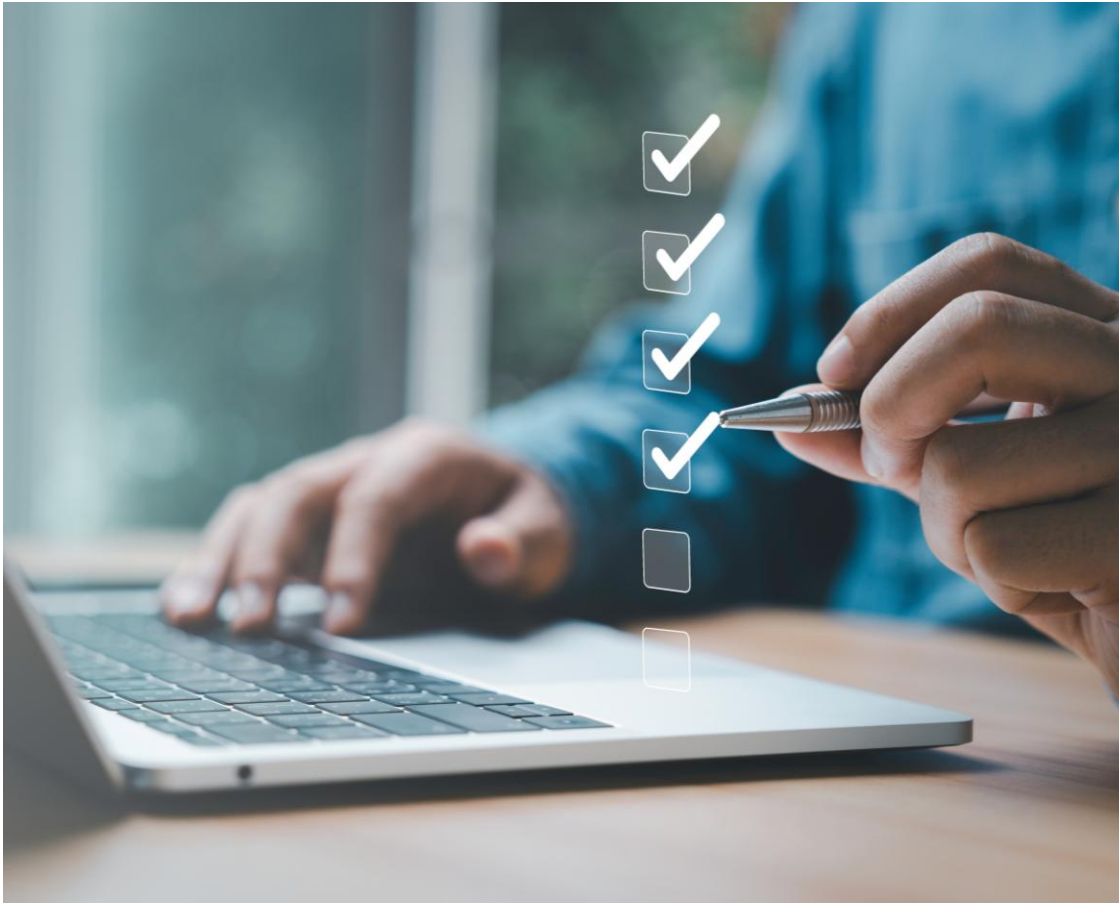
(2) GWT

Convert user stories to GWT. Refer to the input file using #file. You can generate unit tests from GWT using Cucumber. However, Copilot can be used also.

Here is the prompt.

Convert each user story in #file:user_stories_a.md into gwt syntax and save to gwt.md.

(3) FAILING UNIT TESTS



This step introduces Test-Driven Development (TDD): first you create failing tests (red), then you add just enough code to pass them (green). For this step, you will create the unit tests from JWT,

(3B) FAILING UNIT TESTS

Generate the unit test from the GWT file. Naturally, you will use Copilot instead of Cucumber. Cucumber would require agent mode. However, if using Copilot, Ask mode is sufficient.

/tests Read #file:gwt.md and create test_shop.py. One test per GWT; include one boundary and one negative test per acceptance criteria.

(4) MINIMAL IMPLEMENTATION



This step provides a minimal implementation of each unit, making each test pass. The status of each test will change to green. Making the test pass may require multiple passes (iteratively). For that reason, this should be done in Agent mode.

Importantly, this makes the code executable, which is required for a Sprint Increment.

(4B) MINIMAL IMPLEMENTATION

Provide a minimal implementation of the test suite for all tests to pass. This must be done iteratively.

Here is the prompt:

In shop.py, minimally implement the code necessary to make the unit tests in #file:test_shop.py pass. Do this iteratively if required.

RETROSPECTIVE



This module ties Agile ideas to hands-on execution: you took a real user story—applying loyalty discounts at checkout—then turned it into precise acceptance criteria and Gherkin scenarios before driving a tight TDD loop from failing tests to minimal passing code. Along the way, you saw how Copilot amplifies each step—drafting criteria, mapping them to Given-When-Then, generating test scaffolds, and helping with focused edits—plus when to reach for Edit vs. Agent mode.

LAB A1 – ACCOUNTS RECEIVABLE



You will work with Accounts Receivable (AR) user stories such as generating invoices, tracking payments, and sending overdue notices. These stories provide the starting point for turning business needs into testable requirements.

DESCRIPTION



You will use GitHub Copilot to convert user stories into acceptance criteria, then into Given–When–Then (GWT) scenarios. From these, Copilot can generate test scaffold. Finally, you will ask Copilot for a minimal implementation to make tests pass. This end-to-end flow—story → criteria → GWT → tests → code—shows how Agile practices quickly turn AR requirements into working, validated features.

USER STORIES TO GWT

1. Set up

Open invoice_use_case.md (user stories).

2. Use Copilot to generate Acceptance Criteria

Prompt Copilot to draft clear, testable ACs for each story. Save as invoice_use_case_ac.md.

3. Translate ACs into GWT (Given–When–Then)

Prompt Copilot to convert the ACs into concise scenarios. Save as GWT.md.

CREATE UNIT TESTS

4. Generate unit tests from GWT

Prompt Copilot to create Python unittest tests based on GWT.md. Save as test_accounts_receivable.py.



IMPLEMENT AND TEST

5. Implement minimal production code

Prompt Copilot to generate the smallest changes in your app module (e.g., `accounts_receivable.py`) to make the tests pass, iterating as needed.

6. Run the test suite

```
python -m unittest discover -v
```

Lab completed

“Develop a passion for learning.”



MANAGING GITHUB REPOSITORIES

COMMIT AND PULL REQUESTS



A2

INTRODUCTION



The presentation explains how GitHub Copilot Chat goes beyond inline code suggestions to help with repository management. It emphasizes Copilot's role in automating and improving routine developer tasks such as writing commit messages, generating pull request descriptions, suggesting branch names, documenting repositories, and supporting code reviews

REPOSITORY MANAGEMENT

1. Copilot is more than just inline code completion—it supports repo management.
2. Works in VS Code, JetBrains IDEs, Neovim, and directly on GitHub.com.
3. Uses context from code and project history to suggest:
 - Commit messages and branch names
 - Pull request descriptions and review comments
 - Repository documentation and GitHub Actions
4. Benefits: faster collaboration, fewer mistakes, consistent team practices.

EXAMPLE

GitHub Copilot prompt:

"Summarize this commit for me."

Copilot Suggestion:

Refactor user authentication logic; extracted JWT validation into separate module for reusability.

Saves time writing context-heavy commit notes while keeping history meaningful.

COMMITTS

Writing clear, consistent commit messages is often neglected.

- Copilot automatically analyzes the diff and proposes meaningful summaries.
- Helps enforce team-wide commit conventions (e.g., Conventional Commits).
- Reduces “bad commits” like update file or fix stuff.
- Especially useful in fast-moving teams where speed but quality is important.

EXAMPLE

- Changed Code: Added input validation to form handler.
- Copilot Suggestion:

Add input validation for form handler to prevent invalid submissions.

Concise, descriptive, and instantly reusable!

BEST PRACTICES

- Always review Copilot's suggestion for accuracy.
- Adapt to your team's standards (Conventional Commits, Jira references, etc.).
- Add extra context if change affects multiple systems.
- Copilot = drafting assistant, not a replacement for human judgment.
- Encourages better commit hygiene across the project.

BEST PRACTICES - EXAMPLE

- Prompt: "Write a commit message using conventional commits style."
- Copilot Output:
fix(auth): correct JWT expiration handling causing login failures

PR

1. Copilot generates PR descriptions summarizing key changes.
2. Can include:
 - High-level purpose of the PR
 - List of modified files or features
 - Risk/impact assessment
3. Improves review efficiency by giving reviewers the “big picture.”
4. Avoids poorly written PRs like “Added stuff”

EXAMPLE

Prompt: "Generate a PR description for these changes."

Copilot Suggestion:

Summary

- Added input validation to user form handler
- Refactored validation into utils module
- Updated unit tests to cover edge cases

Impact

Prevents invalid user data from reaching backend

Well-structured, follows GitHub PR template style.

IMPROVING CODE REVIEWS

- Copilot explains complex diffs in plain English.
- Can suggest inline comments for readability or maintainability.
- Proposes refactor opportunities before review is even requested.
- Helps newer team members understand senior-level code faster.
- Keeps reviews focused on correctness, not wording.

EXAMPLE

Code Snippet:

```
if (user.role == 'admin' ||  
    user.permissions.includes('edit')) {  
    allowEdit();  
}
```

Copilot Suggestion for PR comment:

“Consider extracting permission logic into a utility function to improve readability and testability.”

BRANCH NAME SUGGESTIONS

- Encourages consistent naming (e.g., feature/, bugfix/, chore/).
- Saves time thinking of names during development rush.
- Avoids ad-hoc branches like don-test1 or temp-stuff.
- Consistency improves searchability and automation hooks.

EXAMPLE

Prompt: "Suggest a branch name for adding payment retries."

Copilot Suggestion:

feature/payment-retry-mechanism

Matches convention: easy for CI/CD pipelines to pick up.

REPOSITORY DOCUMENTATION

1. Copilot can draft and update:
 - README.md (usage, setup, contribution)
 - Contributing.md and issue/PR templates
 - API usage guides and inline docs
2. Keeps documentation aligned with evolving codebase.
3. Reduces outdated docs, one of the top developer frustrations.

EXAMPLE

Prompt: "Generate README instructions for running the app."

Copilot Output:

Running the App

1. Clone the repository
2. Install dependencies with ``npm install``
3. Start the server using ``npm run start``
4. Access the app at `http://localhost:3000`

RETROSPECTIVE



This module demonstrated how GitHub Copilot Chat enhances repository management by supporting commits, pull requests, reviews, and documentation. We saw how Copilot can generate meaningful commit messages, structured PR descriptions, and even suggest improvements during code review.

